3

LIFE CYCLE ANALYSES AND METHODS

3.1 QUALITY CHARACTERISTICS AND APPROACHES

3.1.1 Introduction to Quality Characteristics

ISO/IEC/IEEE 15288 (2023), Section 3.36 defines *Quality Characteristic* (QC) as: *inherent characteristic of a product, process, or system related to a requirement*. QCs are how the stakeholders will judge the quality of the system. Approaches exist that help ensure these characteristics are present in the SoI and its broader context or environment.

The objective of the following sections is to give enough information to a Systems Engineering (SE) practitioner to appreciate the significance of various QC approaches, even if they are not an expert in the subject. In previous editions of the handbook, the QC approaches were known as Specialty Engineering or the Engineering Specialties. These approaches are also known as Design for X (DFX) and Through-Life Considerations. The QCs are informally known as the -ilities since many, but not all, end in "ility" in the English language.

QC approaches, as used in this handbook, are life cycle perspectives that need to be considered to ensure the system is developed and its ecosystem cultivated so that QCs are present when the system is produced, utilized, supported, and ultimately retired. QC approaches often generate non-functional requirements. Some QC approaches, such as safety, security, and resilience may also generate functional requirements. These QC approaches are applied throughout the system's life cycle, as notionally shown in Figure 3.1. Consideration beyond the engineered system, including the system, SoS, or enterprise that it is a part of, and its interoperating and enabling systems, is also necessary.

The QC approaches in this section are covered in alphabetical order by name to avoid giving more weight to one over another. Table 3.1 summarizes the QC approaches included in the handbook. Not every QC approach will be applicable to every system or every application domain. It is recommended that subject matter experts are consulted and assigned as appropriate to conduct QC approaches. More information about the QC approaches can be found in references to external sources.

INCOSE Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities, Fifth Edition.
Edited by David D. Walden, Thomas M. Shortell, Garry J. Roedler, Bernardo A. Delicado, Odile Mornas, Yip Yew-Seng, and David Endler.

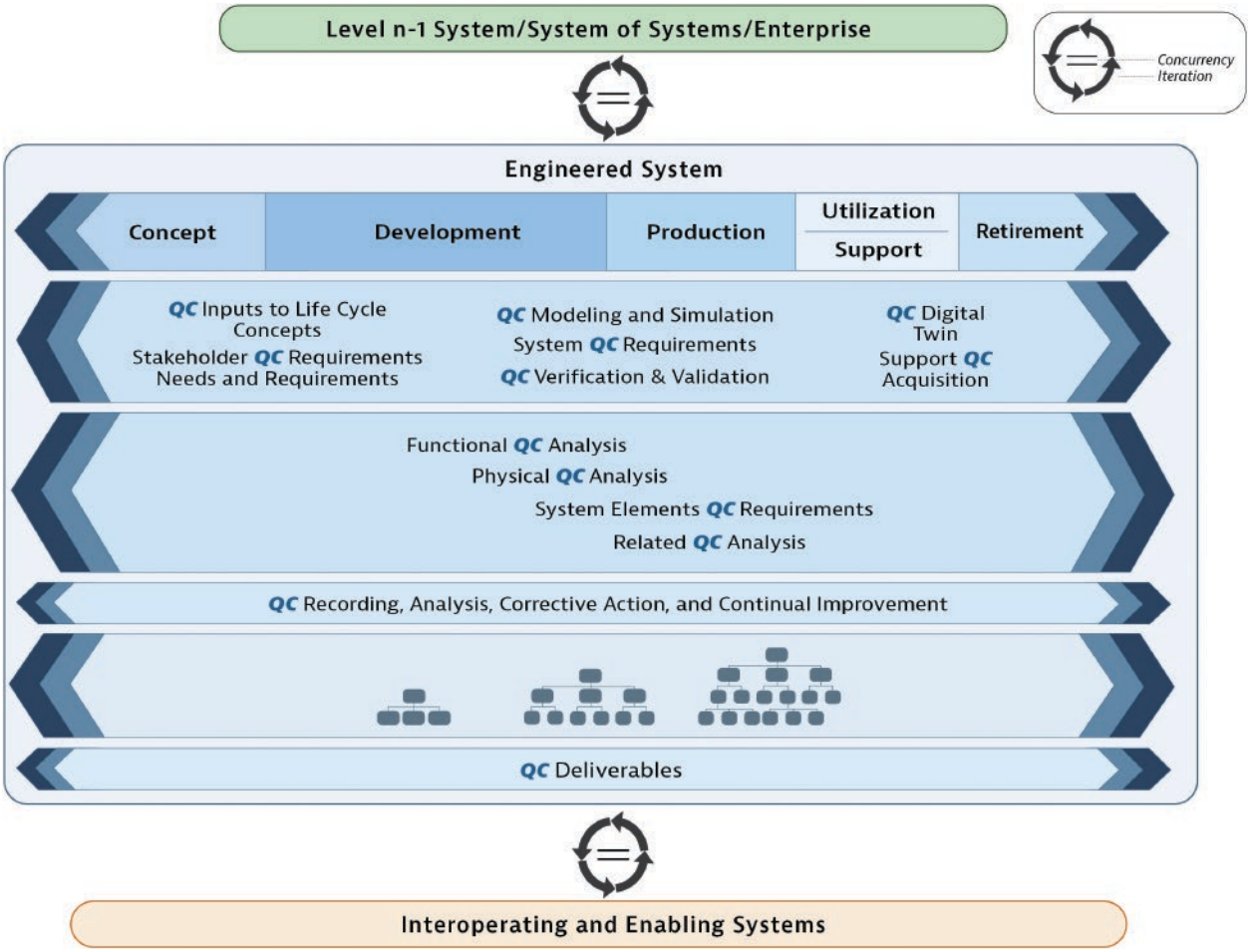


FIGURE 3.1 Quality characteristic approaches across the life cycle. INCOSE SEH original figure created by Taljaard, Kemp, and Walden. Usage per the INCOSE Notices page. All other rights reserved.

This handbook includes a set of QC approaches that are generally applicable in various applications and domains. However, the SE practitioner should ensure that any additional applicable QC approaches are also addressed.

3.1.2 Affordability Analysis

Definition Affordability Analysis is an approach that maximizes value, providing cost-effective capability over the entire life cycle.

INCOSE has defined system affordability as follows:

> Affordability is the balance of system performance, cost, and schedule constraints over the system life while satisfying mission needs in concert with strategic investment and organizational needs.

Key Concepts As stated in Blanchard and Fabrycky (2011),

Many systems are planned, designed, produced, and operated with little *initial* concern for affordability and the total cost of the system over its intended lifecycle… The technical [*aspects* are] usually considered first, with the economic [aspects] deferred until later.

TABLE 3.1 Quality Characteristic approaches

QC approach	An approach that …	Representative QCs
Affordability Analysis	maximizes value, providing cost effective capability over the entire life cycle	Affordability, Cost-Effectiveness, Life Cycle Cost (LCC), Value Robustness
Agility Engineering	enables change in a timely and cost-effective manner	Adaptability, Agility, Changeability, Evolvability, Extensibility, Flexibility, Modularity, Reconfigurability, Scalability
Human Systems Integration	integrates technology, organizations, and people effectively	Desirability, Ergonomics, Habitability, Human Factors, Human-Computer Interaction (HCI), Human-Machine Interface (HMI), Usability, User Interface (UI). User eXperience (UX)
Interoperability Analysis	ensures the system interacts effectively with other systems	Compatibility, Connectivity, Interoperability
Logistics Engineering	enables support for the entire life cycle	Supportability
Manufacturability/ Producibility Analysis	enables production in a responsible and cost effective manner	Manufacturability, Producibility
Reliability, Availability, Maintainability Engineering	enables the system to perform without failure, to be operational when needed, and to be retained in or restored to a required functional state	Accessibility, Availability, Interchangeability, Maintainability, Reliability, Repairability, Testability
Resilience Engineering	provides required capability when facing adversity	Resilience, Robustness, Survivability
Sustainability Engineering	supports the circular economy over its life	Disposability, Environmental Impact, Sustainability
System Safety Engineering	reduces the likelihood of harm to people, assets, and the wider environment	Safety
System Security Engineering	identifies, protects from, detects, responds to, and recovers from anomalous and disruptive events, including those in a cyber contested environment	Cybersecurity, Information Assurance (IA), Physical Security, Trustworthiness

This section addresses economic and cost factors under the general topics of affordability and cost-effectiveness. The concept of life cycle cost (LCC) is also discussed. Improving design methods for affordability is critical for all application domains (Bobinis, et al., 2013; Tuttle and Bobinis, 2013). Case 4 (Design for Maintainability-Incubators) from Section 6.4 provides an illustration of its importance.

A system is "affordable" if it can be developed to meet its requirements within cost and schedule constraints. The concept can seem straightforward. The difficulty arises when an attempt is made to specify and quantify the affordability of a system. This is significant when writing requirements or when comparing two solutions to conduct an affordability trade study. Affordability analysis is contextually sensitive, often leading to a misunderstanding and incompatible perspectives on what an "affordable system *is*."

Key affordability concepts include:

- Affordability context, system(s), and portfolios (of systems capabilities) need to be consistently defined and included in any understanding of what an affordable system is.
- An affordability process/framework needs to be established and documented.
- Accountability (system governance) for affordability needs to be assigned across the life cycle, which includes stakeholders from the various contextual domains.

Affordability costs include acquisition, operating, and support costs. It may be expanded to encompass additional elements required for the Life Cycle Cost (LCC) of a system, as an outcome of various contexts in which any system is embedded. In the SE domain, affordability as an attribute must be determined both inside the boundaries of the system of interest (SoI) and outside. The concept of affordability must encompass everything from a portfolio (e.g., family of automobiles) to an individual project (specific car model).

An affordability design model must be able to provide the ability to effectively manage and evolve systems over long life cycles. One of the major assumptions for measuring the affordability of competing systems is that given two systems, which produce similar output capabilities, it will be the *nonfunctional* attributes of those systems that differentiate system value to its stakeholders. As shown in Figure 3.2, the affordability model is concerned with operational attributes of systems that determine their value and effectiveness over time, typically expressed as the system's quality characteristics as they are called in this handbook. These attributes are properties of the system as a whole and as such represent the salient features of the system and are measures of the ability of the system to deliver the capabilities it was designed for over time.

Managing a system within an affordability trade space means that we are concerned with the actual performance of the fielded system, defined in one or more appropriate metrics, bounded by cost over time. The time dimension extends a specific "point analysis" (static) to a continuous life cycle perspective (dynamic). Quantifying a relationship between cost, performance, and time defines a functional space that can be graphed and analyzed mathematically. Then it becomes possible to examine how the output (e.g., performance, availability, capability) changes due to changes in the input (e.g., cost constraints, budget availability). This functional relationship between cost and outcome defines an affordability trade space to analyze the relationship between money spent and system performance and possibly determine the point of diminishing returns. This is illustrated in Figure 3.3. The capabilities and schedule have been fixed leaving either the cost or the performance to be the evaluation criteria, while the other becomes the constraint. This results in a relatively simple relationship between performance and cost. The maximum budget and the minimum performance are identified.

Below the maximum budget line in Figure 3.3 lie solutions that meet the definition of "conducting a project at a cost constrained by the maximum resources." The solutions to the right of the minimum performance line satisfy the threshold requirement. Thus, in the shaded rectangle lie the solutions to be considered since they meet the minimum performance and are less than the maximum budget. On the curve lay the solutions that are the "best value," in the sense that for a given cost the corresponding point on the curve is the maximum performance that can be achieved. *In actuality, the curve is rarely smooth or continuous and multiple curves need to be considered simultaneously*. Similarly, for a given performance, the corresponding point on the curve is the minimum cost for which that performance can be

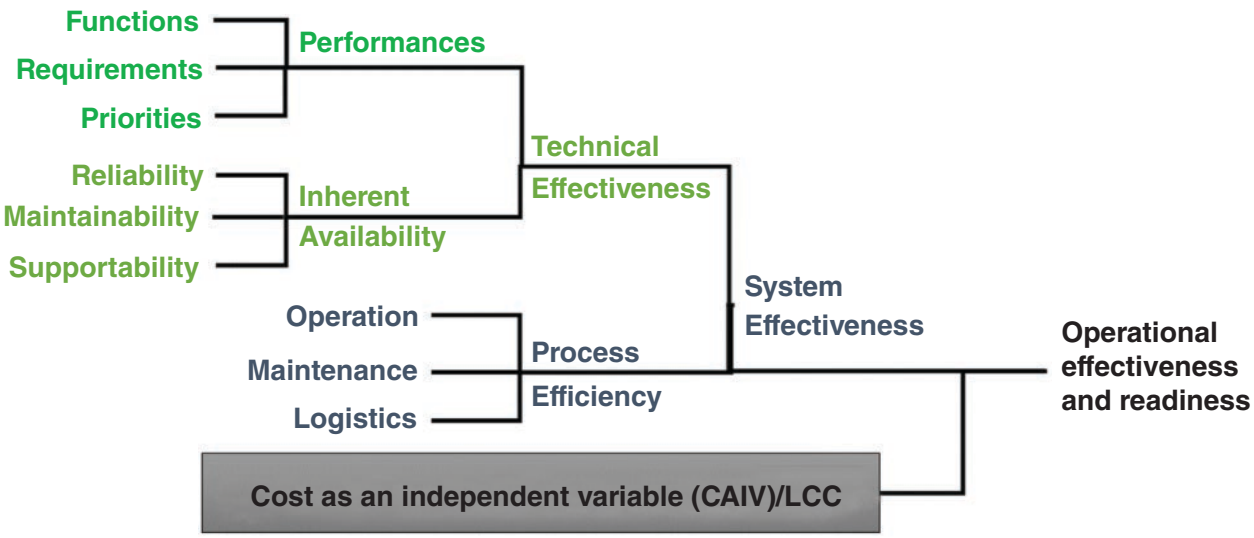


FIGURE 3.2 System operational effectiveness. From Bobinis et al. (2013). Used with permission. All other rights reserved.

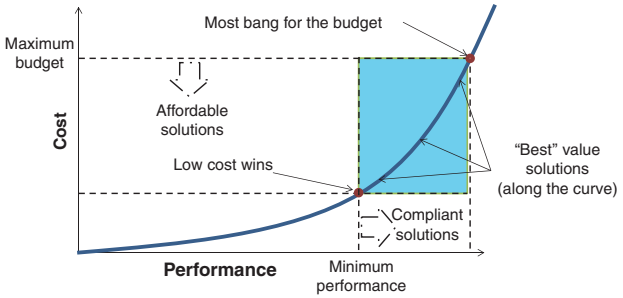


FIGURE 3.3 Cost versus performance. INCOSE SEH original figure created by Bobinis on behalf of the INCOSE Affordability Working Group. Usage per the INCOSE Notices page. All other rights reserved.

achieved. Selecting the decision criterion as cost will result in achieving the threshold performance. If the decision criterion is performance, all of the budget would be expended. Consequently, to specify affordability for a system or project requires determining which affordability element is the basis for the decision criteria and which elements are being specified as constraints.

Affordability is the result of a disciplined decision-making process requiring systematic methodologies that support selection of the most affordable technologies and systems.

Elaboration

Cost-Effectiveness Analysis

Cost-effectiveness (CE) is a measure relating cost to system effectiveness. It is defined below with the achieved systems effectiveness as the numerator and cost as the denominator (Blanchard, 1967):

$$CE = SE/(IC + SC)$$

Where SE = System Effectiveness, IC = initial cost and SC = sustainment cost.

Reliability and maintainability are major factors in determining the cost effectiveness of a system since they impact sustainment costs.

System effectiveness is a term used in a broad context to reflect the technical characteristics of a system (e.g., performance, availability, supportability, dependability) such as examples mentioned in the preceding section. It may be expressed differently depending on the specific application. Sometimes a single-figure of merit is used to express system effectiveness and sometimes multiple figures-of-merit are employed (Blanchard and Fabrycky, 2011). The IC and SC can also be expressed in different ways depending on the application or system parameters under evaluation. It may include costs for concept, development, production, utilization, support, and retirement.

Cost-Effectiveness Analysis (CEA) is distinct from cost–benefit analysis (CBA). The approach to measuring costs is similar for both techniques, but in contrast to CEA where the results are measured in performance terms, CBA uses monetary measures of outcomes. This approach has the advantage of being able to compare the costs and benefits in monetary values for each alternative to see if the benefits exceed the costs. It also enables a comparison among projects with very different goals if both costs and benefits can be placed in monetary terms. Other closely related, but slightly different, formal techniques include cost–utility analysis, economic impact analysis, fiscal impact analysis, and social return on investment (SROI) analysis.

The concept of cost effectiveness is applied to the planning and management of many types of organized activity. It is widely used in many system aspects. Some examples are:

- Studies of the desirable performance characteristics of commercial aircraft to increase an airline's market share at lowest overall cost over its route structure (e.g., more passengers, better fuel consumption)
- Urban studies of the most cost-effective improvements to a city's transportation infrastructure (e.g., buses, trains, motorways, and mass transit routes and departure schedules)
- In health services, where it may be inappropriate to monetize health effect (e.g., years of life, premature births averted, sight years gained)
- In the acquisition of military hardware when competing designs are compared not only for purchase price but also for such factors as their operating radius, top speed, rate of fire, armor protection, and caliber and armor penetration of their guns

LCC Analysis

LCC refers to the total cost incurred by a system throughout its life. This "total" cost varies by circumstances, the stakeholders' points of view, and the system. For example, when purchasing an automobile, the major cost factors are the cost of acquisition, operation, maintenance, and disposal (or trade-in value). A more expensive car (acquisition cost) may have lower LCC because of lower operation and maintenance costs. But the car manufacturer has other costs such as development and production costs, including setting up the production line, to be considered. The SE Practitioner needs to look at costs from several aspects and be aware of the stakeholders' perspectives. LCC should not be equated to Total Cost of Ownership (TCO), Total Ownership Cost (TOC), or Whole Life Cost (WLC). These measures may only include costs once the system has been purchased or acquired.

LCC estimates are sometimes used to support internal project trade-off decisions and need only be accurate enough to support the relative trade-offs. The analyst should always attempt to prepare as accurate cost estimates as possible and assign risk to them. These estimates should be reviewed by upper management and potential stakeholders. Future costs, while unknown, can be predicted based on assumptions and risk assigned. All assumptions when doing LCC analysis should be documented.

LCC analysis can be used in affordability and system cost-effectiveness assessments. The LCC is *not* the definitive cost proposal for a project since LCC "estimates" (based on future assumptions) are often prepared early in a project's life cycle when there is insufficient detailed design information. Later, LCC estimates should be updated with actual costs from early project stages and will be more definitive and accurate due to hands-on experience with the system. A major purpose of LCC studies is to help identify cost drivers and areas in which emphasis can be placed during the subsequent life cycle stages to obtain the best decisions. Accuracy in the estimates will improve as the system evolves and the data used in the calculation is less uncertain.

LCC analysis helps the project team understand the total cost impact of a decision, compare between project alternatives, and support trade studies for decisions made *throughout* the system life cycle. LCC normally includes the following costs, represented in Figure 3.4:

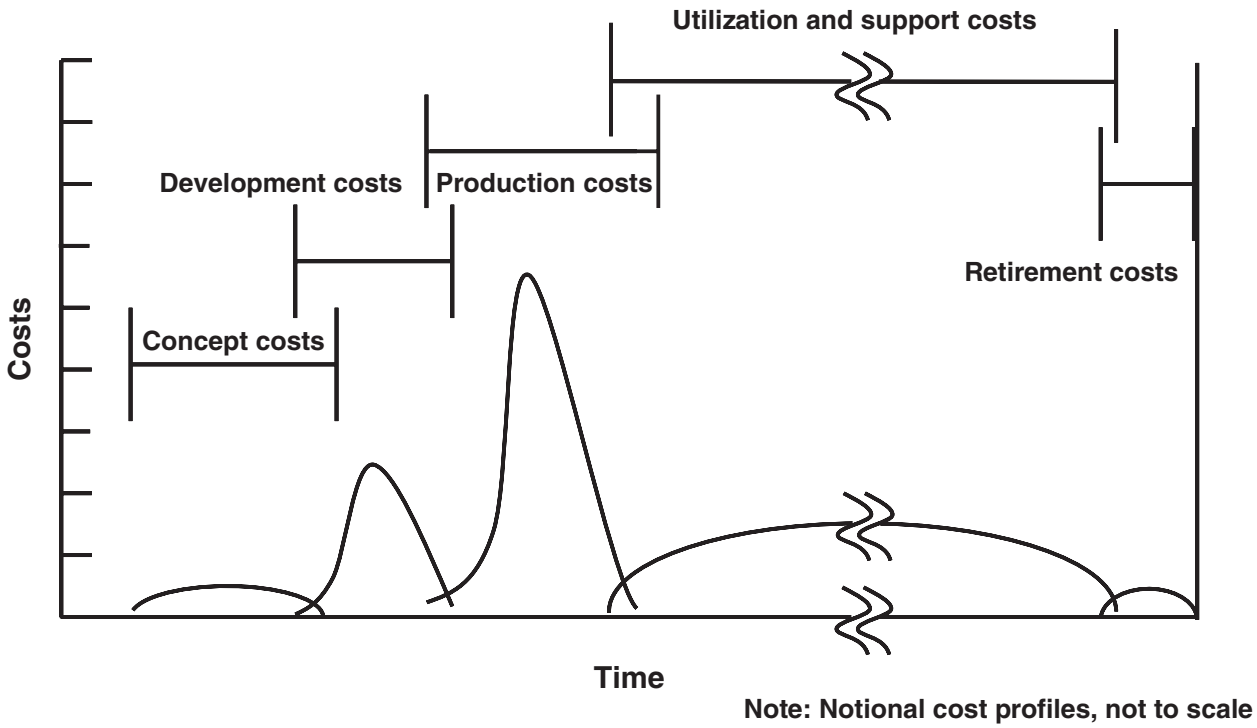


FIGURE 3.4 Life cycle cost elements. INCOSE SEH original figure from INCOSE SEH v2 Figure 4-83. Usage per the INCOSE Notices page. All other rights reserved.

- *Concept costs*—Costs for the initial concept development efforts. These could be estimated based on average staffing and schedule spans and may include overhead, general and administrative (G&A) costs, and fees, as necessary.
- *Development costs*—Costs for the system development efforts. Similar to concept costs, these can be estimated based on average staffing and schedule spans and may include overhead, G&A costs, and fees, as necessary. Parametric cost models may also be used.
- *Production costs*—Usually driven by tooling and material costs for large-volume systems. Labor cost estimates are prepared by estimating the cost of the first production unit and then applying learning curve formula to determine the reduced costs of subsequent production units.
- *Utilization and support costs*—Typically based on future assumptions for ongoing operation and maintenance of the system, for example, fuel costs, personnel levels, and spare parts.
- *Retirement costs*—The costs for removing the system from operation and includes an estimate of trade-in or salvage costs. Could be positive or negative and should be mindful of the environmental impacts to dispose.

For global products, other sources of cost may include compliance costs (government regulations, import/export requirements, etc.) or other incidental costs of international business.

Common methods/techniques for conducting LCC analysis that may be suitable for different situations and/or used in combinations follow:

- *Analogy*—Reasoning by comparing the proposed project with one or more completed projects that are judged to be similar, with corrections added for known differences. May be acceptable for early estimations.
- *Bottom up*—Identifies and estimates costs for each lower-level element separately and rolls them up for the total cost.
- *Delphi technique*—A structured approach to build estimates iteratively from multiple domain experts. Surveys are used, and in each round feedback on the group statistics is provided for experts to help revise their estimates.
- *Design-to-Cost (DTC)*—Using a predetermined cost (e.g., the SoI material cost) as a constraint on the design solution.
- *Expert judgment*—Estimate performed by one or more experts using their experience and judgment. It can be used for comparison and sanity check against other methods.
- *Parametric (algorithmic)*—Uses mathematical algorithms to compute cost estimates as a function of cost factors based on historical data. This technique is supported by public domain and commercial tools and models. Examples include the Constructive Systems Engineering Cost Model (COSYSMO) for SE effort and the Constructive Cost Model (COCOMO) for software engineering effort.
- *Parkinsonian technique*—Work estimates based on the available resources or schedules (Parkinson's Law states that work expands to fill the available volume).
- *Price to win*—Focuses on providing an estimate, and associated solution, at or below the price judged necessary to win the contract.
- *Taxonomy method*—Using a hierarchical structure or classification scheme as a basis of the estimates.
- *Top down*—Developing costs based on overall project characteristics at the top level of the architecture.

3.1.3 Agility Engineering

Definition Agility Engineering is an approach that enables change in a timely and cost-effective manner.

Key Concepts Agility is the ability to thrive and survive in uncertain, unpredictable operational environments; and manifests as effective response to situations presented by the environment (Dove and LaBarge, 2014). Effective response has four metrics:

- timely (fast enough to deliver value),
- affordable (at a cost that can be repeated as often as necessary),
- predictable (can be counted on to meet the need), and
- comprehensive (anything and everything within mission boundary).

Agile systems-engineering and *agile-systems engineering* are two different things (Haberfellner and de Weck, 2005) that share the word agile. In the first case the SoI is an engineering process (e.g., using an agile SE process). This is addressed in Section 4.2.2. In the second case, the SoI is what is produced by an engineering process (e.g., engineering an agile system). This is the subject of this section. Sustained agility is enabled by an architectural pattern and a set of design principles that are fundamental and common to both agile SE processes and engineered agile systems.

Elaboration

Agility Architectural Framework

The architecture that enables agility will be recognized in a simple sense as a drag-and-drop plug-and-play loosely coupled modularity, with some critical aspects not often called to mind with the general thoughts of a modular architecture. The architectural objective is to enable rapid and effective composability of processes and systems from available resources, appropriate for the needs at hand (Dove and LaBarge, 2014). Construction toys, like Lego or Meccano sets, are iconic architectural examples.

There are three critical elements in the architecture: a roster of drag-and-drop *encapsulated modules*, a *passive infrastructure* of minimal but sufficient rules and standards that enable and constrain plug-and-play operation, and an *active infrastructure* that designates specific responsibilities that sustain agile operational capability:

Encapsulated modules—Modules are self-contained encapsulated units complete with well-defined interfaces that conform to the plug-and-play passive infrastructure. They can be dragged and dropped into a system of response capability with relationship to other modules determined by the passive infrastructure. Modules are encapsulated so that their interfaces conform to the passive infrastructure, but their methods of functionality are not dependent on the functional methods of other modules except as the passive infrastructure dictates.

Passive infrastructure—The passive infrastructure provides drag-and-drop connectivity between modules. Its value is in isolating the encapsulated modules so that unexpected side effects are minimized and new operational functionality is rapid. Selecting passive infrastructure elements is a critical balance between requisite variety and parsimony—just enough in standards and rules to facilitate module connectivity but not so much to overly constrain innovative system configurations.

Active infrastructure—An agile system is not something designed and deployed in a fixed event and then left alone. Agility is most active as new system configurations are assembled in response to new requirements—something which may happen very frequently, even daily in some cases. In order for new configurations to be enabled when needed, five responsibilities are required:

- Module mix evolution—Who (or what process) is responsible for ensuring that new modules are added to the roster and existing modules are upgraded in time to satisfy response needs?
- Module readiness—Who (or what process) is responsible for ensuring that sufficient modules are ready for deployment at unpredictable times?
- Situational awareness—Who (or what process) is responsible for monitoring, evaluating, and anticipating the operational environment?
- System assembly—Who (or what process) assembles new system configurations when new situations require something different in capability?
- Infrastructure evolution—Who (or what process) is responsible for evolving the passive and active infrastructures as new rules and standards are anticipated and become appropriate?

Responsibilities for these five activities must be designated and embedded within the system to ensure that effective response capability is possible at unpredictable times

Agility Architectural Design Principles

Ten reusable, reconfigurable, scalable design principles are briefly itemized in this section:

Reusable principles are as follows:

- *Encapsulated modules*—Modules are distinct, separable, loosely coupled, independent units cooperating toward a shared common purpose.
- *Facilitated interfacing (plug compatibility)*—Modules share well-defined interaction and interface standards and are easily inserted or removed in system configurations.
- *Facilitated reuse*—Modules are reusable and replicable, with supporting facilitation for finding and employing appropriate modules.

Reconfigurable principles are as follows:

- *Peer–peer interaction*—Modules communicate directly on a peer-to-peer relationship; and parallel (rather than sequential) relationships are favored.
- *Distributed control and information*—Modules are directed by objective (rather than method); decisions are made at point of maximum knowledge, and information is associated locally and accessible globally.
- *Deferred commitment*—Requirements can change rapidly and continue to evolve. Work activity, response assembly, and response deployment that are deferred to the last responsible moment avoid costly wasted effort that may also preclude a subsequent effective response.
- *Self-organization*—Module relationships are self-determined where possible, and module interaction is self-adjusting or self-negotiated.

Scalable principles are as follows:

- *Evolving infrastructure standards*—Passive infrastructure standardizes intermodular communication and interaction, defines module compatibility, and is evolved by designated responsibility for maintaining current and emerging relevance.
- *Redundancy and diversity*—Duplicate modules provide capacity right-sizing options and fail-soft tolerance, and diversity among similar modules employing different methods is exploitable.
- *Elastic capacity*—Modules may be combined in responsive assemblies to increase or decrease functional capacity within the current architecture.

Agility Metrics

Agility measures are enabled and constrained principally by architecture—in both the process and the product of development:

- *Time to respond*, measured in both the time to understand a response is necessary and the time to accomplish the response.
- *Cost to respond*, measured in both the cost of accomplishing the response and the cost incurred elsewhere as a result of the response.

- *Predictability of response*, measured before the fact in architectural preparedness for response and confirmed after the fact in repeatable accuracy of response time and cost estimates.
- *Scope of response*, measured before the fact in architectural preparedness for comprehensive response capability within mission and confirmed after the fact in repeatable evidence of broad response accommodation.

3.1.4 Human Systems Integration

Definition Human Systems Integration (HSI) is an approach that integrates technology, organizations, and people effectively.

HSI is an essential, transdisciplinary, sociotechnical, and management approach of SE used to ensure that the system's technical, organizational, and human elements are appropriately addressed across the whole system life cycle, service, or enterprise system. HSI considers systems in their operational context together with the necessary interactions between and among their human and technological elements to make them work in harmony and cost effectively, from concept to retirement.

Key Concepts

Human

The "human" in HSI includes all individuals and groups interacting within the SoI. Within HSI, these are typically referred to as "stakeholders." Stakeholders can include system acquirers, owners, users, operators, maintainers, trainers, support personnel, and the general public. While most people who interact within the SoI will be cooperative or have a vested interest in its performance, consideration may also need to be given to non-cooperative people or those with malign intent such as competitors, adversaries, criminals (physical and cyber), and those seeking to use the system outside of its design intent. Life, human, and social sciences have different representations of the human element and can all bring different perspectives to HSI activities.

Systems

HSI adopts a sociotechnical system perspective that considers a system as a representation of natural and artificial elements that are organizations of humans and machines (where machines include both hardware and software). Therefore, HSI considers that all systems include both humans and machines, and to optimize the system, all of these elements must be considered within SE activities.

Integration

HSI considers integration from two key viewpoints. The first is the effective integration of the human and technological elements in a system. The second is the efficient integration of the different perspectives of both human and machine elements within the system. An example of these different HSI perspectives can be seen in Figure 3.5. The specific perspectives relevant to a project will vary depending on the nature of the system and the organization's activities.

All systems involve or affect people and exist within a wider sociotechnical and organizational context. Therefore, HSI is an essential enabler to SE practice. The sociotechnical approach provided by HSI supports analysis, design, and evaluation activities in holistically understanding and effectively integrating the technological, organizational (including processes), and human elements of a system. As shown in Figure 3.5, HSI emerges from the overlapping of three main circles: (1) technology, organization, and people (the TOP Model) within an environment at the heart; (2) HSI perspectives; and (3) contributing disciplines associated with the operational domain shown in the periphery. It is particularly important that systems are designed to meet human capabilities, limitations, and goals.

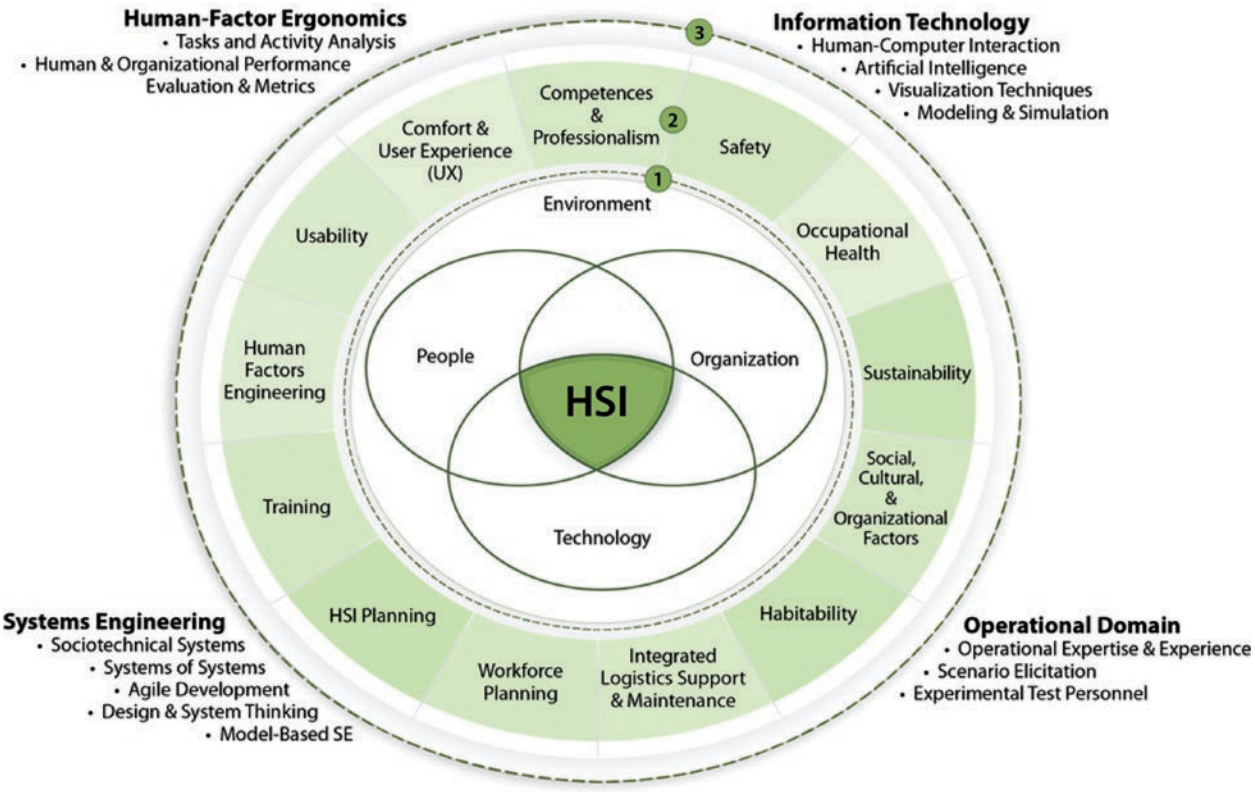


FIGURE 3.5 HSI technology, organization, people within an environment. INCOSE SEH original figure created by Boy. Usage per the INCOSE Notices page. All other rights reserved.

Elaboration

Purpose and Value of HSI

The purpose of HSI is to optimize total system performance and stakeholder satisfaction through the mutual integration of technology, organizations (including processes), people, and environment.

The benefits which can be realized by HSI vary from domain to domain, depending on their priorities and purpose (e.g., safety, cost, efficiency, performance, acceptability) and the nature of the system. They can be broken down into the following areas:

- *holistic optimization of system performance and efficiency*: participatory design, and human-in-the-loop (HITL) activities
- *improved safety*: hazard, risk, performance limitations and emergent properties analysis
- *reduced development costs*: consider the TOP Model
- *reduced system LCC costs*: HSI from the beginning of the SE life cycle
- *improved sales*: resulting from product or service usability
- *user experience (UX) and desirability*: focus on Human-Centered Design (HCD) and user needs
- *improved adoption of new systems by the workforce or user groups*: considering sociotechnical factors
- *HSI value to a project*: from intuition to expertise in HSI

Scope and Breadth of HSI

HSI is based on the convergence of four key communities of practice (third circle in Figure 3.5):

- *human factors and ergonomics (HF/E)* that provides human-centered and organization-centered analysis, performance evaluation techniques, and metrics (Boehm-Davis, et al., 2015);
- *information technology (IT)* that includes human-computer interaction, artificial intelligence, visualization techniques, and modeling and simulation;
- *systems engineering* that includes socio-technical systems, systems of systems (see Section 4.3.6), agile development (see Section 4.2.2), design and system thinking (see Sections 3.2.7 and 1.5), and model-based SE (MBSE) (see Section 4.2.1); and
- *the operational domain* that includes operational expertise and experience, scenario elicitation, and experimental test personnel (see Section 4.4).

These communities enable support of HSI through HCD as a major process that involves development and use of domain ontology, prototypes and digital modeling, scenario-based design, modeling and HITL activities (simulations and physical tests), formative evaluations, agile design and development, as well as human performance and organizational metrics (e.g., maturity and flexibility) (Boy, 2013) (Boy, 2020). HCD validation both requires certification approval and contributes to certification rules evolution.

HSI considers systems complexity analysis as a baseline. It seeks simplification (where possible) and familiarity with complex systems (where necessary). HITL activities enable discovery and elicitation of complex systems' emergent behaviors, properties, functions, and structures, which are incrementally integrated into the SoI through its whole life cycle. HITL activities provide SE and HCD teams with improved understanding of the SoI early in the life cycle, contributing to design flexibility and better resource management. HSI is a foundational enabler for industrial endeavors, such as Industry 4.0, where digital engineering, enabling virtual HCD, requires increased physical and cognitive tangibility testing across the life cycle of a system (see Section 5.4). Case 5 (Artificial Intelligence in Systems Engineering - Autonomous Vehicles) from Section 6.5 illustrates the importance of all these aspects.

HSI can be considered as both an *enabling process*, associating HCD and SE during the life cycle of a system, and a *product* resulting from this process. HSI is the result of this HCD-based convergence, which requires optimizing the TOP Model. User eXperience (UX) and User Interface (UI) development are integral parts of the HSI process from the early stages and throughout the system life cycle. HSI processes are iterative and supported by two main types of assets, methods, and tools: expertise elicitation and creativity. The former enables effective elicitation from subject matter experts through knowledge and know-how, supporting design teams during system formative evaluations, agile development, and certification. The latter enables out-of-the-box projections that are validated using prototyping and HITL activities.

HSI Perspectives

HSI encompasses several important perspectives displayed in Figure 3.5 (second circle) and described in more detail in Table 3.2.

A wide variety of HSI methods, models, knowledge, and approaches can be used to support decisions made across the whole system life cycle. This can include support to requirements analysis, trade-studies, life cost benefit analysis, options or tender down select, risk management, safety case development, design decisions, acceptance testing, and workforce planning. Human-related trade studies are critical to determining holistic of operational concept (OpsCon) and thereby informing the design team in terms of effectivity, efficiency, suitability, usability, safety, and affordability. See the INCOSE HSI Primer (2023) for more detail.

TABLE 3.2 HSI perspective descriptions

Human Factors Engineering (*HFE*) is the scientific discipline concerned with the understanding of interactions among humans and other elements of a system, and the profession that applies theory, principles, data, and other methods to design in order to optimize human well-being and overall system performance.
Social, Cultural, and Organizational Factors consider the organizational aspects of socio-technical systems and includes the organizations who will be using and supporting the operational system, as well as the organizations who are involved throughout the entire life cycle of the system.
HSI Planning addresses the implementation of HSI through the SE process to ensure the human element is effectively integrated with the system. HSI strategies and priorities need to be set up-front, can be formalized in the HSI Plan, and potentially adjusted during the life cycle, upon mission definition, and carried throughout the allocation of resources and project personnel.
Integrated Logistics Support (ILS) & Maintenance covers human performance during the whole life cycle of a system based on an ILS plan supported by an HSI plan. ILS includes training, operations, maintenance, potential redesign, and dismantling.
Workforce Planning addresses the number and type of personnel and the various occupational specialties required and potentially available to develop, train, operate, maintain, and support the system.
Competences and Professionalism consider the type of knowledge, skills, experience levels, and aptitudes (cognitive, physical, and sensory) required to operate, maintain, and support a critical system and the means to provide such people (through selection, recruitment, training, etc.).
Training encompasses designing to account for ease and reduction of operation time needed to provide training through trade studies evaluated to assess their impact on training, as well as the instructions and resources required to provide personnel with requisite competence, knowledge, skills, and attitudes to properly operate, maintain, and support systems.
Safety promotes system characteristics and procedures to minimize the risk of accidents or mishaps that cause death or injury to operators, maintainers, support personnel, or others who could come into intentional or unintentional contact with the system; threaten systems operations; or cause cascading failures in other systems. It includes survivability.
Occupational Health promotes system design features and procedures that serve to minimize physiological mental and social health hazards which might result in injury, acute or chronic illness, and disability; and to enhance job performance and wellbeing of personnel who operate, maintain, or support the system.
Sustainability covers the environmental considerations that can affect operations and particularly human performance and considers wider ranging concerns and long-term goals of how the humans within the system can affect the environment, society, and economy without compromising future generations' needs.
Habitability involves characteristics of system living and working conditions.
Usability involves objective evaluation methods to address aspects such as efficiency, conformity to human expectations, tolerance/resistance toward human errors, and learnability to improve the degree to which humans can reach their objectives when interacting with a system.
Comfort and UX are personal internal human aspects such as joy, guilt, opinions, and unconscious aspects which are to be considered, not only in regard to the primary users of the final product, but in regard to all humans involved in the systems engineering process.

3.1.5 Interoperability Analysis

Definition Interoperability Analysis is an approach that ensures the system interacts effectively with other systems. In the domains of data/information exchange and communications, there are four definitions of interoperability:

- The capability of systems to communicate with one another and to exchange and use information including content, format, and semantics (NIST SP 500-230, 1996).
- The ability of two or more systems or system elements to exchange data and use information (IEEE 610.12, 1990).
- The ability of two or more systems to exchange information and to mutually use the information that is exchanged (US Army, 1997).
- The condition achieved among communications-electronics systems or items of communications—electronics equipment when information or services can be exchanged directly and satisfactorily between them and/or their users (US DoD, 2021).

Key Concepts Interoperability reflects the ability of a system to work in conjunction with other system(s) to achieve an outcome. For example, a mobile phone can operate on different networks across the world, agricultural implements from different companies can work on each other's tractors, or a system provides an interface allowing remote control of its capabilities. Originally described in terms of computer/software systems, the concept of interoperability applies more widely, such as human interactions. A broad definition of interoperability also takes into account social, political, and organizational factors that impact system-to-system performance. Interoperability is a key enabler for a System of Systems (SoS), because it allows the elements of a large and complex system to work together as a single entity, toward a shared purpose (see Section 4.3.6).

Interoperability may be achieved in two principal ways, which can also be combined:

- Agreeing on one or more *published standards* as the definition of the interface. This exposure of interfaces complying with open interfaces is increasingly common in the consumer product area where "plug and play" is expected.
- Defining and implementing a *custom interface*. When a standard interface does not exist, or is not suitable, a custom interface can be defined as the agreed way in which two or more systems will connect, communicate, interact, or cooperate to achieve their shared purpose.

Elaboration Interoperability will increase in importance as the world grows smaller due to expanding communications networks (e.g., the internet of things (IoT)), as nations continue to perceive the need to communicate seamlessly across international coalitions of commercial organizations or national defense forces, and as individuals increasingly expect that products and services will "work together."

The Øresund Bridge (see Section 6.2) demonstrates the interoperability challenges faced when just two nations collaborate on a project. For example, the meshing of regulations on health and safety, interfacing a left-handed (Sweden) and right-handed (Denmark) railway, and the resolution of two power supply systems for the railway. Hence careful choices were necessary for the standards selected for the bridge itself, and for its interfaces at the Swedish and Danish ends.

3.1.6 Logistics Engineering

Definition Logistics Engineering is an approach that enables support for the entire life cycle.

Key Concepts Logistics engineering, which may also be referred to as product support engineering, is the engineering discipline concerned with the identification, acquisition, procurement, and provisioning of all support resources required to sustain operation and maintenance of a system (Blanchard and Fabrycky, 2011). Logistics engineering is also concerned with engineering the inherent supportability of the design. Logistics should be addressed from a life cycle perspective and be considered in all stages and especially as an inherent part of system concept and development. Furthermore, logistics should be approached from a system perspective to include all activities associated with design

for supportability, the acquisition and procurement of the elements of support, the supply and distribution of required support material, and the maintenance and support of systems throughout their planned period of utilization.

The scope of logistics engineering is thus
- to determine logistics support requirements,
- to design the system for supportability,
- to acquire or procure the support, and
- to provide cost-effective logistics support for a system during utilization and support stages.

Logistics engineering has evolved into several related elements such as supply chain management (SCM) in the commercial sector and integrated logistics support (ILS) in the defense sector.

Elaboration

Support Elements

Support planning starts with the definition of the support (including maintenance) concept in the concept stage and continues through supportability analysis in the development stage, to the ultimate development of a support plan. The support concept describes the support environment in which the system will operate and which inherent supportability and support system elements are required for establishing the system operational capability.

The following elements of support are to be fully integrated with the system at the lowest possible LCC:
- *Product support integration and management*—Plan and manage cost and performance across the product support value chain, from concept to retirement.
- *Design interface*—Participate in the SE process to impact the design from inception throughout the life cycle. Facilitate supportability to maximize availability, effectiveness, and capability at the lowest LCC. Early application of the support concept drives the design inherent supportability objectives and trade-offs. It is an important mechanism for aligning design Reliability, Maintainability, and Supportability (RMS), maintenance planning, and establishment of support capabilities for the operational environment. It guides design modularity, reliability, maintainability, testability, and overall repair policies.
- *Sustained logistics engineering of the fielded system*—This effort spans those technical tasks (engineering investigations and analyses) to ensure continued dependable operation, including maintenance, for the life cycle. It characterizes the system and support capabilities' RMS performance as an input to dependable planning of operational use. It involves applying improved confidence level RMS characteristics data, gained from the operational experience, to enhance maintenance strategy and the support system, and to propose design RMS improvements.
- *Maintenance planning*—Identifying the system maintenance requirements, determining the maintenance strategy, and implementing the maintenance capabilities required to deliver the system operational capability. The support concept guides overall repair policies, such as "repair vs. replace" criteria.
- *Operation and maintenance personnel*—Identify, plan, and acquire personnel, with the training, experience, and skills required to operate, maintain, and support the system.
- *Training and training support*—Establish and maintain the required operator and maintainer skill levels across the system life cycle. Identify, develop, and acquire Training Aids, Devices, Simulators, and Simulations (TADSS) to maximize the effectiveness of the personnel to operate and sustain the system equipment.
- *Supply support*—determine requirements for supply, and acquire, catalog, receive, store, transfer, issue, and dispose of spares, repair parts, and supplies. This means having the right spares, repair parts, and all classes of supplies available, in the right quantities, at the right place, at the right time, at the right price.
- *Computer resources (hardware and software)*—Computers, associated software, networks, and interfaces necessary to enable long-term logistics engineering, maintenance management, system technical and associated support operations data management, and storage.

- *Technical data, reports, and documentation*—Represents recorded information of scientific or technical nature (e.g., equipment technical manuals, engineering drawings), engineering data, specifications, and standards.
- *Facilities and infrastructure*—This includes facilities (e.g., buildings, warehouses, hangars, waterways, associated facilities equipment) and infrastructure (e.g., IT services, fuel, water, electrical service, machine shops, dry docks, test ranges).
- *Packaging, handling, storage, and transportation (PHS&T)*—Ensure that all system equipment and support items are preserved, packaged, handled, and transported properly, including environmental considerations, equipment reservation for short and long storage, and transportability. Some items may require special environmentally controlled, shock-isolated containers for transport to and from storage, operational, and repair facilities via all modes of transportation (e.g., land, rail, sea, air, space).
- *Support equipment*—All equipment (mobile and fixed) required to sustain the operation and maintenance of a system, including, but not limited to, handling and maintenance equipment, trucks, air conditioners, generators, tools, metrology and calibration equipment, and manual and automatic test equipment.

Supportability Analysis

As shown in the Figure 3.1, supportability analysis addresses all elements of design supportability and of the support system required during all life cycle stages:

- *Functional failure analysis*—A Functional Breakdown Structure (FBS) is used as reference to perform functional FMECA (Failure Mode Effects and Criticality Analysis), FTA (Fault Tree Analysis) and/or RBD (Reliability Block Diagram) analysis. These analyses can be used to identify functional failure modes and to classify them according to criticality (e.g., severity of failure effects and probability of occurrence). The functional failure analysis can also provide valuable system design input (e.g., redundancy requirements). In describing functional failure compensation means, including compensation by support, the functional failure analysis provides early means of illustrating the system supportability interface and criticality of support.
- *Physical failure analysis*—A Product Breakdown Structure (PBS) is used as reference to perform hardware FMECA, FTA, and/or RBD analysis with the objective of optimizing the design and to identify all maintenance tasks for potential failure modes. An objective of logistic engineering is to minimize operational maintenance tasks and resource requirements. The FMECA (in assessing the design inherent reliability, protection, and testability versus reliance on preventive or corrective maintenance) allows in context trade-offs of the operational value of improving the design versus defaulting to reliance on operational maintenance. The FMECA findings are used to balance the level of repair allocation. Failure probability, criticality, detection means, the design modularity, and the complexity of failure restoration need to be in balance with the level of repair capabilities framed by the system support concept.
- *Task identification and optimization*—Corrective maintenance tasks are primarily identified using FMECA, while preventive maintenance tasks are identified using RCM (Reliability-Centered Maintenance). Trade-off studies may be required to achieve an optimized maintenance strategy. Associated support tasks, such as operational transportation, are identified from analysis of the operational concept and support workflows.
- *Detail task analysis*—Detail procedures for support tasks should be developed, and support resources identified and allocated to each task. The system Level of Repair Analysis (LORA), in conjunction with the support concept, may be used to determine the most appropriate location for executing these tasks.
- *Support element specifications*—Support element specifications should be developed for all support deliverables. Depending on the system, specifications may be required for training aids, facilities, support equipment, publications, and packaging material. Establishment of support elements, such as facilities, may involve extended lead times requiring identification of requirements and initiation of acquisition from as early as the system concept stage. The support element requirements analysis is therefore iterated from the system concept stage to highlight long-lead time support element acquisition requirements.
- *Support deliverables, test, and evaluation*—All support deliverables should be acquired based on the individual specifications. The support deliverables should be tested and evaluated against support element specifications and the overall system requirements.

- *Support modeling and simulation*—Modeling and simulation are integral parts of supportability analysis that should be initiated during the early stages to frame and develop a compliant and optimized system design, maintenance strategy, and support system. Modeling and simulation during acquisition are progressed to become decision and planning optimization tools for the operational and support stages. The predictive modeling information during acquisition is progressively matured as experience is gained with the operational system and operational support capabilities (e.g., digital twin for operation and support).
- *Recording and corrective action*—Failure recording and corrective action during the utilization and support stages form the basis for continuous improvement. System operational value delivery metrics should be applied to continuously monitor the system to improve support where deficiencies are identified, and to highlight focus areas for operational enhancements to system inherent reliability, maintainability, and supportability.

3.1.7 Manufacturability/Producibility Analysis

Definition Manufacturability/Producibility Analysis is an approach that enables production in a responsible and cost-effective manner.

Key Concepts Production involves the repeated manufacture of the developed system. The capability to manufacture or produce a system or its elements is as essential as the ability to properly develop it. A system that cannot be effectively produced causes unnecessary costs and may lead to rework and project delays with associated cost overruns. For this reason, manufacturability/producibility analysis is an integral part of the SE process.

Producibility considerations differ depending upon the type and number of systems being produced. For example, the manufacture of satellites (limited production runs), military tanks (medium production runs), and mobile phones (high production runs) would be vastly different. A unique aspect of infrastructure systems is that production typically takes place on-site, rather than in a factory (see Section 4.4.5). Multiple production cycles require the consideration of production maintenance and downtime.

One objective is to determine if existing production enabling systems are satisfactory (see Section 1.3.3), since this could be the lowest risk and most cost-effective approach. If not, the requirements for the production enabling systems and processes need to be determined, and the production enabling systems developed so they are ready when needed. A SE approach to manufacturing and production is necessary because the production enabling systems can sometimes cost more than the system being produced (Maier and Rechtin, 2009).

Elaboration Producibility analysis is a key task in developing cost-effective, quality products. Multidisciplinary teams work to simplify the design and stabilize the manufacturing process to reduce risks, manufacturing costs, lead times, and cycle times and to minimize strategic or critical material use. Producibility analysis draws upon the production and support life cycle concepts. Producibility requirements are identified in the Business or Mission Analysis and Stakeholder Needs and Requirements Definition processes (see Sections 2.3.5.1 and 2.3.5.2) and included in the project risk analysis, if necessary. Similarly, long-lead-time items, sole source items (where only one supplier for the required item is available), material limitations, special processes, and manufacturing constraints are evaluated. Design simplification also considers ready assembly and disassembly for ease of maintenance and preservation of material for recycling. When production requirements create a constraint on the system, they are communicated and documented. The selection of manufacturing methods and processes is included in early decisions. Manufacturing test considerations are captured and are taken into account in built-in test and automated test equipment.

IKEA® is often used as an example of supply chain excellence. IKEA has orchestrated a value creating chain that begins with motivating customers to perform the final stages of furniture assembly in exchange for lower prices and a fun shopping experience. They achieve this through designs that support low-cost production and transportability (e.g., the bookcase that comes in a flat package and goes home on the roof of a car).

3.1.8 Reliability, Availability, Maintainability Engineering

Definition Reliability, Availability, Maintainability Engineering is an approach that enables the system to perform without failure, to be operational when needed, and to be retained in or restored to a required functional state.

RAM (sometimes expressed as RMA) is a well-known acronym for Reliability, Availability, and Maintainability. These QCs are completely interrelated with each other and have a strong relationship with logistics and supportability.

Key Concepts From a SE perspective, RAM should not only be viewed as quality characteristics, but as nonfunctional requirements. RAM activities are often neglected during system development, resulting in a substantial increase in risk of project failure or stakeholder dissatisfaction. Since RAM often drives other system requirements, it is essential that these activities be selected, tailored, planned, and executed in an integrated manner with other SE processes. A practical way to achieve this is to develop detailed reliability and maintainability plans early in the system development process and to integrate these plans with the SE management plan (SEMP).

RAM, being important inputs to the system maintenance concept, support other SE processes in two ways. First, they should be used to influence both system and system support definitions (e.g., the system architecture depends on RAM requirements). Second, they should be used as part of system verification (e.g., system analysis or system test).

Depending on the particular industry, availability is often seen as the most important of these three quality characteristics, especially from the viewpoint of a user or acquirer. Any availability loss can usually easily be translated to mission or production loss and increased costs.

Elaboration

Reliability

The IEEE Reliability Society defines:

> Reliability is a design engineering discipline which applies scientific knowledge to assure a product will perform its intended function for the required duration within a given environment. This includes designing in the ability to maintain, test, and support the product throughout its total life cycle. Reliability is best described as product performance over time. This is accomplished concurrently with other design disciplines by contributing to the selection of the system architecture, materials, processes, and system elements—both software and hardware; followed by verifying the selections made by thorough analysis and test.

"To be reliable, a system must be robust—it must avoid failure modes even in the presence of a broad range of conditions including harsh environments, changing operational demands, and internal deterioration" (Clausing and Frey, 2005). An in-depth understanding of the interaction between the system, the environment where it will be used, the operating conditions it will be subjected to, and potential failure modes and failure mechanisms is thus essential to design and manufacture reliable systems. Figure 3.6 shows the interaction between these aspects.

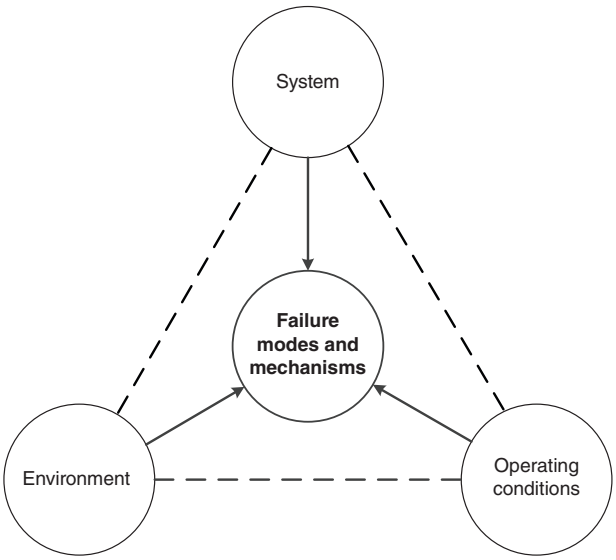


FIGURE 3.6 Interaction between system, environment, operating conditions, and failure modes and failure mechanisms. INCOSE SEH original figure created by Barnard. Usage per the INCOSE Notices page. All other rights reserved.

Reliability can be formally defined as "the ability of a system to perform as designed, without failure, in an operational environment, for a stated period of time" (Tortorella, 2015). Since "ability" is an abstract concept, many reliability metrics are available which can be used to measure and manage the reliability of a system during the development, utilization, and support stages (e.g., number of failures per time period, failure-free period, expected lifetime of non-repairable parts, Mean Time Between Failure [MTBF]).

O'Connor and Kleyner (2012) state the objectives of reliability engineering, in the order of priority, are:

1. To apply engineering knowledge and specialist techniques to prevent or to reduce the likelihood or frequency of failures.
2. To identify and correct the causes of failures that do occur, despite the efforts to prevent them.
3. To determine ways of coping with failures that do occur, if their causes have not been corrected.
4. To apply methods for estimating the likely reliability of new designs and for analyzing reliability data.

The priority emphasis is important, since proactive prevention of failure is always more cost-effective than reactive correction of failure. Timely execution of appropriate reliability engineering activities is of utmost importance in achieving the required system reliability.

Modern approaches to reliability place strong emphasis on the engineering processes required to prevent failure during the expected life of a system. The concept of "design for reliability" has recently shifted the focus from a reactive "test-analyze-fix" approach to a proactive approach of designing reliability into the system. This requires that design attention be given to the early identification of potential failure modes, with subsequent mitigation actions implemented during development (i.e., reliability objective 1). Understanding of "how" (i.e., failure modes) and "why" (i.e., failure mechanisms) a system can fail is key to the achievement of reliability. In practice, this proactive approach to reliability is always complemented by a reactive approach where observed failure modes are managed and corrected (i.e., reliability objectives 2 and 3). Finally, reliability prediction, test, and demonstration play an important role during development stages (i.e., reliability objective 4).

"Design for reliability" implies that reliability should receive adequate attention during requirements analysis. Reliability requirements may be specified either in qualitative or quantitative terms, depending on the specific industry. Care should be taken with quantitative requirements, since verification by test of reliability is often not practical (especially for high reliability requirements). Also, the misuse of some reliability metrics (e.g., MTBF) frequently results in "playing the numbers game" during system development, instead of focusing on the engineering effort necessary to achieve reliability (Barnard, 2008). For example, MTBF is often used as an indicator of "expected life" of an item, which is incorrect. It is therefore recommended that other reliability metrics be used for quantitative requirements (e.g., reliability (as success probability) at a specific time, or failure-free period).

Appropriate reliability engineering activities should be selected and tailored according to the objectives of the specific project. These activities should be captured in the reliability program plan. The plan should indicate which activities will be performed, the planned timing of the activities, the level of detail required for the activities, and the parties responsible for execution of the activities. ANSI/GEIA-STD-0009 Reliability Program Standard for Systems Design, Development, and Manufacturing which supports a system life cycle approach to reliability engineering, can be referenced for this purpose. This standard addresses not only hardware and software failures but also other common failure causes (e.g., manufacturing, operator error, operator maintenance, training, quality). "At the heart of the standard is a systematic 'design-reliability-in' process, which includes three elements:

- Progressive understanding of system-level operational and environmental loads and the resulting loads and stresses that occur throughout the structure of the system.
- Progressive identification of the resulting failure modes and mechanisms.
- Aggressive mitigation of surfaced failure modes."

ANSI/GEIA-STD-0009 (2008) consists of the following objectives:

- Understand acquirer / user requirements and constraints.
- Design and redesign for reliability.
- Produce reliable systems / products.
- Monitor and assess user reliability.

The reliability program plan is often used to capture a forward-looking view on how to achieve reliability objectives. Complementary to the reliability program plan is the reliability case which provides a retrospective (and documented) view on achieved objectives during the system life cycle.

"Failure mode avoidance" approaches attempt to improve reliability of a system primarily early during development stages. It is performed by evaluating system functions, technology maturity, system architecture, redundancy, design options, etc., in terms of potential failure modes. The most significant improvements in system reliability can be achieved by avoiding physical failure modes in the first place and not by minor improvements after the system has been conceived, designed, and produced.

Reliability engineering activities can be divided into two groups: engineering analyses and tests and failure analyses. These activities are supported by various reliability management activities (e.g., design procedures, design checklists, design reviews, electronic part derating guidelines, preferred parts lists, preferred supplier lists).

Engineering analyses and tests refer to traditional design analyses and test methods performed during system development. Included in this group are Finite Element Analysis (FEA), Computational Fluid Dynamics (CFD), vibration and shock analysis, load-strength analysis, thermal analysis and measurement, electrical and mechanical stress analysis, wear-out life prediction, Accelerated Life Testing (ALT), and Highly Accelerated Life Testing (HALT).

Failure analyses refer to traditional reliability engineering analyses to improve understanding of cause-and-effect relationships. Included in this group are Failure Mode and Effects Analysis (FMEA), Fault Tree Analysis (FTA), Reliability Block Diagram (RBD) analysis, systems modeling, Monte Carlo simulation, failure data analysis, root cause analysis, and reliability growth analysis.

Availability

As part of system effectiveness, availability requirements should be carefully derived from user needs and specified during system definition. These requirements play a key role in influencing a multitude of design decisions and availability should be monitored during the utilization and support stages. The simplest definition of availability is the ratio between uptime and total time of a system, usually expressed as a percentage. Since total time consists of uptime and downtime, availability is therefore dependent on the reliability (influencing uptime) and maintainability (influencing downtime) of the system. Furthermore, downtime is obviously highly dependent on the system support environment during the support stage (influencing delay times). Due to these direct relationships, availability is governed by reliability, maintainability, and various logistics engineering aspects. Since availability is a function of both reliability and maintainability (including logistics aspects), achievement of a required availability usually requires trade-offs between reliability and maintainability, and other requirements and constraints (e.g., performance, cost).

Availability can be formally defined as "the probability that a system, when used under stated conditions, will operate satisfactorily at any point in time as required" (Blanchard, 2004). It may be expressed and defined as inherent, achieved, or operational availability:

- *Inherent availability (Ai)* is based only on the inherent reliability and maintainability of the system. It assumes an ideal support environment (e.g., readily available tools, spares, maintenance personnel) and excludes preventive maintenance, logistics delay time, and administrative delay time.
- *Achieved availability (Aa)* is similar to inherent availability, except that preventive (i.e., scheduled) maintenance is included. It excludes logistics delay time and administrative delay time.

- *Operational availability (Ao)* assumes an actual operational environment and therefore also includes logistics delay time and administrative delay time.

Inherent availability thus focusses primarily on "design for reliability and maintainability" activities. Achieved availability takes a broader view to include preventive maintenance, and operational availability includes possible logistics and administrative delays.

A service-level agreement (SLA) between a service provider and an acquirer typically includes availability performance, usually measured for a certain period (e.g., one year) and is then translated into the maximum duration of downtime allowed for that period.

Maintainability

An objective in SE is to design and develop a system that can be maintained effectively, safely, in the least amount of time, in a cost-effective manner, and with a minimum expenditure of support resources without adversely affecting the mission of that system. Maintainability refers to all measures and activities implemented during the design, production, and use of a system that reduces the required maintenance (as measured in maintenance frequency, repair hours, tools, cost, skills, and facilities). Maintainability is thus the ability of a system to be maintained, whereas maintenance constitutes a series of actions to be taken to restore or retain a system in an effective operational state. Maintainability must be inherent or "built into" the design, while maintenance is the result of design. Maintainability can formally be defined as "the ability of a system to be repaired and restored to service when maintenance is conducted by personnel using specified skill levels and prescribed procedures and resources" (Tortorella, 2015). Case 4 (Design for Maintainability-Incubators) from Section 6.4 illustrates the importance of maintainability.

Maintenance can be broken down into the following groups:

- *Corrective maintenance*: unscheduled maintenance accomplished, as a result of failure, to restore a system to a specified level of performance.
- *Preventive maintenance*: scheduled maintenance accomplished to retain a system at a specified level of performance by providing systematic inspection and servicing or preventing impending failures through periodic item replacements.
- *Predictive maintenance*: scheduled maintenance based on the in-service condition of a system to estimate when maintenance should be performed.
- *System upgrades*: periodic maintenance to support system life extension and performance upgrades.

A maintainability engineering plan is often used to capture activities such as quantitative maintainability modeling and simulation, development of the system maintenance concept, level of repair analysis (LORA), diagnostic capabilities, identification of preventive maintenance activities, etc. It is thus closely related to logistics engineering (see Section 3.1.6). The maintainability engineering plan should consider various aspects such as interchangeability of parts, accessibility to parts for removal, and testability of equipment. Testability includes aspects such as built-in test (BIT) capability, diagnostic test equipment, and support software. Service providers such as telecommunication operators that serve the mass market may use OTA (Over-the-Air) technology to remotely provide maintenance (e.g., data transfer to update software or firmware). Like reliability, maintainability requirements should be derived from system availability requirements.

Various maintainability metrics can be used to specify or measure maintainability. The most widely used metric, Mean Time to Repair (MTTR), measures the elapsed time to perform a certain maintenance activity. It typically includes time for activities such as failure detection/failure isolation (FD/FI), disassembly, active repair, reassembly, and finally system testing. It is important to note that MTTR refers to the mean time of the underlying probability distribution. Maintenance times tend to be lognormally distributed, especially for electronic systems without a built-in test capability and for many other electromechanical systems.

Relationship with Other Engineering Disciplines
As discussed in this section, RAM engineering is closely related to several other engineering disciplines. The primary objective of reliability engineering is prevention of failure. The primary objective of safety engineering is prevention and mitigation of harm under both normal and abnormal conditions (see Section 3.1.11). The primary objective of logistics engineering is the development of efficient logistics support (see Section 3.1.6). Furthermore, RAM is also related to engineering disciplines such as affordability (see Section 3.1.2), resilience engineering (see Section 3.1.9), and reusability of products in a product line (see Section 4.2.4). The life cycle cost (LCC) of a system is highly dependent on reliability and maintainability, which are considered major drivers in support resources and related in-service costs (see Section 3.1.2).

Many of these not only have "failure" as common theme, but they may also use similar activities, albeit from different viewpoints. For example, an FMEA may be applicable to reliability, safety, and logistics engineering. However, a reliability FMEA will be different to a safety or logistics FMEA, due to the different objectives. Common to all disciplines is the necessity of early implementation during the system life cycle.

More information on RAM can be found in ANSI/GEIA-STD-0009 (2008), Barnard (2008), Blanchard (2004), Clausing and Frey (2005), O'Connor and Kleyner (2012), and Tortorella (2015).

3.1.9 Resilience Engineering

Definition Resilience Engineering is an approach that provides required capability when facing adversity.

Resilience is a relatively new term in SE, appearing in the 2006 timeframe and becoming popularized around 2010. Resilience typically subsumes survivability. The recent application of "resilience" to engineered systems has led to a proliferation of alternative definitions. While the details of definitions will continue to be discussed and debated, there is general agreement that resilience of engineered systems is the ability to provide required capability when facing adversity.

Key Concepts System development often focuses on system capability under nominal conditions. Resilience directs the SE focus to the system's ability to deliver capability when faced with adverse conditions. This perspective can be important to stakeholders but is sometimes overlooked. Resilience in the realm of SE involves identifying:

- the capabilities that are required of the system,
- the adverse conditions under which the system is required to deliver those capabilities, and
- the architecture and design that will ensure the system can provide the required capabilities.

It is important to emphasize that resilience focuses on providing the required capability—not necessarily with maintaining the architecture or composition of the system. While system continuity is one approach to achieving resilience, so is adaptability.

Elaboration
Scope of Resilience
The fundamental objectives of resilience are avoiding, withstanding, and recovering from adversity. In non-engineering contexts, resilience is often limited to the ability to recover after degradation. In the context of engineered systems, it is recommended that "avoiding" and "withstanding" adversity be considered in scope (Jackson and Ferris, 2016). Resilience, as does SE, applies to cyber-physical, organizational, and conceptual systems.

Scope of the Adversity
For the purpose of resilience, adversity is anything that might degrade the capability provided by a system. Achieving resilience requires consideration of all sources (e.g., environmental sources, human sources, system failure) and types of adversity (e.g., from adversarial, friendly, or neutral parties; adversities that are malicious or accidental; adversities

that are expected or not). Adversities may be issues, risks, or unknown-unknowns. Adversities may arise from inside or outside the system. Adversity may be a single event or may take the form of complex causal chain of conditions and events that stress the system over multiple periods of time.

Taxonomy of Resilience Objectives

Resilience, and engineering its achievement, can be facilitated by considering a taxonomy of its objectives. A two-layer objectives-based taxonomy includes:

- First layer, the *fundamental objectives* of resilience and
- Second layer, the *means objectives* of resilience.

The layers relate by many-to-many relationships (Brtis, 2016) (Jackson and Ferris, 2013).

Taxonomy Layer 1: Resilience can be said to equate to achieving its three *fundamental objectives*. These are:

- **Avoid**: eliminate or reduce exposure to stress.
- **Withstand**: resist capability degradation when stressed.
- **Recover**: replenish lost capability after degradation.

Taxonomy Layer 2: These fundamental objectives can be achieved through the pursuit of *means objectives*. Means objectives are not values or ends in themselves. Their value resides in their ability to help achieve resilience and its three fundamental objectives. The means objectives include:

- **Adaptive Response:** reacting appropriately and dynamically to the specific situation to limit consequences and avoid degradation of system capability.
- **Agility:** ability of a system to adapt to deliver required capability in unpredictably evolving conditions.
- **Anticipation:** establishing awareness of the nature of potential adversities, their likely consequences, and appropriate responses, prior to the adversity stressing the system.
- **Constrain:** limit the propagation of damage within the system.
- **Continuity:** ensuring the endurance of the delivery of required capability, while and after being stressed.
- **Disaggregation:** dispersing missions, functions, or system elements across multiple systems or system elements.
- **Evolution:** restructuring the system to address changes to the adversity or needs over time.
- **Graceful Degradation:** ability of the system to transition to a state that has acceptable, potentially limited capabilities.
- **Integrity:** the quality of being complete and unaltered (ISO 13008 (2022)).
- **Prepare:** developing and maintaining courses of action that address predicted or anticipated adversity.
- **Prevent:** deterring or precluding the realization of adversity.
- **Re-architect:** modifying the system architecture for improved resilience.
- **Redeploy:** restructuring resources to provide capabilities after stress.
- **Robustness:** the ability of a structure to withstand adverse and unforeseen events or consequences of human errors without being damaged (damage insensitivity) (ISO 8930 (2021)).
- **Situational Awareness**: perception of elements in the environment, and a comprehension of their meaning, and could include a projection of the future status of perceived elements and the risk associated with that status (ISO 17757 (2019)).
- **Tolerance:** the ability of a material/structure to resist failure due to the presence of flaws for a specified period of unrepaired usage (damage tolerance) (ISO 21347 (2005)).

- **Transform:** changing aspects of system behavior.
- **Understand:** developing and maintaining useful representations of required system capabilities, how those capabilities are generated, the system environment, and the potential for degradation due to adversity.

The SEBOK section on resilience provides a more extensive taxonomy of design, architecture, and operational techniques for achieving resilience.

Key Activities, Methods, and Tools

While resilience must be considered throughout the SE life cycle, it is critical that resilience be considered in the early life cycle stages: those that lead to the development of resilience requirements. Once resilience requirements are established, they can, and should, be managed along with all other requirements in the trade space through the system life cycle. As shown in Table 3.3, Brtis and McEvilley (2019) identify specific considerations that need to be included in the early life cycle activities.

TABLE 3.3 Resilience considerations

Business or Mission Analysis Process
• Defining the problem space includes identification of adversities and expectations for performance under those adversities.
• ConOps, OpsCon, and solution classes consider the ability to avoid, withstand, and recover from the adversities
• Evaluation of alterative solution classes consider the ability to deliver required capabilities under adversity
Stakeholder Needs and Requirements Definition Process
• The stakeholder set includes persons who understand potential adversities and stakeholder resilience needs.
• Identifying stakeholder needs identifies expectations for capability under adverse conditions, and degraded/alternate, but useful, modes of operation.
• Operational concept scenarios include resilience scenarios.
• Transforming stakeholder needs to stakeholder requirements includes stakeholder resilience requirements.
• Analysis of stakeholder requirements includes resilience scenarios in the adverse operational environment.
System Requirements Definition Process
• Resilience is considered in the identification of requirements.
• Achieving resilience and other adversity-driven considerations is addressed holistically.
System Architecture Definition Process
• Viewpoints selected support the representation of resilience.
• Resilience requirements significantly limit and guide the range of acceptable architectures. It is critical that resilience requirements are mature when used for architecture selection.
• Individuals developing candidate architectures are familiar with architectural techniques for achieving resilience.
• Achieving resilience and other adversity-driven considerations are addressed holistically.
Design Definition Process
• Individuals developing candidate designs are familiar with design techniques for achieving resilience.
• Achieving resilience and the other adversity-driven considerations are addressed holistically.
Risk Management Process
• Risk management is planned to handle risks and opportunities identified by resilience activities.

From Brtis and McEvilley (2019). Used with permission. All other rights reserved.

Content, Structure, and Development of Resilience Requirements

Brtis and McEvilley (2019) investigated the content and structure needed to specify resilience requirements. Resilience requirements often take the form of a resilience scenario. There can be many such scenario threads in the ConOps or OpsCon. The following information is often part of a resilience scenario:

- Operational concept/scenario name
- System or system element of interest
- Capability(s) of interest their metric(s) and units
- Target value(s) (required amount) of the capability(s)
- System modes of operation during the scenario (e.g., operational, training, exercise, maintenance, update)
- System states expected during the scenario
- Adversity(s) being considered, their source, and type
- Potential stresses on the system, their metrics, units, and values (Note: Stresses are a type of adversity. They are proximate forces or influences, directly affecting the system that can cause degradation of the system's ability to deliver required capability.)
- Resilience related scenario constraints (e.g., cost, schedule, policies, regulations)
- Timeframe and sub-timeframes of interest
- Resilience metric(s), units, determination method(s), and resilience metric target(s) (e.g., expected availability of required capability, maximum allowed degradation, maximum length of degradation, total delivered capability). Note: There may be multiple resilience targets (e.g., threshold, objective, As Resilient as Practicable (ARAP)).

Importantly, many of these parameters may vary over the timeframe of the scenario. Figure 3.7 notionally shows the required capability, the stress on the system, and the delivered capability as they vary as a function of time. A single resilience scenario may involve multiple stresses, which may be involved at multiple times throughout the scenario.

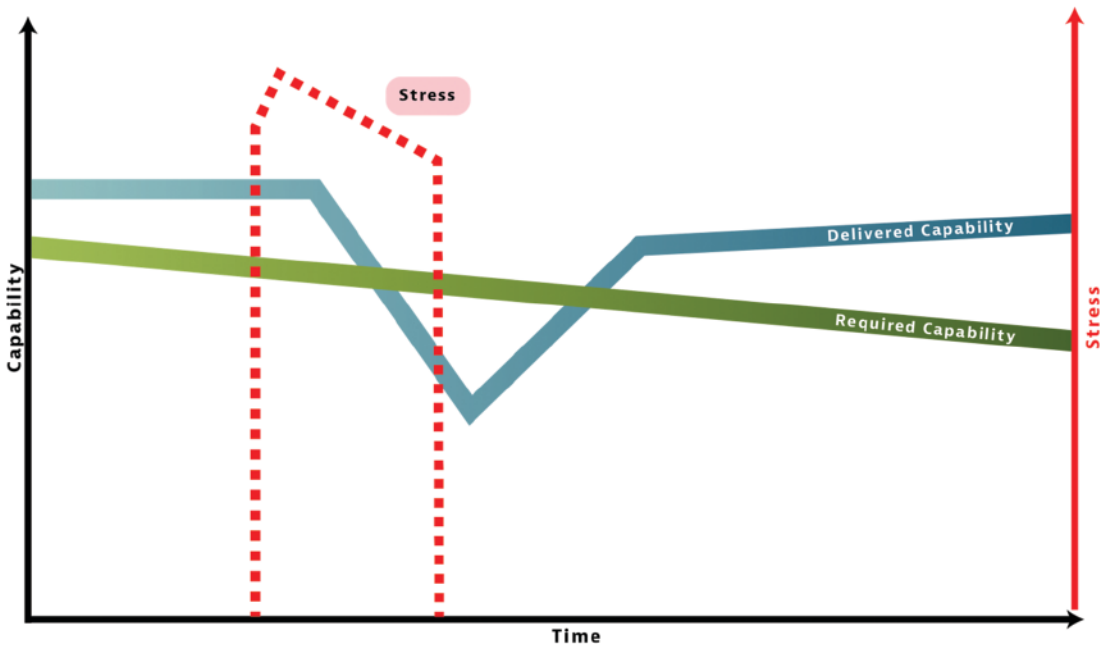


FIGURE 3.7 Timewise values of notional resilience scenario parameters**.** INCOSE SEH original figure created by Brtis and Cureton. Usage per the INCOSE Notices page. All other rights reserved.

Requirement Patterns for Resilience
An example of a natural language pattern for representing this information would be:

The **<system, mode(t), state(t)>** *encountering* **<adversity(t), source, type>**, *which imposes* **<stress(t), metric, units, value(t)>** *thus affecting delivery of* **<capability(t), metric, units>** *during* **<scenario timeframe, start time, end time, units>** *and under* **<scenario constraints>**, *shall achieve* **<resilience target(t) (include excluded effects)>** *for* **<resilience metric, units, determination method>**.

Here, "(t)" is used to indicate that the item may vary as a function of time.

System State Considerations
When a system encounters an adversity, the system may pass through a number of states, from a fully capable state to a state minimally acceptable to the system stakeholders. Intermediate states include damaged or partially capable states. The transitions between these states fall into three categories. The *robustness* category defines the transitions for which the system maintains its level of capability. These transitions include maintaining fully capable or partially capable states. The *tolerance* category includes passing from any higher level of capability to a lower level of capability (e.g., a degraded capability). The *recovery* category includes passing from any lower-level capability to a higher-level capability including the original fully capable state. The system should be designed applying design principles that will manage the transitions between states to result in context appropriate behavior. Guidance on techniques is provided in Jackson and Ferris (2013) and Brtis (2016).

Related Quality Characteristics
Resilience has commonality and synergy with a number of other QCs. Examples include availability, maintainability, reliability, safety, security, and sustainability. This group of quality areas are referred to as loss-driven areas because they all focus on potential losses involved in the development and use of systems (see Section 3.1.13). These areas frequently share: the assets considered, losses considered, adversities considered, requirements, and architectural, design, and process techniques. It is imperative that these areas work closely with one another and share information and decision-making in order to achieve a holistic approach that avoids unbalanced emphasis in any one area.

Further information and references on resilience (including the state-of-the-art) can be found in the resilience section of the SEBoK.

3.1.10 Sustainability Engineering

Definition Sustainability Engineering is an approach that supports the circular economy over its life.

Key Concepts Design for sustainability is defined as the process that considers environmental and social aspects as key elements in product design to reduce the harmful impacts of the product throughout its life cycle (Sharma, et al., 2020). It entails environmentally conscious decisions that promote responsible disposability via product recycling and materials reuse as options for the preservation of scarce material resources. Sustainability and disposability are critical components toward the circular economy, which is based on a production and consumption model that involves sharing, reusing, repairing, and recycling existing products and materials as much as possible, expanding the life cycle of products, minimizing waste and pollution, and creating a closed-loop system (Geissdoerfer, et al., 2020). These goals are consistent with the 17 Sustainable Development Goals that were adopted by all UN Member States in 2015, as part of the 2030 Agenda for Sustainable Development (Haskins, 2021).

Elaboration
Role of Sustainability and Disposability in SE
Addressing sustainability effectively in context is highly complex, requiring the integration of multiple disciplines in balancing a wide range of interdependent issues (Pearce, et al., 2012). Sustainability is essential given the significant

impacts that the SE processes, and the resulting systems, have had, and continue to have, on the environment. Achieving sustainability must include a holistic adoption of environmental stewardship in engineering activities (Alwi, et al., 2014) (Rosen, 2012).

The focus of environmental impact analysis is on potential harmful effects of a proposed system's development, production, utilization, support, and retirement stages. Concern extends over the full life cycle of the system, from the materials used and scrap waste from the production process, operation of the system, replacement parts, consumables and their packaging, to final disposal of the system.

Disposal analysis is a significant analysis area within environmental impact analysis. During System Architecture Definition and Design Definition (see Sections 2.3.5.4 and 2.3.5.5), one goal is to maximize the economic value of the residual system elements after useful life and minimize the amount of waste materials. Design for disassembly has become an important consideration in the design process so that the products are created in a way that minimizes destructive separation of system elements such that the material can be reused in future generations of products, remanufacturing, or recycling processes (Abuzied, et al., 2020). This may include designing for transformation (e.g., decomposition, biodegradation).

Key Activities, Tools, and Methods of Sustainability and Disposability

The ISO 14000 (2015) series of environmental management standards are an excellent resource for methods to analyze and assess industrial operations and their impacts on the environment. Attention to environmental regulations should be addressed in the earliest activities of requirements analysis. The Øresund Bridge (see Section 6.2.) is an example of how early analysis of potential environmental impacts ensures that measures are taken in concept, development, and production to protect the environment with positive results. Two key elements of the success of this initiative were the continual monitoring of the environmental status and the integration of environmental concerns into the requirements of the two countries.

Another effort in the ISO community is the development of a standard for Environmental Product Declarations (EPD), based on carbon footprints, as an indicator of the global environmental impact of a product expressed in carbon emission equivalents (He, et al., 2018). EPD and labeling, such as the Nordic Swan and Blue Angel, offer consumers assistance in their purchasing decisions. Methods associated with life cycle assessment (LCA), life cycle impact assessment (LCIA), life cycle optimization (LCO), and life cycle management (LCM) are increasingly sophisticated and supported by software (Avraamidou, et al., 2020).

Related QCs

Achieving a circular closed-loop system relies on integrating additional quality characteristics. Useful life extensions rely on reliability and maintainability (see Section 3.1.8) alongside efficient logistical support (see Section 3.1.6) and products designed to be resilient (see Section 3.1.9). Recovery of valuable resources after useful life is highly dependent on decisions made when considering manufacturability (see Section 3.1.7).

More information on Sustainability/Disposability can be found in the Journal of Cleaner Production (2023), the Journal of Environmental Management (2023), Wood et al. (2023), ICE (2023), and MDPI (2023).

3.1.11 System Safety Engineering

Definition System safety engineering is an approach that reduces the likelihood of harm to people, assets, and the wider environment.

Key Concepts The goal of system safety engineering is to reduce and mitigate hazards of systems to an acceptable level of risk. Engineered systems have safety risks; they are not 100% safe. The definition of what is acceptably safe, safety regulations, processes, and culture vary across different industries and countries.

System safety engineering is not limited to ensuring that the engineered system is acceptably safe. It includes minimizing the risks to everyone involved in the production, utilization, support, and retirement of the system, as well as

third parties who could also be affected by these activities. System safety engineering is about engineering the SoI, the wider socio-technical operational system, and the socio-technical (or even purely social) management system.

Safety is an emergent property of the engineered system in its real operational environment. How the system is used, maintained, and managed can have as big an impact on system safety as its inherent design. Understanding, and aligning, the mental models of designers, operators, and managers is critical.

Safety is managed by minimizing the hazards that can lead to an accident. This is either through reducing the likelihood the hazard will occur or minimizing the impact if it does. This is either through designing the hazard out, technical mitigations in the system, or procedural controls. This requires a mixture of suitably qualified people, effective processes, appropriate governance, and culture.

In-service systems need careful monitoring to ensure that design assumptions remain valid, no new hazards have been identified, and that operations/maintenance is as expected. Slow feedback loops and misaligned mental models can be particularly problematic, as issues can grow unseen for years before they appear with catastrophic impact.

Good systems safety engineering seeks to ensure operators do not misuse systems, leaders set the right tone and culture, and maintainers don't take shortcuts. System safety engineers and senior leaders are often accountable for predictable misuse of systems, failure to address poor behavior, and ineffective oversight of maintenance and operations.

Elaboration

Acceptably Safe

The safety regulations, processes, and culture vary across different industries and countries. What is acceptable in one industry and country may not be acceptable in another. There is a wide diversity in regulators, definitions, evidence, and perceived benefits that adds further complexity. Even the definition of what is "acceptably safe" varies. Typical perceptions of "acceptably safe" include:

- *"We have complied with the necessary [product] regulations."* This is generally accepted for simple, well understood, standardized system elements (e.g., electrical cable, bolts).
- *"We have evaluated all identified hazards and have mitigated each to be 'as low as reasonably practicable' (ALARP)."* This would be the typical approach adopted for complicated, safety critical civilian systems (e.g., a railway signaling system, passenger aircraft).
- *"We have evaluated all the identified hazards of the system, and they are either ALARP, or the level of hazard of the new system is less than the alterative (of not having it)."* This would be the typical approach adopted for military, medical, or emergency response systems (e.g., artillery, pacemaker).

Eliminating all safety risk is not possible; therefore, no system can be described as 100% safe. There may be unknown hazards and hazards that cannot be eliminated but are determined to be acceptable given the perceived benefit. Similarly, assuming that the system must be safe because there haven't been any accidents yet, or reported, is equally incorrect. There may have been near misses (see below), the hazardous functionality/performance may not have been used, or the effect/damage has not yet been recognized.

Emergence, Accidents, and Hazards

Safety is an emergent property of a system. It is not the sum of system element level safety or reliability. Rather, it is impacted by interactions between system elements and affected by the environment in which it is used. Many factors affect the level of safety risk (e.g., who uses and maintains a system, how they do it, in what environment). Safety is not a static property as systems (and the surrounding ecosystems) are dynamic and evolve. The designers' expectations of risks often differ from the system under test and evaluation and from the system in operation (which continues evolves over time). The mental models of the humans interacting with the system and the related processes are also dynamic and subject to change.

Most accidents result from more than a single causal factor. When an unmitigated hazardous situation does occur but does not result in harm, it is referred to as a *near miss*. Near misses serve as critical feedback on the system safety level in operation. Safety culture often determines how near misses are treated and responded to. However:

- Accident investigations often reveal dependencies between hazards exist, despite earlier beliefs that causal factors were independent. A failure to recognize dependencies when applying statistical methods can result in flawed safety decisions.
- A hazardous situation can occur without a system element failure. This could be because of a design error, an implementation error, or a misalignment between mental models of designer and operators or maintainers.

Regulators typically assess safety risk in terms of both:

- the *likelihood* of hazards occurring and leading to harm and
- the *severity* of the resulting harm.

Safety is managed by eliminating hazards where possible; and when not possible, by reducing risks to an acceptable level. When possible, design changes to eliminate potential hazards are the preferred options. The next preferred options are design mitigations to reduce the likelihood of hazards occurring. When designs changes or mitigations are not possible, other means are typically employed such as operational controls and limitations, maintenance inspections or activities, warnings via labeling, and training.

Hazards may result from a range of sources such as intrinsic, functional, socio-technical, or management/wider culture. Intrinsic hazards are typically caused by the material, or other design factors of the system elements used in the system. Functional hazards result from incorrect, unexpected, or undesirable functions or performance of the system. Socio-technical hazards result from interactions between the physical system and its operators and the wider environment. Finally, management/cultural hazards relate to the system and the wider management controls needed to realize and sustain the system.

Examples of safety hazards include:

- Interactions between system elements, the operating environment, and operators: A car on an icy road and an inexperienced driver is likely to result in an accident. Traction control, trained drivers, or not driving in icy conditions mitigates this hazard.
- Mistakes in system/system element requirements, design, manufacturing, or installation: A failure to specify the Maneuvering Characteristics Augmentation System (MCAS) system element as safety critical resulted in loss of two 737 Max aircraft (Cantwell, 2021).
- The system creates hazards in the wider SoI: Bull-bars / kangaroo bars reduce the risk of injury to a driver in vehicle to vehicle or vehicle to large animal collisions. However, they significantly increase the risks to pedestrians in vehicle to pedestrian collisions in urban areas (Desapriya, et al., 2012).
- The inherent material used in system elements: Asbestos or flammable substances present inherent hazards.
- Incorrect operation or maintenance: A failure to properly remove old wiring led to the Clapham junction rail accident (Hidden, 1989).
- Misaligned mental models between operators, maintainers, and designers: Prior to the Smiler accident, false alarms were a common occurrence. This led to operators believing all alarms were false positives. This resulted in alarms being overridden without investigation (Kemp and O'Neil, 2018).
- The activities undertaken to design, manufacture, test and maintain the system: The Piper-Alpha oil platform accident was caused by a failure to properly manage design changes, poor maintenance management, and poor contingency planning (Cullen, 1990).

Managing and Controlling Hazards

System safety engineering seeks to control hazards by:

- Understanding the system environment, wider SoI, proposed system, and how it will be used.

- Specifying the safety requirements for the system. System requirements flow from stakeholder needs and requirements and are derived into system element requirements (see Section 2.3.5.3). Maintaining traceability is key, as it may not be immediately obvious to a system element designer that a specific requirement is safety critical (see Section 3.2.3).
- Analyzing the potential safety hazards. There is a range of hazard analysis techniques looking at the system, and its functions, physical, process, and human interactions. An effective hazard analysis will use multiple techniques.
- Mitigating/controlling the known hazards, either by reducing the likelihood or severity of a hazard occurring. Approaches include removing the hazard entirely, designing-in passive or active controls to mitigate the hazard, or including operational or maintenance controls. A key element of including controls is ensuring effective feedback loops and recognizing the full control model beyond the engineered system (including the operating environment and management systems) (Leveson, 2011).
- Establishing a safety management system to ensure the system remains safe throughout its life cycle.

Establish an Appropriate Safety Management System

Each organization's safety management system needs to be tailored based on country, region, and industry/application considerations. Organizations need to understand the regulatory, operational, and physical environments that the systems they develop will be used. For example, the safety management system needed for a safety critical industry such as rail or aerospace would be inappropriate for consumer electronics.

The safety management system needs to: define the organizations approach to developing safe systems; manage the infrastructure, processes and information required to support system delivery; oversee the operations and maintenance of in-service systems; and ensure an effective safety culture.

The safety management system needs to be fully integrated into the wider organizations' business management systems. It needs to ensure that:

- Projects to deliver new systems are given clear safety objectives, measures, and targets.
- In-service systems are operated and maintained safely, with appropriate monitoring of changes to use, environment, and material state of the system.
- Incidents and near misses are reported without fear of retribution, and on-going monitoring and implementation of mitigation actions create an ongoing learning/improvement cycle (Dekker, 2014).

A key facet of the safety management system is the organization's safety and ethical culture, or "how we behave when no-one is looking" (see Section 5.1.4). Regular measurement of the safety is useful to track organization's safety culture (Hudson, 2001). Regular, frequent communications and stories of the behavior needed are necessary by senior and influential people to reinforce the safety culture (Kemp and O'Neil, 2018).

Establish and Run Projects with Safety at Their Core

Key issues to be aware of include:

- Ensuring that the tailoring of the SE approach is appropriate for the system under development.
- Ensuring that the safety engineering processes integrated as tightly as possible to the wider engineering and business processes, driving both operational efficiencies and reducing the risk of incorrect assumptions between the safety team and the wider project.
- Accepting the need to make assumptions, but recognizing that when the assumptions turn out to be incorrect and appropriate rework will be required (and accepting the cost and time impact of the rework).
- Ensuring the selection of an appropriate life cycle model. As Akroyd-Wallis (2018) notes, direct adoption of Agile software techniques to safety critical systems may be problematic. SE Practitioners should exercise caution when employing agile for safety critical systems.

- Terminating projects when they can no longer meet their safety objectives at a reasonable cost, timescales, or performance.

Embed Safety in the Core of the SE Process
Specific safety engineering considerations include:

- Ensuring Business or Mission Analysis (see Section 2.3.5.1) includes a high-level assessment of hazards and that analysis of alternatives includes their inherent safety potential
- Including a comprehensive hazard analysis during Stakeholder Needs and Requirements Definition (see Section 2.3.5.2) to identify potential hazards with the system being developed and the wider operational capability the SoI will be deployed into
- Ensuring that key safety functions and safety performance is captured during System Architecture Definition (see Section 2.3.5.4). A safety viewpoint will help ensure traceability from high level needs down to system element requirements.
- Where the system is to be part of a wider SoS, ensuring that system functions/performance necessary to mitigate SoS hazards are captured as safety requirements (see Section 4.3.6).
- Ensuring that verification and validation of safety requirements is sufficiently rigorous to meet the agreed levels of "acceptably safe."
- Ensuring that hazards that are managed by Operational or Maintenance (see Sections 2.3.5.12 and 2.3.5.13) activities are clearly embedded in relevant processes and are clearly communicated, precisely defined, and reasonable.
- Ensuring that the assumptions in the safety case remain valid through life. (Is the environment as expected? Have the operators (and their mental models) changed? Has the use of the system changed? Is maintenance being done as required and is the maintenance as effective as planned? Are there any new potential new hazards being seen? Is the frequency and severity of hazards as expected?)

Ensure the System Is Delivered Safely
System safety engineering needs to ensure that systems can be designed, built, and verified safely. The systems that manufacture, test, and maintain complicated and complex systems can be as hazardous as the SoI itself. The traditional split between product safety and occupational safety is becoming less clear as:

- Development becomes more agile.
- Organizations shift from product to service delivery.
- Increased use of non-expert operators and maintainers.

Suitably Qualified and Experienced Personnel Drive Safety Performance
The effectiveness of safety management system is driven by the people who work within it. Good people may build safe systems despite poor processes and tools. Good processes and tools cannot make up for poorly qualified people.

Effective safety practitioners need to be numerate, critical thinkers, and system thinkers who understand the technologies being used, the environment the system will be deployed into, and the mental models of operators and maintainers. In addition, they need the influencing and persuasion skills to convince others of the right approach to take and the moral courage to "say no" when necessary (see Section 5.1.2).

Safety practitioners need to be in the room when key decisions are made. They need to lead the safety decision making, capturing key information in an audit trail. If they are involved too late, organizations can be left with systems that cannot be deployed. This happened to the UK Air Force when acquiring eight Chinook Mk3 helicopters (NAO, 2008).

For more illustrations on the importance of system safety, refer to Case 1 (Radiation Therapy - The Therac-25) from Section 6.1 and Case 5 (Artificial Intelligence in Systems Engineering – Autonomous Vehicles) from Section 6.5.

3.1.12 System Security Engineering

Definition System Security Engineering is an approach that identifies, protects from, detects, responds to, and recovers from anomalous and disruptive events, including those in a cyber contested environment.

Key Concepts System Security Engineering (SSE) is focused on ensuring a system can function under anomalous and disruptive events associated with misuse and malicious behavior. SSE involves a disciplined application of SE principles in analyzing security threats and vulnerabilities to the system and assessing and mitigating security risks to assets of the system during the life cycle. It blends technology, management principles and practices, and operational rules to ensure sufficient protections are available at all times.

Sources of potential anomalous and disruptive events (threats) are many and varied. They may emanate from external sources (e.g., theft, denial of service attacks, power interruptions) or may be caused by internal forces (e.g., user actions, supporting systems). A disruption may be unintentional (misuse) or intentional (malicious) in nature

Physical security protects a system from unauthorized access, misuse, or damage caused by physical actions and events such as theft, vandalism, and intrusion. Protecting physical facilities, equipment, resources, and personnel can involve the use of multiple layers of interdependent systems such as surveillance and intrusion detection systems, deterrent systems, security guards, protective barriers, locks, and access control. Hardware devices can employ anti-tampering features to detect unauthorized opening or altering of the packaging, either to ensure the content is authentic or to trigger actions to protect sensitive information in the devices.

As our world becomes increasingly digital, both hardware and software systems are increasingly at risk for disruption or damage caused by threats taking advantage of digital technologies. Integrating and implementing systems security using SSE approaches is the most efficient and effective way to ensure that security is addressed at each stage of the life cycle and becomes part of the overall SE solution instead of being done separately and isolated from other SE activities (NIST SP 800-160 Vol. 1, 2022 and NIST 800-160 Vol. 2, 2021). SSE provides the needed complementary engineering capability that extends the notion of trustworthiness to deliver trustworthy secure systems, which are less susceptible to the effects of modern adversity such as attacks orchestrated by an intelligent adversary (NIST SP 800-160 Vol. 1, 2022).

Cybersecurity generally refers to the confidentiality, integrity, and availability of information assets. Security management includes controls (e.g., policies, practices, procedures, organization structures, and software). Trustworthiness is a concept that includes privacy, reliability, resilience, safety, and security, therefore worthy of being trusted to fulfill whatever critical requirements may be needed for a particular system element, system, network, application, mission, business function, enterprise, or other entity (NIST 800-160 Vol. 2, 2021).

Elaboration SSE practitioners should have skills, expertise, and experience in multiple areas. Examples include security requirements, security architecture views, threat assessment, networking, security technologies, hardware and software security, security test and evaluation, vulnerability assessment, penetration testing, and supply chain security risk assessment. A major challenge in managing engineering projects is unclear security roles, responsibilities, and accountability. To assist in the security role development and understanding responsibilities, an SE/SSE roles and responsibilities framework can be used to break down tasks into a matrix format that enables the SE practitioner to understand the role contributions and identify the types of artifacts created by the execution of the SE life cycle processes. (Nejib, et al., 2017)

Through NIST 800–160 Vol. 1 (2022) and Vol. 2 (2021), it has been determined that the best way to integrate cybersecurity into systems is through an SE process. NIST 800–160 Vol. 1 (2022) and Vol. 2 (2021) are based on ISO/IEC/IEEE 15288 (2023) and this handbook. They use the same terminology so that both SE and SSE practitioners can understand the key relationship that exists between the two disciplines. There is a direct correlation between SE and SSE, and SE practitioners need to understand and incorporate security components into each SE life cycle process. Table 3.4 shows an example of how the SSE technical processes in NIST SP 800–160 Vol. 1 (2022) can be reused and referenced by SE, SSE, and other disciplines practitioners. Specifically, Table 3.4 is an example of the Implementation process (see Section 2.3.5.7) breakout defined with extensions for SSE to include the purpose, outcomes, activities and

TABLE 3.4 Implementation process breakout

Implementation Process Breakout	
Purpose	• Realize the security aspects of the system element • Results in a system element that satisfies specified system security requirements, architecture, and design
Outcomes	• Security aspects of the implementation strategy are developed • Security aspects of implementation that constrain the requirements, architecture, or design are identified • Security system element • System elements securely packaged and stored • Enabling systems or services needed for security aspects of implementation • Traceability of security aspects of implemented system elements
Activities and Tasks	1. Prepare for the security aspects of implementation 2. Perform the security aspects of implementation 3. Manage results of the security aspects of implementation
Inputs	Security strategy, plan, traceability, requirements, design, architecture, secure system elements, assurance evidence, assurance results, and anomalies report
Responsible and Supporting Roles	Responsible: Systems Security Engineer (SSE) Supporting: Program Manager (PM), Chief Engineer (CE), Systems Engineer (SE), Systems Architect (SA), and Test Engineer (TE)

From NIST 800–160 Vol. 1 (2022). Used with permission. All other rights reserved.

tasks, inputs, and responsible and supporting roles. This same format was used to breakout each of the technical SSE processes in NIST 800–160 Vol. 1 Rev. 1 (2022) and the SE Technical Processes (see Section 2.3.5) to build an understanding of the relationships between the SE and SSE processes.

Case Study 3, Cybersecurity Considerations in Systems Engineering-The Stuxnet Attack on a Cyber Physical System (see Section 6.3) provides an example of the importance of system security.

3.1.13 Loss-Driven Systems Engineering

Loss-Driven Systems Engineering (LDSE) is the value adding unification of the QCs that address the potential losses associated with developing and using systems (Brtis, 2020). SE methodologies often focus on the delivery of desired capability. As a result, SE methodologies are largely capability-driven, and may not provide integrated attention to the potential losses associated with developing and using systems. Loss and loss-driven QCs are often considered in isolation—if at all. Examples of loss-driven QCs include resilience, safety, security, sustainability/disposability, and availability.

There is significant commonality and synergy among the loss-driven QCs, which needs to be leveraged. To do this, work on the loss-driven QCs should be collaborative on:

- the adversities considered,
- the weakness, defects, flaws, exposures, hazards, and vulnerabilities considered,
- the assets and losses considered, and
- the coping mechanism considered.

Further, SE practitioners should:

- elicit, analyze, and capture loss-driven requirements as an integrated part of the overall stakeholder and system requirements development,
- make architectural and design decisions holistically across the loss-driven QC areas, and
- integrate the management of risks associated with all loss-driven areas into the project's risk management activities.

3.2 SYSTEMS ENGINEERING ANALYSES AND METHODS

Part II of this handbook provided a set of SE life cycle processes used across the system life cycle. Each process contains a set of process activities and elaborations in the context of that specific life cycle process. This section provides insight into topics, techniques, and methods that cut across the SE life cycle processes, reflecting various aspects of the concurrent, iterative, and recursive nature of SE.

3.2.1 Modeling, Analysis, and Simulation

Overview and Purpose The INCOSE Systems Engineering Vision 2035 (2022) predicts that "The future of systems engineering is model-based, leveraging next generation modeling, simulation, and visualization environments powered by the global digital transformation, to specify, analyze, design, and verify systems. High fidelity models, advanced visualization, and highly integrated, multidisciplinary simulations will allow SE Practitioners to evaluate and assess an order of magnitude more alternative designs more quickly and thoroughly than can be done on a single design today."

The essential artifact of modeling, analysis, and simulation (MA&S) is an explicit model, an idealized representation of one or more aspects of the as-is or to-be SoI. Systems Modeling and Simulation has been defined (NAFEMS and INCOSE, 2019) as the use of interdisciplinary functional, architectural, and behavioral models (with physical, mathematical, and logical representations) for all life stages.

The terms "modeling," "analysis," and "simulation" are sometimes used interchangeably. However, they clearly refer to distinct activities. *Modeling* is the conception, creation, and refinement of models. *Analysis* is the process of systematic, reproducible examination to gain insight. *Simulation* is the process of using a model to predict and study the behavior or performance of the SoI—for aspects represented in the model. Simulation is often performed to support a particular kind of analysis, but not all analysis is performed through simulation. In the classification of models presented below, two major types of models are distinguished: *physical models*, and *digital models*. The SE discipline primarily makes use of digital models, since they provide many benefits to the SE processes in a timely and affordable manner, in particular during the early life cycle stages. Other engineering disciplines make use of both physical and digital models throughout the product life cycle. Although sometimes the term "simulation" is used in conjunction with a *physical model*, in this section *simulation* always involves a *digital model*, and any examination involving a *physical model* is always a *test*.

For effective MA&S, digital models are often parameterized to enable analysis or simulation of multiple configurations or situations with one model. Each configuration is typically defined by grouping selected parameter values in an *experiment*, see also Minsky (1965). Alternatively, the terms "analysis case," "load case," or "verification case" are used, depending on the application domain or convention. An *experiment* specifies the purpose of the analysis or simulation as well as the combination of target scenario, environment, initial and boundary conditions, and any other user-defined parameters. Figure 3.8 shows a typical workflow.

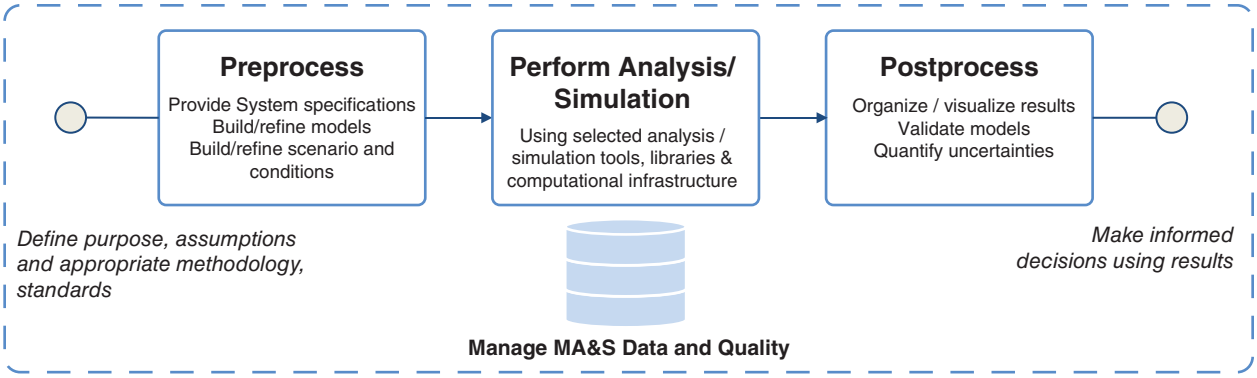


FIGURE 3.8 Schematic view of a generic MA&S process. INCOSE SEH original figure created by the NAFEMS-INCOSE Systems Modeling and Simulation Working Group (SMSWG)). Usage per the INCOSE Notices page. All other rights reserved.

MA&S links directly to the Core Competency "Systems Modeling and Analysis" in the INCOSE SE Competency Framework (INCOSE SECF, 2018). MA&S supports all Technical, Management and Integrating Competencies. Appropriate MA&S practices can clearly support SE professionals (individuals and teams) to perform better SE, more efficiently.

MA&S Related to Life Cycle Processes Creation and refinement of descriptive models in the Business or Mission Analysis process (see Section 2.3.5.1) and the Stakeholder Needs and Requirements Definition process (see Section 2.3.5.2) can be used to ensure that the business or mission proposition is understood correctly, the problem is understood correctly, is specified at the appropriate level of detail, and is fully shared with the stakeholders. MA&S can be used in the System Requirements Definition process (see Section 2.3.5.3) to flow-down the system requirements to system elements. This may include models that specify functional, interface, performance, and physical requirements, as well as other nonfunctional requirements (e.g., reliability, maintainability, safety, and security). In addition to bounding the system design parameters, MA&S can also be used to validate that the system requirements reflect stakeholder needs and requirements before proceeding with subsequent life cycle processes.

MA&S can be used in the System Architecture Definition process (see Section 2.3.5.4) concurrently with the Design Definition process (see Section 2.3.5.5) to synthesize and define alternative system concepts, compare and evaluate candidate options, and enable discovery of the best architecture and design, including the integration with other systems and unambiguously defining the system's capabilities and the value it is expected to deliver to its stakeholders (e.g., in the form of MOEs and MOPs). MA&S is often used extensively to realize an iterative model-based or model-driven design workflow. In many application domains, it is much more cost—and schedule-effective to perform analysis and simulation with digital models than to prototype with physical models. Digital models also allow for full and continuous access to all model parameters and properties, which is often infeasible with physical models. MA&S lends itself to fast iterations between problem specification, architectural design, detailed design, and V&V, as well as between system elements at different levels of decomposition. Using MA&S in the System Analysis process (Section 2.3.5.6), related system analyses can be used to explore a trade space by modeling alternative system solutions, or even generate many candidate solutions, and assessing the impact of critical properties such as mass, speed, energy consumption, accuracy, reliability, and cost on the overall adequacy and performance.

MA&S can be used in the Implementation process (see Section 2.3.5.7) to support definition, understanding and prediction of behavior for various aspects of the enabling production (manufacturing) and supply chain processes for the envisaged SoI. Models that reflect the "as produced" state of the SoI can be used to develop production facilities (factory) and "digital twins." MA&S can be used in the Integration process (see Section 2.3.5.8) to support integration of the elements into a system, as well as in the Verification process (see Section 2.3.5.9) to support verification that the system satisfies its requirements. This often involves integrating lower-level hardware and software design models with system-level design models, thereby allowing verification that the system requirements are satisfied. Systems integration and verification may also include replacing selected hardware and design models with actual hardware and software products to incrementally verify that system requirements are satisfied: so-called hardware-in-the-loop and software-in-the-loop testing. In cases where testing is impossible or carries prohibitive cost, verification of the SoI can be done by analysis or simulation using high fidelity digital models. Models can be used to simulate relevant operational environments where actual environments are unattainable, too costly, or not reproducible. Simulation can use observed data as inputs for computation of critical parameters that are not directly observable.

In the Operation process (see Section 2.3.5.12), MA&S can support definition, understanding, and prediction of behavior for various aspects of the envisaged or actual operation of the SoI, to help train (future) users to interact with the system, and to develop training material. Models may form a basis for developing a simulator of the system with varying degrees of fidelity to represent user interaction in different usage scenarios. Models and simulators can also be used to perform dry runs to prepare for complex or risky operations with the real, deployed system. In the Maintenance process (see Section 2.3.5.13), models that reflect the "as maintained" state of deployed systems can be used to develop "digital twins," possibly for individual deployed systems. Such models can be connected to data acquisition in the field and provide valuable insight and support for health monitoring and preventive maintenance. They can also be used to plan system upgrades and evolutions. MA&S in the Disposal process (see Section 2.3.5.14) can be

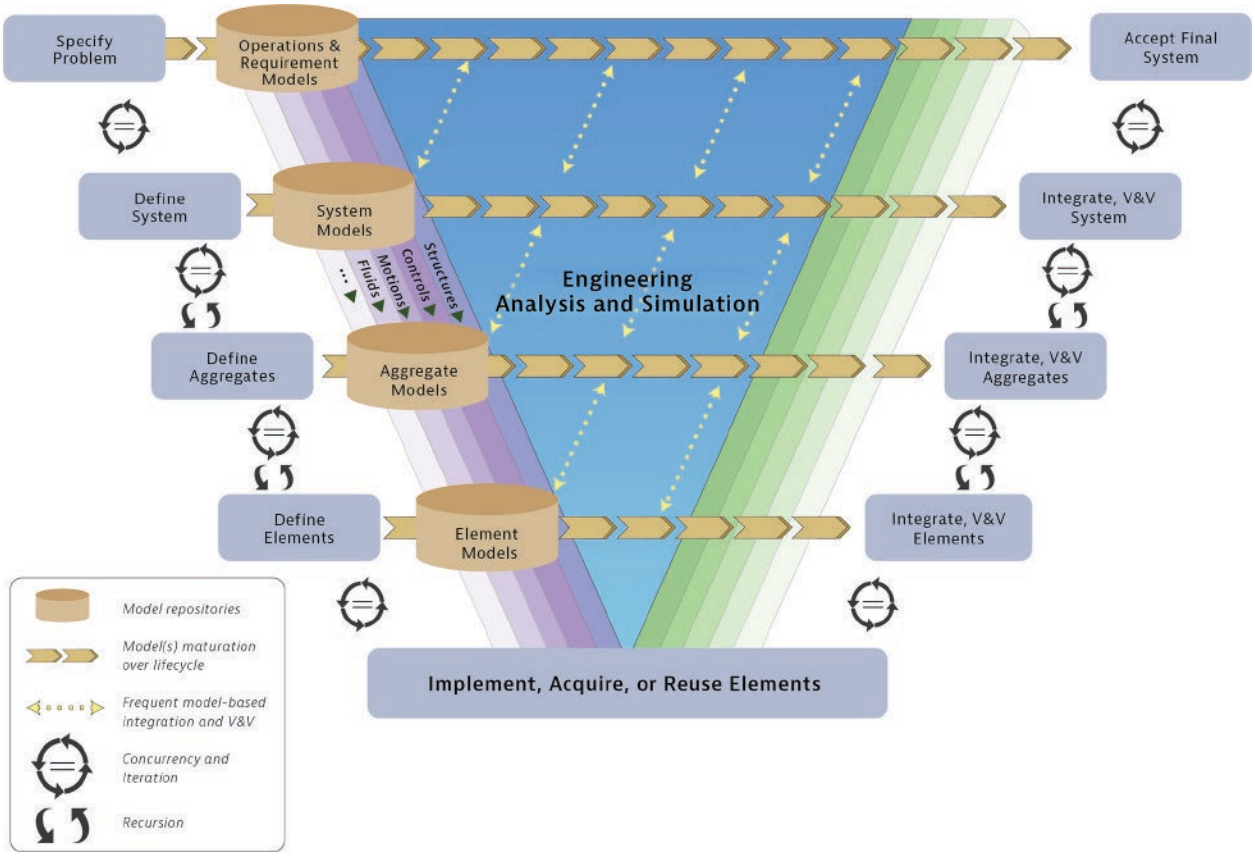


FIGURE 3.9 System development with early, iterative V&V and integration, via modeling, analysis, and simulation. Derived from NAFEMS and INCOSE (2019), based on NDIA, et al. (2011). Used with permission. All other rights reserved.

used to predict and monitor system disposal. MA&S also enables model-based iterations between the development processes mentioned above, as depicted in Figure 3.9.

Cross-Cutting MA&S There are several cross-cutting uses of MA&S that are not tied to a particular life cycle process, including:

Characterizing an existing system—Existing systems may be poorly documented (in whole or in part). Modeling such a system can provide a concise way to capture its architecture and design. This model can then be used to facilitate maintenance of the system or its further evolution.

Knowledge transfer within teams—MA&S enables the creation and maintenance of more precise, elaborate, and consistent specifications, including the rationale behind many requirements or design choices. Capturing specifications in rich models helps to mitigate the risk of loss of knowledge in case of team changes in long-duration projects.

Automated mapping and transformation—Digital models can be transformed by declarative or procedural algorithms (i.e., automated generation of a new or modified digital model from an existing one). Model transformations are a very powerful means to increase the value of model-based engineering (e.g., convert model formulations from one modeling language to another, move from a systems architecture to a (partial) software architecture, package a model for use outside the owning organization by encapsulating and protecting its intellectual property, creation of a surrogate model from a much more detailed discipline-specific engineering model). The value of transformation increases even more when it can be made bi-directional.

Knowledge capture and system design evolution—MA&S can be an effective means for capturing knowledge about a system as part of the Knowledge Management process (see Section 2.3.3.6). Established modeling can help transform tacit into explicit organizational knowledge. MA&S in projects enables identification and capture of reusable patterns or modules from problem and solution models that have proven their worth. Catalogs of such reusable model patterns and modules can become important assets for organizational knowledge management.

Benefits MA&S have many advantages including:

Separation of representation and presentation—Models capture representations of an SoI. MA&S tooling can then be used to maintain an "authoritative source of truth" and produce many different presentations, or views, of the model(s) that are *correct-by-construction* and enable effective communication with all engineering and non-engineering actors (humans and machines). The model-based approach can thus overcome the main problem of the document-centric approach in which representation and presentation are often combined in single information containers, which leads to a lot of information duplication and therefore cumbersome maintenance and potential errors. This is one of the most important advantages of the model-centric over the traditional document-centric approach.

More explicit SE—SE Practitioners can use MA&S to systematically check their own thinking, assumptions, and decision making with quantitative analyses. They can capture rationales and decisions in an accessible, traceable, consistent way.

Better problem specification—The stakeholder needs and requirements can be refined and formalized as an integral and structured set of goals, assumptions, requirements, constraints, actors, typical usage scenarios, and critical capabilities, with full traceability between all model elements. Anticipated system behaviors and performances can be explored and vetted with the stakeholders before proceeding with the development of an actual solution and committing significant resources.

Rigorous, well-documented design—MA&S facilitates development of solutions in a more rigorous and consistent way which leads to higher quality specifications. Systematic design space exploration and structured trade-studies become possible. Interfaces can be defined rigorously. Simulation with digital models enables design experimentation and optimization that is impossible, not affordable, or not on time with physical models. Technical and business decision-making can also be integrated into the model repositories for future consultation.

Early V&V to reduce risk—Early validation and verification of solutions with respect to the problem specification can be performed. This enables stakeholders to be informed of the implications of their preferences, provides perspective for evaluating alternatives, and builds confidence in the solution as it develops. Systematic, regular checking of interfaces and actual interconnections is feasible. It also allows catching issues early in the life cycle, when mitigation is affordable and change of scope is still feasible. The ability to detect limitations and incompatibilities early in a project helps avoid cost and schedule overruns in later life cycle stages.

Multi-user collaboration—Use of modern MA&S tooling allows for multi-user and multi-discipline collaboration with integrated configuration and version control, including splitting work into distinct parallel branches and merging results back into a main branch, using well-established workflows. Once deployed, this workflow is much more sophisticated and effective than what could be achieved with a document-centric approach.

Better change impact assessment—Modeled specifications with traceability (see Section 3.2.3) and MA&S tooling allow for change impact assessments by highlighting the consequences of a considered change.

Improved mastering of complexity—The value of MA&S increases with the complexity of the SoI, be it functional or physical. MA&S is a means to master greater complexity by structuring, refining, evaluating, and sharing all information within integrated project teams. On-demand multiple views with dynamic filtering to a suitable level of detail are possible. Quick views to share and capture information for effective communication with other actors, such as non-engineering disciplines and stakeholders, become more feasible and affordable.

Better team planning and handover—Development, deployment, and operational staff can more easily comprehend the design specifications, appreciate imposed limits from technology and management, and ensure an adequate degree of sustainability. Adequate, accurate, and timely MA&S helps an organization and its suppliers to plan and put in place the necessary and sufficient personnel, methods, tools, and infrastructure for system realization.

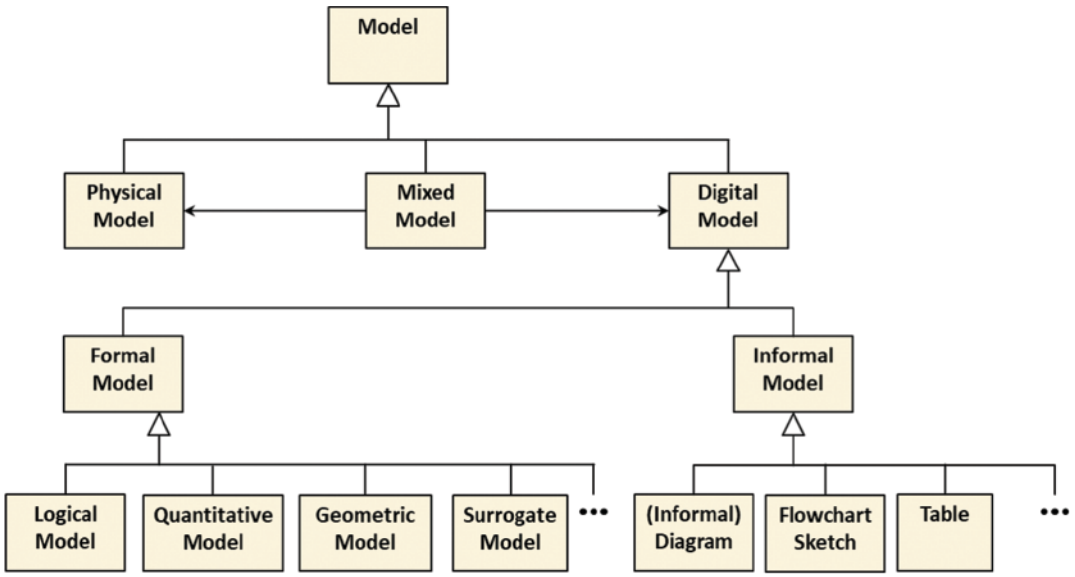


FIGURE 3.10 Illustrative model taxonomy (non-exhaustive). INCOSE SEH original figure created by the NAFEMS-INCOSE Systems Modeling and Simulation Working Group (SMSWG) derived from Friedenthal. Usage per the INCOSE Notices page. All other rights reserved.

Enable efficient maintenance and traceability—Digital models (supported with appropriate tools) enable bookkeeping of SE artifacts as they evolve through the life cycle stages in a reliable, consistent, traceable, and timely manner, while also providing hyperlinked navigation.

Flexible and repeatable querying—A rigorous MA&S approach enables gaining insight into many different aspects by querying the models with pertinent questions (what-if, impact of change, etc.) and getting answers efficiently, in support of decision making. Queries can generate many different views from the modeled information that address concerns of selected stakeholders. Query formulations can also be persisted for reuse.

Classifying and Characterizing Models There are many different kinds of models to address different system aspects and different kinds of systems. Generally, a specific type of model focuses on some subset of the system characteristics, such as timing, process behavior, measures of performance, interfaces, and connections. It is useful to classify the types of models to assist in selecting the appropriate one. Figure 3.10 shows one possible (non-exhaustive) taxonomy as an example.

Physical model—A physical model represents (aspects of) a system with real parts. Examples are a physical mockup, a scaled model airplane, a wind tunnel model, and a 3D-printed scale model from a digital model specification (the latter could be considered a physical view of a digital model). If simulation is performed with a physical model, it is typically called a *test*.

Digital model—Digital models can have many different expressions to represent (e.g., a system, entity, phenomenon, or process), each of which may vary in degrees of formalism. Therefore, the next level of classification is between informal and formal models.

Formal models—A formal model is expressed in a machine-readable language with explicitly defined semantics. The language may be textual and/or graphical, but with only one way of interpretation. Formal models can be further classified as logical, quantitative (i.e., mathematical), geometric, or surrogate models. A logical model, also referred to as a descriptive model or a conceptual model, represents logical relationships about the system such as whole–part relationships, interconnection relationships between elements, or precedence relationships between activities, to name

a few. Logical models are often depicted using network graphs (with nodes and edges) or tables. A quantitative model represents quantitative relationships (e.g., mathematical equations) about the system or its elements that yield numerical results. A geometric model represents the geometry, geometric shapes, and spatial relationships of the system or any of its (physical) elements. A surrogate model is a reduced model that is derived from a higher fidelity, more detailed model using a data-driven, typically automated, transformation. The goal (and challenge) is to create a surrogate that adequately represents essential aspects of the modeled system while requiring substantially less computational resources. Surrogate models then enable running large numbers of (parameterized) experiments in order to facilitate design exploration, optimization, or validation.

Informal models—An informal model is expressed using some convention understood by humans, where the convention is defined casually without formal semantics. The model does not need to be machine-readable. An informal model can be created by hand or with simple tools (e.g., word-processing, spreadsheet, diagramming, mind-mapping). While such informal representations can be useful, they often lack the rigor to be considered a type of model that is truly usable for MA&S for non-trivial systems. Informal model presentations may be used as views that are generated from or ingested into formal models in order to communicate with people not familiar with the notation.

Mixed models—A mixed model is a combination of physical and digital models.

In addition to a selected type of model, any model can be further characterized for its intended purpose through the following three characteristics:

- The *model breadth* reflects what aspects of the SoI—and possibly its (actual or intended) environment(s)—are represented, and to what extent.
- The *model granularity* characterizes the amount of visible detail captured in the model, in terms of the represented depth of system decomposition as well as the represented level of details of individual system elements.
- The *model fidelity* indicates how accurately the model represents the real-world system. Where applicable, this includes the computational precision to be achieved and the discretization scheme to be used.

The type of model and the model characteristics must be balanced against project needs and resources. Another important aspect of modeling is to explicitly state the assumptions and limitations that almost inevitably apply to any model.

Model Interoperability Since the development of complex systems requires collaboration between all project members and disciplines, it is very important to have the ability to exchange and share models as well as analysis and simulation results across disciplines, projects, organizations, and life cycle stages. This is also referred to as *digital interchange*. In most projects and (extended) enterprises it is not possible to standardize on a single set of tools. The alternative is to develop and utilize open, tool-independent standards that enable information exchange and sharing. There is an increasing awareness and consensus between user communities and tool developers on the merit of international royalty-free standards. Standards can be categorized in terms of how MA&S is supported. The main categories are:

- Standardized data exchange file,
- Application programming interface (API),
- Modeling language, and
- Process.

Data exchange files are used for on-demand transfer of complete models or results. APIs usually support more fine-grained data access and sharing, often implementing a service-oriented software architecture. Modeling languages can be graphical, textual or both, and are used to standardize the way of expressing a model. Process standards specify (aspects of) the MA&S processes. Most modeling languages do not prescribe a particular methodology to be followed. This flexibility is a feature of a general-purpose modeling language that enables economies of scale for implementations

which are in the interest of the SE community as a whole. However, in order to align how SE practitioners in a team, organization, or application domain approach MA&S, a methodology is needed. A methodology provides guidance and examples on how to organize MA&S over a typical system life cycle, how to structure model artifacts, as well as what stages and milestones to respect. A methodology can also capture proven modeling patterns and checklists, as well as good practices in general. For further details se Section 4.2.1 or consult the OMG MBSE Wiki (2023).

Tools For physical models, the tools are generally the same as those used for production of the final SoI, plus general or dedicated test facilities. For digital models, the tools are typically MA&S software applications running on general-purpose digital computers. For some computationally intensive applications, HPC (High Performance Computing) facilities may be needed. A MA&S software application may consist of one integrated tool or a set of tools that each implement part of the needed capabilities. The typical features of such tools are a graphical user interface with a hierarchical model structure browser, palettes of model constructs, a graphical and/or textual editor for creation and modification of the model, and multiple views for visualization, reporting, diagnostics, etc. The tool typically checks on-the-fly for adherence to the supported modeling formalism or language. If analysis or simulation support is included, there are also model execution views. For further details consult INCOSE SETDB (2021) or NAFEMS (2021).

Modeling Quality and Metrics The quality of a model should not be confused with the quality of the design that the model represents. A perfect model can represent a bad design. On the other hand, a low-quality model can in principle represent a good design, although that is not very useful.

A completed model or simulation can be considered a system or a product in its own right. Therefore, the general steps in the development and application of a model are closely aligned to the SE processes described within this handbook. MA&S needs to be planned and tracked, just like any other developmental effort. An essential good practice is to define clearly the purpose and intended life cycle of any (type of) model upfront. In particular, verification and validation of the MA&S methods, procedures, and infra-structure themselves are essential to ensure that the resulting models, analyses, and simulations possess the required quality and credibility that make them "fit for purpose" in an application domain or project. The required rigor of the approach depends on the criticality of the SoI. As an example, the US DoD Modeling and Simulation Enterprise has developed comprehensive guidance on Verification, Validation and Accreditation (VV&A, 2021).

A valuable feature of digital models is that they are amenable to many other kinds of computation than pure analysis or simulation. This enables the assertion of many modeling metrics such as:

- compliance with design or certification rules, including naming conventions, and associated model quality requirements,
- structural consistency of the system architecture,
- compliance of system element interconnections with interface specifications,
- coverage, consistency, and completeness of traceability, such as requirements satisfaction and verification, as well as function allocation,
- consistency and completeness of logical to physical architecture mapping and allocation,
- statistics that assist in monitoring and establishing specification maturity, uncertainty quantification, and resource planning,

MA&S Industrial Practice A big driver for the adoption of MA&S via all engineering disciplines is the trend that complex systems are becoming more and more software intensive. Analysis and simulation using digital models is much more economical and scalable than prototyping and testing with physical models, especially in the earlier stages of the life cycle. In the later stages, a mixed approach is often used. An example of the latter is an incremental hardware-in-the-loop approach in performing dynamic simulations. When it can be justified, verification through a purely digital model may be used.

Since a major responsibility of SE is to regard the system as a whole and coordinate between all disciplines in a multidisciplinary team, it follows that SE MA&S must interoperate at some level with the modeling and simulation of each of the other engineering disciplines in a team. As shown in Figure 3.11, the trend is to use an integrated system model to ensure information consistency between all engineering disciplines through a hub-and-spokes pattern, where a system model repository forms the hub.

A different way to look at multidisciplinary MA&S coordination is shown in the life cycle view presented in Figure 3.12. The information needed by two or more disciplines in a project team is shared via the integrated system model repository, which acts as the "authoritative source of truth." Within a project, there is then one authoritative repository. The one authoritative repository is a logical concept that may be implemented as a federation of distributed

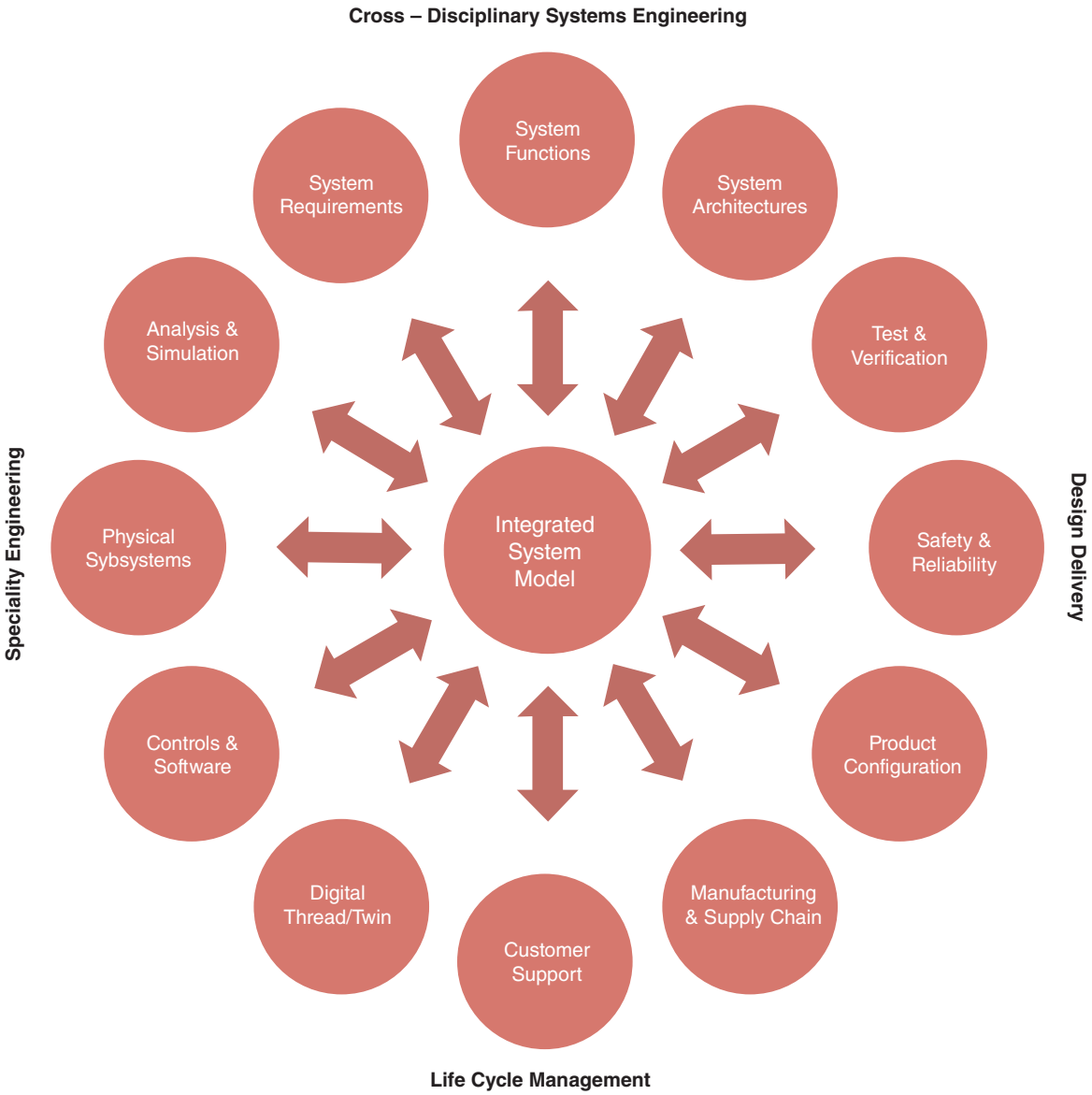


FIGURE 3.11 Model-based integration across multiple disciplines using a hub-and-spokes pattern. Derived from NAFEMS and INCOSE (2019). Used with permission. All other rights reserved.

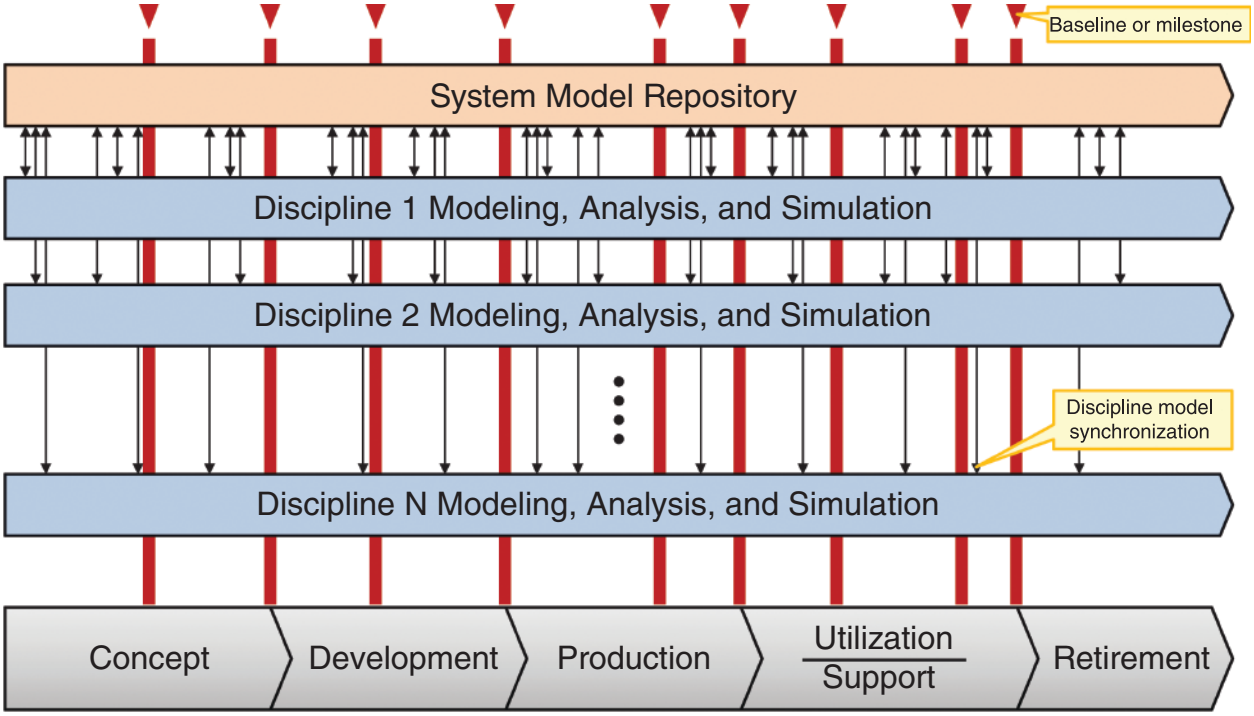


FIGURE 3.12 Multidisciplinary MA&S coordination along the life cycle. INCOSE SEH original table created by the NAFEMS-INCOSE Systems Modeling and Simulation Working Group (SMSWG). Usage per the INCOSE Notice pages. All other rights reserved.

physical repositories. The figure schematically depicts several examples of synchronizations, milestones, and baselines (in a real project there will of course be many more). In addition, MA&S data management supporting versioning, branching, merging, and archiving needs to be implemented for each of the threads, as well as across all organizations in an extended enterprise.

When different organizations collaborate in an extended enterprise, the need may arise to protect intellectual property, which naturally includes the know-how captured in models. To enable such collaboration, often so-called *black box* (also known as *opaque box*) *models* are created and maintained, which hide or obfuscate intellectual property, while still providing a publicly accessible external interface for using them to perform simulations. In contrast, *white box* (also known as *transparent box*) *models* provide full visibility of their internals.

3.2.2 Prototyping

Prototyping is a technique that can significantly enhance the likelihood of providing a system that will meet the stakeholder's needs. In addition, a prototype can facilitate both the awareness and understanding of stakeholder needs and requirements. The original use of a prototype was as the first-of-a-kind product from which all others were replicated. However, prototypes are not "the first draft" of production entities. Prototypes are intended to enhance learning and should be set aside when this purpose is achieved. Once the prototype is functioning, changes will often be made to improve performance or reduce production costs. Thus, the production entity may exhibit different behavior. Two types of prototyping are commonly used: rapid and traditional.

Rapid prototyping is an easy and one of the fastest ways to get system performance data and evaluate alternate concepts (Noorani, 2008). A rapid prototype is a particular type of physical model or simulation quickly assembled from a menu of existing physical, graphical, or mathematical elements. Examples include tools such as laser lithography or computer simulation shells. 3-D printing, or additive manufacturing, has significantly enhanced the physical elements that can be prototyped (Gebhardt and Hotte, 2016). Rapid prototypes are frequently used to investigate form and fit, human–system interface, operations, or producibility considerations. They are widely used and are particularly useful; but except in rare cases, they are not traditional "prototypes."

Traditional prototyping is a tool that can reduce risk or uncertainty and has two primary variants. A *partial prototype* is used to verify critical elements of the system. A *full prototype* is a complete representation of the system. They must be complete and accurate in the aspects of concern. Objective and quantitative data on performance times and error rates can be obtained from higher-fidelity interactive prototypes. SE practitioners are in a much better position to evaluate modifications that will be needed to develop the system because of the existence of a traditional prototype.

3.2.3 Traceability

Traceability for products and systems is defined as "the ability to trace the history, application, or location of an object/entity/item" (ISO 9000, 2015). From an SE perspective, traceability is establishing an association or relationship between two or more objects/entities/items such as life cycle concepts, needs, requirements, architectural definition artifacts (e.g., systems, system elements), design definition artifacts, verification artifacts, validation artifacts, information, models, and acquired or supplied systems or system elements.

Bidirectional traceability is the ability to trace an object/entity/item to another object/entity/item while automatically establishing a reverse link back to the initial object/entity/item. Thus, once a given object/entity/item has been linked to its source/destination, the source/destination is automatically linked to that object/entity/item. Bidirectional traceability is facilitated by SE tools which support the establishment of two-way links (bidirectional traceability) between objects/entities/items.

Vertical traceability is most often referred to in context of organization levels or architectural levels of the system or product under development. From a hierarchical architecture view (see Section 1.3.5), there are various system levels. The SoI level (Level n) has lower-level systems elements (Level n+1), some of which are further decomposed into lower-level system elements (Levels n+2, n+3, etc.) until the elements are defined to the level at which they can be made, bought, or reused. Entities at each level have objects/entities/items defined at various levels of abstraction. As the objects/entities/items are refined level-by-level, bidirectional traceability is established. Many times, these vertical traceability relations are referred to as "parent" and "child" relationships, depending upon the perspective (parent being the relationship to the higher level, child being the relationship to the lower level).

Horizontal traceability involves traceability across the elements of a given level of the architectural or system structure and across the life cycle. From a hierarchical architecture view, as relationships between objects/entities/items at the same level (i.e., Level n) are identified, bidirectional traceability is established. Many times, these horizontal traceability relations are referred to as "peer" relationships. Horizontal traceability also links objects/entities/items generated in one life cycle stage or process to data, information, and artifacts generated in other life cycle stage or process, resulting in connecting these objects/entities/items across the life cycle. For example, from a life cycle stage perspective, concept objects/entities/items are traced to development; which are traced to production; which are traced to utilization and support; which are ultimately traced to retirement. From a life cycle process perspective, a stakeholder requirement can be traced to its system requirements; which can be traced to architecture and design artifacts; which can be traced to the realized product; which can be traced to the system verification and system validation artifacts.

Establishing traceability is a critical activity of the Technical Processes (see Section 2.3.5), especially Business or Mission Analysis; Stakeholder Needs and Requirements Definition; System Requirements Definition; System Architecture Definition; Design Definition; System Analysis; Verification; and Validation. Traceability is facilitated through the appropriate application of the Configuration Management (CM) process (see Section 2.3.4.5). The CM

Identification activity enables the SE practitioner to "connect the dots" and understand the identity, location, relationships, pedigree, origin of data, materials and parts of the objects/entities/items. CM also enables the traceability of the history and location of the product after delivery. The management of products/systems, their system elements, and their configuration information requires unique identification so traceability of these items can be accurately determined. For traceability purposes the product/system identifier consists of a unique identifier which, once issued to a specific project/product/system, should never be reused.

Traceability is also a crucial component of the digital thread, enabling the connection between uniquely identified configurations of digital system models, digital twins, and physical assets (see Section 5.4). In an MBSE environment (see Section 4.2.1), the underlying system model enables a stakeholder requirement to be traced through the functional representations, to the physical product, thus enabling the identification of specific physical elements (and their specific configurations) that are impacted by a change in a given requirement. Vice versa, traceability enables the identification of the requirements that will need to be assessed when a given physical element is modified (e.g., due to a change of supplier or manufacturing process). Digitally enabled traceability methods help ensure the stakeholders get what they asked for. Digitally enabled traceability also supports the transparency of information.

More information on traceability can be found in INCOSE GtNR (2022) and the INCOSE NRM (2022).

3.2.4 Interface Management

The purpose of Interface Management is to facilitate and manage the identification, definition, design, and management of interfaces of the system across the system life cycle. It manages interface boundaries and interactions across those boundaries, the definition and agreement for each interaction, and interface requirements for all interactions identified by the various Technical Processes. Interface Management cuts across the Agreement, Technical Management, and Technical Processes. Because of its importance, the project team should focus on Interface Management as a distinct activity across all life cycle process activities.

Given that the behavior of a system is a function of the interaction of its elements and the interaction of the SoI and external systems, it is critical for the project team to identify and define each of the interactions between all system elements that make up the integrated system as well as interactions of the integrated system with external systems and users. Failing to do so will result in costly and time-consuming rework during system integration, system verification, and system validation. Because of the criticality of interfaces, the project team must define how they will manage interfaces in their project planning (e.g., SEMP). For more complex systems, projects often develop a separate Interface Management Plan. It is often useful to have the interfaces managed using an Interface Control Working Group. Additional elaboration concerning interface identification, interface definition, interface requirements, risk assessment, and managing interfaces across the life cycle is included in the INCOSE NRM (2022).

When interface management is applied as a distinct objective and focus of the SE processes, it will help highlight underlying critical issues much earlier in the project than would otherwise be revealed that could impact the project's budget, schedule, and system performance. Identifying interface boundaries and interactions across those boundaries early in the life cycle facilitates definition of the SoI's boundaries and clarifies the dependencies the SoI has on other systems and dependencies other systems have on the SoI (see Sections 1.3.1 and 1.3.3). Identifying interface boundaries and interactions across those boundaries also helps ensure compatibility between the SoI and those external systems in which it interacts. Of particular importance is the Human Machine Interface (HMI), as ultimately it is the interaction between users, operators, and maintainers that will result in acceptance of the SoI for its intended use by its intended users (see Section 3.1.4). Failure to identify all interface boundaries and interactions across those boundaries is a significant risk to the project, especially during system integration, system verification, system validation, operations, and maintenance. Because of this, it is extremely important the project defines life cycle concepts for how it will make sure the system will work safely and securely with all the external systems and personnel with which it must interact in the intended operational environment when operated by its intended users and is protected from outside threats across those interfaces.

A key characteristic of today's increasingly complex, software-intensive systems is the number of internal interactions within systems and between a system and external systems. The increased number of interactions relates directly to the complexity of a system. It greatly increases the complexity of integration of the system elements that are part of the SoI, and integration of the realized SoI within the system it is part of. It also increases the complexity of assessing the behavior of the integrated system when operated as part of a larger system. Another key characteristic of modern software-intensive systems is the form of the interactions. In the past, when many of the systems were mostly mechanical/electrical, the interactions were more visible involving connectors, wires, pipes, mechanical parts, bolts, etc. In software-intensive systems, there can be multiple computer modules, each with software that communicates commands, messages, and data across one or more communication busses. For example, in modern automobiles there can be more than 150 computer modules connected to each other and multiple sensors and actuators.

Interface Management Related to Life Cycle Processes Major interface boundaries between the SoI and external systems are identified during preliminary life cycle concept definition activities within the Business or Mission Analysis process (see Section 2.3.5.1). Through the application of the Stakeholder Needs and Requirements Definition process (see Section 2.3.5.2) the life cycle concepts are further elaborated, interface boundaries between external systems are further refined to include all interface boundaries, and interactions across each of those boundaries are identified. Risks associated with each interface boundary and associated interactions are assessed as part of the Risk Management process (see Section 2.3.4.4). Using the System Requirements Definition process (see Section 2.3.5.3), the interactions are further refined and the characteristics of what is involved in each interaction are defined. Using this information, interface requirements are defined.

As the system architecture and system elements are defined, the System Architecture Definition process (see Section 2.3.5.4) concurrently with the System Requirements Definition process identify and define interface requirements for external systems, including enabling systems, which are also allocated to the applicable system elements. Internal interface boundaries and interactions across those boundaries are identified, and interface requirements defined for each of the interactions across the interface boundaries internal to the SoI (i.e., between system elements). The focus is on defining and agreeing on the characteristics of what is involved in the interactions, not on how those interactions are realized. The interface identification, definition, and requirements continue to evolve as the system requirements, architecture, design, and models evolve. These definitions are recorded in some form of interface control artifacts (e.g., Interface Control Document (ICD)) that are put under configuration control. For each interaction across an interface boundary, the identified interaction is input to the System Requirements Definition process to define the interface requirements.

How those interactions are realized is addressed by the Design Definition process (see Section 2.3.5.5). Definitions of interactions across interface boundaries are refined to include what each system element involved in the interaction looks like at the interface boundary and the media (e.g., a data bus, a wiring harness, a physical connection, Wi-Fi, Bluetooth) involved in the interaction is determined. Additional interface boundaries and interactions may need to be identified and defined that were not addressed by the System Architecture Definition and System Requirements Definition processes. The definition of these additional interfaces often drives additional iteration between these processes to capture the interface characteristics and requirements definition. Interactions across interface boundaries are primary considerations in both horizontal and vertical integration across the life cycle as part of the Integration Process (see Section 2.3.5.8).

A major issue concerning interface definition is that when a system element is contracted out to a supplier or the SoI interacts with other supplier-developed system elements. Often the contracts are issued prior to design and thus the design definitions of what the SoI and system elements look like at the interface boundary and the media involved in the interaction have not yet been defined. In addition, it is common for the suppliers to have little insight into the workings of other suppler-developed system elements with which they interact and how changes to those system elements could affect the interactions and performance of their system element (or changes to their system element could affect other system elements). In these cases, it is important that the acquirer clearly addresses how each supplier will support, participate in, and comply with the interface management activities during interface definition, design, system integration, system verification, and system validation in the agreements via the Agreement Processes (see Section 2.3.2).

Using the System Analysis process (see Section 2.3.5.6), the level and type of analysis needed to understand the trade space with respect to the interface requirements and definition is determined and performed. This can include mathematical analysis, modeling, simulation, experimentation, and other techniques. The analysis results are input to trade-offs made through the Decision Management process (see Section 2.3.4.3).

The Implementation process (see Section 2.3.5.7) is used to develop the system element interfaces and record evidence of meeting the interface requirements for an implemented system element.

The Integration process (see Section 2.3.5.8) considers the integration of the system and system element interfaces in the integration planning and integrates the implemented system elements together at the interface boundaries. Each system element is verified to have met their interface requirements using the Verification process (see Section 2.3.5.9). The system interfaces are validated using the Validation process (see Section 2.3.5.11) against the stakeholder needs and stakeholder requirements concerning interactions with systems or users external to the SoI in the operational environment. The Transition process (see Section 2.3.5.10) checks the installation and operational state of the interfaces in the operational environment.

Key activities that are part of interface management include facilitating cooperation and agreements with other stakeholders, defining roles and responsibilities, enabling open communication concerning issues, establishing timing for providing interface information, problem resolution, and agreeing on the interaction characteristics across interface boundaries early in the project. These functions are done through the Project Planning process (see Section 2.3.4.1). An important interface analysis activity is assessing and managing risks as part of the Risk Management process (see Section 2.3.4.4), avoiding potential impacts especially during system integration, system verification, system validation, operation, and maintenance. Other processes may also contribute to the management of the interfaces.

After establishing baselines for interface requirements, interface definitions, architecture, and design, the Configuration Management process (see Section 2.3.4.5) provides the ongoing management and control of the interface requirements and definitions, as well as any associated artifacts.

Recording Definitions of Interactions across Interface Boundaries In a document-centric practice of SE, definitions concerning interface boundaries, the interaction across those boundaries, and the media involved are commonly recorded in some type of interface definition artifact (e.g., Interface Control Document (ICD), Data Dictionary (DD), Interface Definition Document (IDD), Interface Agreement Document (IAD)) or within the project's integrated dataset from which the associated report may be generated. In a data-centric practice of SE, these are often captured in databases and models. A data-centric practice enables effective impact and change analysis, as well as helping ensure consistency of interface requirements and definitions across the architecture.

Interface Analysis In conjunction with the other Technical Processes, the System Analysis process (see Section 2.3.5.6) applies various analysis methods to identify interface boundaries, and interactions across those boundaries, to better understand how the SoI interacts with the other systems that make up the system of which it is a part and to help ensure there are no missing interface boundaries, definitions, and interface requirements. Some common diagrams, methods, and tools used for analysis include functional flow block diagrams (FFBD), data flow diagrams (DFD), context diagrams, boundary diagrams, external interface diagrams, input, process, output (IPO) diagrams, N^2 diagrams, and internal interface diagrams, Failure Modes and Effects Analysis (FMEA), System-Theoretic Process Analysis (STPA), language-based models (e.g., SysML diagrams), and simulations.

A critical part of interface analysis includes an assessment of each interaction across an interface boundary in terms of maturity, stability, documentation, threats, and risks. The SoI is particularly vulnerable when interfacing with external systems over which they may have little or no control. Because of this, the SoI is vulnerable to undesirable effects at and across the interface boundaries. Therefore, identifying and managing risks associated with interface boundaries and interactions across those boundaries is key to exposing potential risks to the project across applicable life cycle stages. Many of the major issues discovered during system integration, system verification, and system validation involve interfaces.

An example analysis tool is the N^2 *diagram* shown in Figure 3.13, which enables a systematic approach to identifying interface boundaries and interactions across those boundaries. N^2 diagrams enable the SE practitioner to

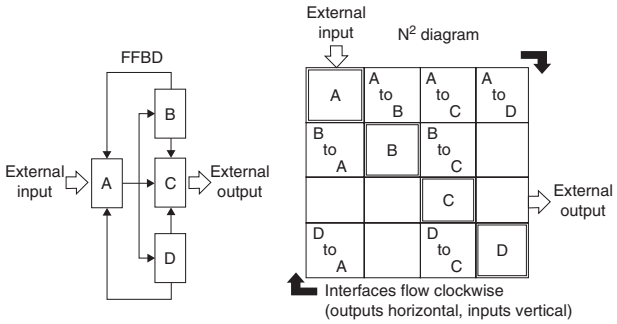


FIGURE 3.13 Sample N-squared diagram. INCOSE SEH original figure created by Krueger and Forsberg. Usage per the INCOSE Notices page. All other rights reserved.

assess and identify interface boundaries and interactions across those boundaries in a structured, bidirectional, fixed framework. The N^2 diagrams can be used at several levels of abstraction of the SoI: a functional view and a physical view.

An N^2 diagram is created using an N × N matrix. The system elements (functional or physical) are placed in squares forming a diagonal from upper left to lower right. The rest of the squares in the matrix represent potential interactions (interfaces) between the elements. In an N^2 diagram, interactions between elements flow in a clockwise direction. For example, the entity being passed from element A to element B, can be defined in the appropriate off-diagonal square. A blank square indicates there is no interaction between the respective elements. Sometimes, characteristics of the entity passing between elements may be included in the *off-diagonal* square where the interacting entity is identified. When all elements have been compared to all other elements, then the matrix is complete. If lower-level elements are identified in the process with corresponding lower-level interactions, then they can be successively described in expanded lower-level N^2 diagrams. The Design Structure Matrix (DSM) is very similar in appearance and usage to the N^2 diagram, but a different input and output convention is typically used (inputs on the horizontal rows and outputs on the vertical columns) resulting in interactions between elements flowing in a counterclockwise direction (Eppinger and Browning, 2012). Figure D-1 illustrates an N^2 diagram for the interactions amongst the system life cycle processes.

One of the main functions of the N^2 diagram, besides the identification of interactions, is to pinpoint areas where conflicts may arise between elements so that systems integration later in the development cycle can proceed efficiently (Becker, et al., 2000) (DSMC, 1983) (Lano, 1977). Alternatively, or in addition, functional and physical diagrams can be used with N^2 diagrams to characterize the flow of information among system elements and between system elements and the external systems. As the system architecture is decomposed to lower levels, it is important to ensure the interface interaction definitions keep pace and that interactions are defined so that decompositions of lower levels are considered.

Coupling matrices (a type of N^2 diagram, shown in Figure 3.14) are a basic method to define the aggregates and the order of integration (Grady, 1994). They can be used during System Architecture Definition (see Section 2.3.5.4), with

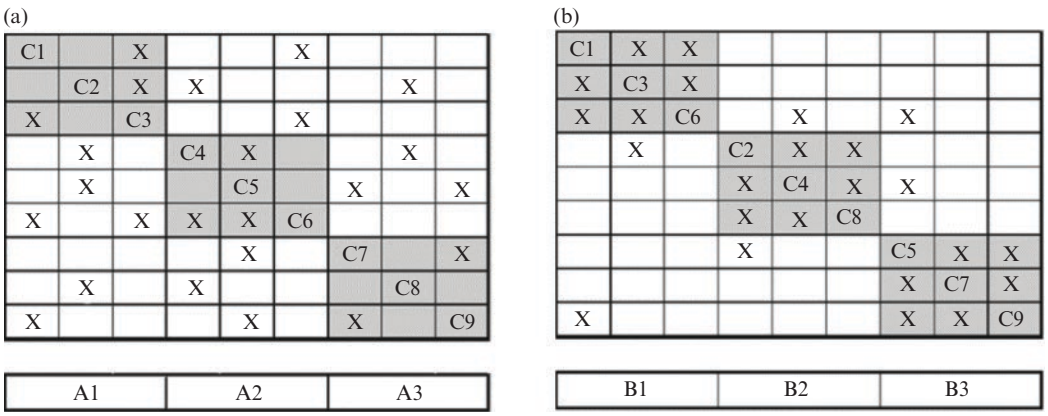


FIGURE 3.14 Sample coupling matrix showing: (a) Initial arrangement of aggregates; (b) final arrangement after reorganization. INCOSE SEH original figure created by Faisandier. Usage per the INCOSE Notices page. All other rights reserved.

the goal of keeping the interfaces as simple as possible. Simplicity of interfaces can be a distinguishing characteristic and a selection criterion between alternate architectural candidates. The coupling matrices are also useful for optimizing the aggregate definition and the verification of interfaces during Integration (see Section 2.3.5.8). Integration can be optimized by reorganizing the coupling matrix in order to group the system elements into aggregates and minimize the number of interfaces to be verified between aggregates. When verifying the interactions between aggregates, the coupling matrix can be an aid for fault detection.

3.2.5 Architecture Frameworks

An *architecture description framework* is defined in ISO/IEC/IEEE 42010 (2022) as:

> A set of "conventions, principles and practices for the description of architectures established within a specific domain of application or community of stakeholders."

The term "description" is used in the definition to avoid confusion between architecture description frameworks and other frameworks (e.g., enterprise architecture framework, architecture evaluation framework). Other definitions for *architecture framework* (AF) can be found in the technical literature, for example The Open Group Architecture Framework (OMG TOGAF, 2023) defines AF as:

> A foundational structure, or set of structures, which can be used for developing a broad range of different architectures.

Architecture frameworks are used in various domains to help ensure harmonization, consistency, and re-use. When a commonly agreed upon architecture framework is adhered to by all project teams involved, better aligned project artifacts typically result. This benefit will be particularly evident in distributed teams and within enterprises when architecture descriptions and architectural artefacts are reused across projects.

Most architecture frameworks are organized to provide one or more *viewpoints* to cover the target domains and their typical stakeholders' concerns (e.g., NATO AF (NAF), Unified AF (UAF), and Department of Defense AF (DoDAF)). Some frameworks also provide one or more of the following:

- A method for describing systems in terms of a set of architecture building blocks, and for showing how the building blocks fit together.
- A set of tools and a common vocabulary.
- Multiple dimensions with coordinates for relating particular groups of concerns or solutions along the dimensional aspects.

Figure 3.15 provides an overview of the Unified Architecture Method (UAM), which provides dimensions for perspectives and aspects.

Others advocate that architecture frameworks should include a list of recommended standards, libraries of patterns, and compliant products that can be used to accelerate architecting. Finally, it is useful for architecture frameworks, or more broadly, architecting environments, to define activities and resources for architecture governance, in addition to the governance of skills and competencies in place, with regard to enterprise objectives.

Framework Support to Architecture Activities Architecture activities are described in the System Architecture Definition process (see Section 2.3.5.4). This section explains how some of the major architecture frameworks can be used to perform the key architecture activities.

Architecture Enablement Frameworks like Pragmatic Enterprise AF (PEAF) (Pragmatic 365, 2023) and Generalized Enterprise Reference Architecture and Methodology (GERAM) (Bernus, 1999) can be used to establish and maintain a set of capabilities, services, and resources that support the architecture process. The enablement activities include:

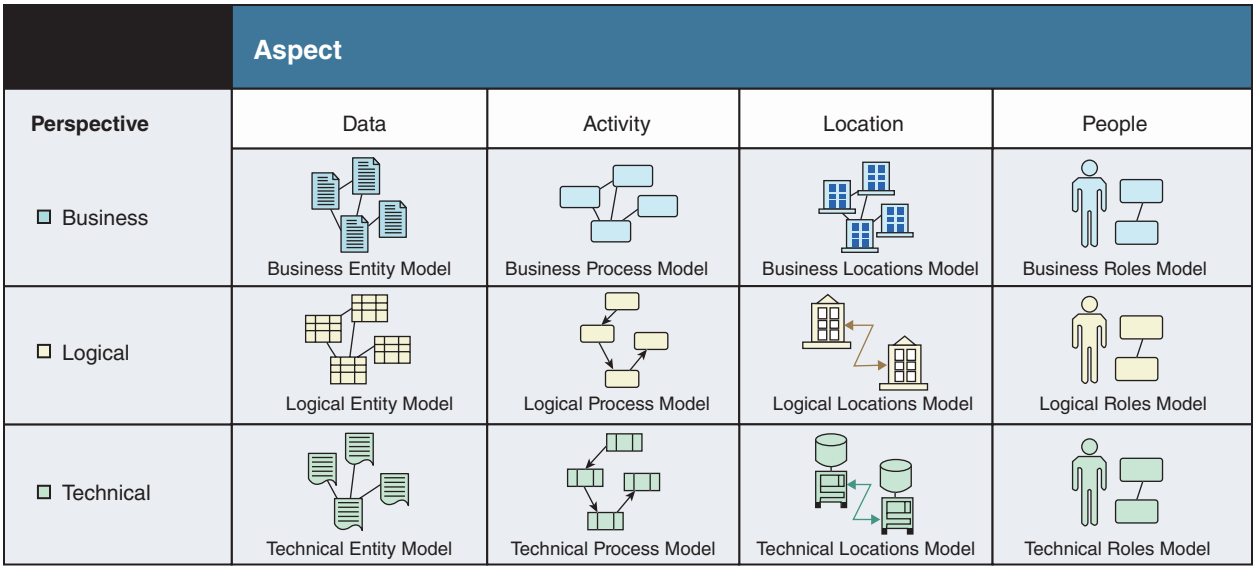


FIGURE 3.15 Unified Architecture Method. From UAM (2022). Used with permission. All other rights reserved.

- Analysis of context in the organization where the architecture activities can take place.
- Definition of the main principles and the overall organization where the processes, methods, roles, and technologies can be used for architecting.
- Implementation of these high-level principles with methodologies provided by frameworks like TOGAF for IT domain or NAF and DoDAF for defense domains. This implementation comprises development of a metamodel to capture the terminology and a collection of architecture development methods, standards library, architecture repository and registry, and architecture capability. Architecture capability includes skills and governance logic.
- Reference documents like Evans (2014) help assess the architecture context and environment with regard to the architecting styles of the enterprise programs and projects.

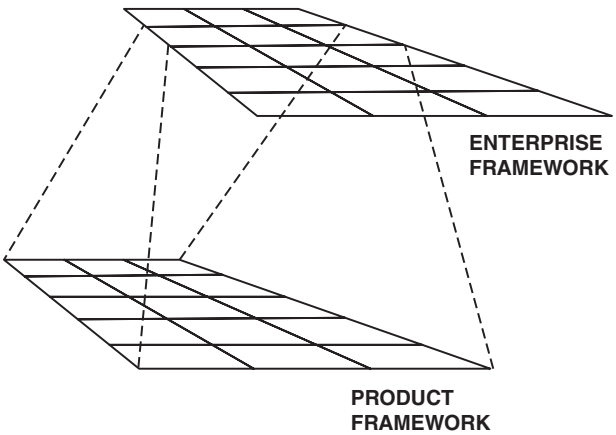


FIGURE 3.16 Enterprise and product frameworks. From Sowa and Zachman (1992). Used with permission. All other rights reserved.

Architecture Governance Related concepts are objective, goal, strategy, policy, directive, roadmap, life cycle stage, and statement of work. Per Sowa and Zachman (1992), as illustrated in Figure 3.16, two levels of architecture frameworks should be established to ensure consistency of products and systems regarding the enterprise strategy. Within most contexts, there exists a need to consider the architecture of multiple entities, each with its own life cycle, and correspondingly a framework, that helps describe or model that entity. In addition, architecting must consider that the life cycles of these entities are interrelated, often in a recursive manner (by one entity contributing to some or all activities in the life cycle of another), and that these activities may have to be synchronized (e.g., for complexity reduction purposes, to achieve or to maintain selected system quality characteristics).

Frameworks like NAF and TOGAF consider two levels of architecture governance:

- At *enterprise* level, in accordance with enterprise objectives, goals, strategy, roadmap, policies and directives.
- At *project* level, with regard to internal or external contracts of the projects of produced architecture(s) along the whole life cycle of the entity of interest related to the architecture(s).

Architecture Management Architecture management implements the governance directives in the frame of the project contract. Frameworks like NAF explain that the architecting effort should be defined in a management plan. This plan should also include the management of the work products throughout their whole life cycle. The management activities coordinate the architecting effort and report to the governance level with rationales about the application of the governance directives and the agreement.

Architecture Description Architecture Description, as defined by ISO/IEC/IEEE 42010 (2022), is accomplished with *Architecture Conceptualization* and *Architecture Elaboration*, as defined by ISO/IEC/IEEE 42020 (2019). These architecting activities are performed as planned by the management plan covering the architecture effort. Frameworks like Zachman explain how architecture viewpoints characterize the problem and represent the solution with regard to the stakeholders' concerns, possibly structured as architecture aspects and stakeholders' perspectives. These viewpoints can be defined in architecture frameworks or developed within the project for the benefit of the enterprise. ISO/IEC/IEEE 42010 defines *views* and *viewpoints* as they apply to the architecture description. Annex F of ISO/IEC/IEEE 42010 includes tables of requirements compliance for Architecture Description Frameworks. Development of project-specific viewpoint specifications needs to be justified because they imply additional effort for architecture description, evaluation, management, and usage of the work products.

Frameworks like NAF and TOGAF include a methodology to develop the architecture views governed by the viewpoints. As far as possible, this development should be based on patterns and standards already proven in the business domain where the entity of interest resides. Formalisms, model kind specifications, and modeling languages are typically defined in AFs.

Architecture Evaluation As defined by ISO/IEC/IEEE 42030 (2019), the evaluation activities determine the extent to which one or more architectures meet their objectives, address stakeholder concerns, and meet relevant requirements. These activities are performed as planned by the management plan covering the architecture effort. Frameworks like Architecture Tradeoff Analysis Method (ATAM) (CMU/SEI, 2000) and Method Framework for Engineering System Architectures (MFESA) (Firesmith, et al. 2008) allow performing architecture evaluation in three steps:

- Definition of the objectives and evaluation criteria agreed by the stakeholders to cover their concerns.
- Definition or development of a method to cover the activities normally structured in analysis, assessment, and evaluation tasks.
- Analysis of the architecture concepts and properties, assessment of the value and utility for the stakeholders, and formulation of findings and recommendations in evaluation reports.

3.2.6 Patterns

Introduction to Patterns The scientific disciplines, whether concerned with phenomena at a molecular, global, or astronomical scale, are based upon discovery and effective modeling of *patterns*. Patterns are recurrences—repeated regularities observed across time, space, or other dimensions. Patterns lie at the heart of physical sciences and the related engineering disciplines, as laws of nature whose mathematical representation and engineering exploitation have transformed the nature and possibilities of human life. In SE, recurring patterns are observable in engineered system requirements, solution architectures, stakeholder value, missions, fitness and trade spaces, parametric couplings, failure modes and risks, markets, system phenomena, principles, and the socio-technical

systems of engineering and life cycle management. For example, there are patterns for requirements for refrigerators, patterns for design of coolant compressors, patterns for refrigerant failures, and patterns for maintaining refrigerators. Patterns are visible for products developed for commercial markets and systems engineered under defense contracts, as well as for the socio-technical systems that produce them, such as methodology patterns for eliciting and validating requirements. Whether the patterns are only implicitly and informally recognized and used, or explicit and formal, they can be found across the System of Innovation Pattern shown in Figure 1.6, where they are the basis of group learning. Explicitly modeled patterns help us surface and more efficiently share (learn, teach, practice) what earlier generations of SE practitioners treated as expertise and intuition only obtainable over decades of personal practice. As in the physical sciences, engineering patterns of all kinds are also subject to issues of credibility, validity, applicability, and trust as a basis for decision-making and action. Patterns are not "one size fits all", but instead have both fixed (recurring) and variable (parameterized) aspects, distilled by abstraction across individual instances. Depending on how they are recognized, represented, managed, and applied, patterns may be *informal* or *formal*.

Informal Patterns The most informally described patterns are those implicit in the expertise or judgement of individual practitioners and teams (as in tribal knowledge), when subject matter experts recognize new occurrences of past experiences. Examples are Jean's expertise in packaging systems, or Jose's expertise in risk assessment. Because of the high value of this experience and interest in making it available to others, historical efforts have been made to explicitly capture and record such patterns, even in informal form, so that they can be transmitted to others. SE *principles* and *heuristics*, often captured as informal prose, illustrate such explicit but informal patterns (see Sections 1.4.3 and 1.4.4). The informal but explicit prose representation of engineering patterns has created popular followings in civil and software engineering communities of practice (Alexander, et al., 1977) (Gamma, et al., 1995). These patterns typically include a prose template description of a problem and an informal description of a design pattern suited to such a problem. Examples of these explicit, informal, but effective patterns include building structural patterns and city layout patterns (in civil architecture patterns), as well as sorting algorithms and graphic user interface designs (in software design patterns). SE practitioners and leaders should not underestimate the value of explicit informal patterns for transmitting knowledge.

Formal Patterns The sciences' transition from informal prose to formal models powered much of the Science, Technology, Engineering, and Math (STEM) revolution's transformative impact, where model-based representations of patterns are the heart of the related physical sciences. These models have also enabled several generations of powerful automation tools for design, simulation, and production across the engineering disciplines, and more recently this is also impacting SE.

The practice of SE has increased use of explicit formal system descriptive models as central to SE methods, described as Model-Based Systems Engineering (MBSE) (see Section 4.2.1). This also enables the shift to formal model-based representation of patterns and their application in SE, because patterns based in models can be readily transformed (including automated assistance) into configured models specific to an application or project. Likewise, such patterns can be used in automated conformance-checking of other models. Provided the credibility of the patterns for the uses intended is managed, this not only shortens time to a trustable specific model, it also helps shift the language and perspective of multiple systems practitioners and teams into common semantic frameworks specific to a domain or specialty, for improved compatibility and interoperability. For example, do designers of tractors and trailers have a common perspective on the interactions between these engineered products? Can their work be readily checked for consistency? These issues have major impacts on SE effectiveness and productivity.

Formal patterns, particularly when model-based, appear under different names and "flavors" across SE practice and this handbook. Among these are ontologies, Architectural Frameworks, schemas, and Product Line Engineering (PLE) datasets. For more on these, refer to INCOSE S*Patterns Primer (2022) and the other sections of this handbook. Formal patterns also include general and domain-specific system modeling languages.

The power of models in the STEM revolution was not simply that they reflected agreements across groups (as in standards), but also agreements with observed natural phenomena, reduced to simplest form in the patterns of physical laws. These phenomena-based patterns continue to provide the theoretical basis for the individual engineering disciplines, as well as for the foundations of SE (Schindel, 2016, 2020). The central question they address is: What is the *smallest* system model content necessary to represent a SoI, across its life cycle, for purposes of engineering and life cycle management (Schindel, 2011)? This question has practical implications, but is also rooted in the foundations of SE:

- The *practitioner* has an interest in keeping things as simple as possible, but not simpler. "Too large" a model implies the burden of more information than is needed, including redundancies which often include inconsistencies. "Too small" a model implies that information needed during the life cycle is missing.
- *Foundations of an engineering discipline* include representing recurring phenomena fundamental to its corresponding science. The smallest set of elements generating a discipline identifies its foundations (e.g., Newton's Laws generating Mechanics, Maxwells' Equations generating Electrical Science). A definition of a system's mathematical complexity is the size of its smallest generating representation (Li and Vitanyi, 2009).

The *Systematica Metamodel (S*Metamodel)* is a formal pattern describing a neutral (independent of specific modeling languages or tools) answer to the above "smallest model" question, mapped into contemporary model tooling and languages, such as SysML, simulations, or modeling frameworks. An *S*Model* is any model, expressed in any modeling language or tooling, that is mapped to the reference S*Metamodel. The S*Metamodel spans disciplines, tooling, and languages, and is rooted in the phenomena-based models of the physical sciences.

Modern word processing tools are powerful, but varying writer composition skills and practices allow authoring that may produce valuable literature or faulty descriptions and broken semantics. Similarly, observed methods of use of contemporary modeling languages and automated tools allow the generation of system models that are both too small (are missing important elements) and too large (contain undetected redundancies and contradictions) at the same time. Fortunately for formal models, the history of the physical sciences provides patterns about the nature of phenomena and their models, and these can guide the users of contemporary tools and languages to more effective models than bare languages and tools alone. Accordingly, the S*Metamodel provides that guidance in any language or tooling into which it is mapped. Three examples from the S*Metamodel are:

- *All behavior is interaction-based*: Physics has made it clear that there is no "naked" behavior in the absence of interactions, although system modelers sometimes create models that incorrectly assert otherwise. Interactions are the heart of system phenomena, emergence, SE, and S*Models. Failure to understand and represent interactions leads to well-known engineering problems such as overlooking the impact of "external" actor behavior on the performance of an in-service engineered system (Schindel, 2013, 2016).
- *Requirement statements are transfer functions*: Models can help make it clear that requirement statements are not simply prose, but always represent input-output relationships parameterized by state variables. S*Models make this clear and enable improved auditing for problematic or overlooked requirements (Schindel, 2005).
- Stakeholder value trade space, failure effects in risk analysis, and configurability of product line families are all manifestations of the same variables: These are frequently treated as relatively independent specialties and dimensions, greatly over-complicating system representation and understanding, when that apparent dimensionality can be substantially reduced by S*Models (Schindel, 2010).

Other aspects of the S*Metamodel, and the S*Models generated from it, are described in Schindel (2011), INCOSE S*MBSE (2022), and INCOSE S*Patterns Primer (2022).

S*Patterns S*Patterns are reusable S*Models of families of systems, often domain-specific, configurable to represent multiple individual applications, market segments, or other configurations (Schindel, 2022a). There are also more generic S*Patterns, such as the S*Metamodel itself (INCOSE S*MBSE, 2022), the System Innovation Ecosystem Pattern introduced in Figure 1.6 (Schindel and Dove, 2016) (INCOSE Innovation Ecosystems, 2022), and the Model Characterization Pattern used to generate requirements and metadata for unified characterization of virtual models of all types (INCOSE S*MCP, 2019). Pattern-based MBSE using S*Patterns involves authoring of system patterns and their configuration to application and project-specific S*Model instances, as summarized by Figure 3.17. Part of the "minimality" of the S*Metamodel is its sufficiency for such representations, including configuration rules. Instructional examples of system pattern representations may be found in Schindel and Peterson (2013). System patterns have been used in automotive, heavy equipment, aerospace, medical device, diesel and gas turbine engines, advanced manufacturing, consumer product, cybersecurity, and other domains (Bradley, et al., 2010) (Cook and Schindel, 2015) (Schindel and Smith, 2002) (Schindel, 2012).

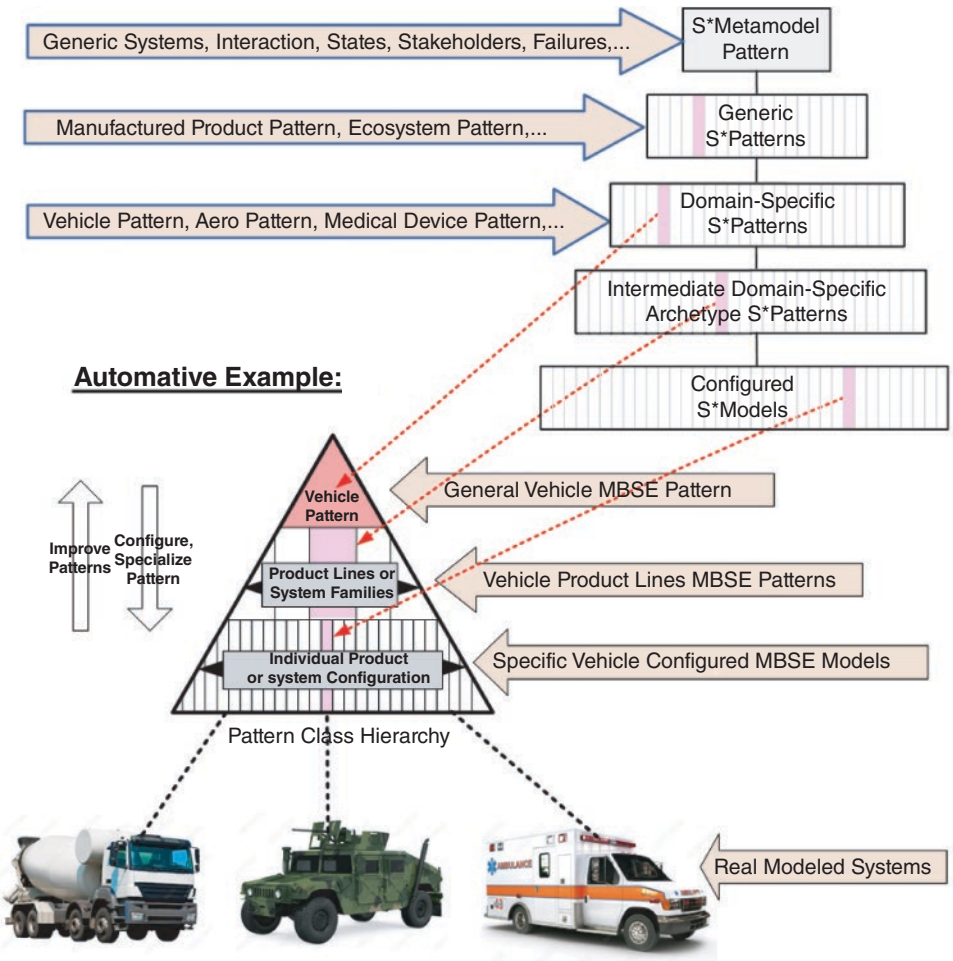


FIGURE 3.17 S*Pattern class hierarchy. From (Schindel and Peterson, 2013). Used with permission. All other rights reserved.

There is more to pattern-based methods than just representing the patterns. Historical descriptions of SE processes can appear to describe all the processes practitioners ought to perform in order to discover, validate, and utilize all the information the system life cycle requires. However, those descriptions have by volume had less to say on the question of "what about what we *already* know?" Such descriptions might be viewed as relying on practitioners to separately work out informal means of exploiting existing knowledge within what the process specifies. To address such questions, the System Innovation Ecosystem Pattern shown in Figure 1.6 describes the curation and mixing of information believed already credible with required new information extraction and validation. Schindel and Dove (2016), INCOSE S*Patterns Primer (2022) and others describe multiple additional levels of detail decomposition of processes, information, ecosystem capabilities, and limitations. Those details show how life cycle processes of ISO/IEC/IEEE 15288 (2023) and this handbook are incorporated to manage group learning and controlled sharing, and especially pattern credibility and uncertainty, across multiple programs of an enterprise, supply chain, or industry group. The System Innovation Ecosystem Pattern is further concerned with the effective linkage between the processes of pattern learning, validation, and curation versus the execution processes of making use of the content of trusted patterns—often by different people, at different times, in different places or organizations.

3.2.7 Design Thinking

Understanding and leveraging the technical, business, and social relationships to successfully design and manage engineered systems is still a challenge in SE practice. SE solutions to this challenge tend to focus on system components, human activities, machine functionality, and human-system integration . Solution design can take advantage of Design Thinking (Dorst, 2015) as a complementary approach to Systems Thinking (see Section 1.5). Design Thinking explores (1) the human needs, (2) the operational and business processes and reasoning by which design concepts are devised and realized, especially those which are creative in nature, together with (3) the systems being realized, (4) its specialization, and (5) their utilities and value provided for the stakeholders.

In a Design Thinking process (Cross, 2000), (Lawson, 1997), context analysis and problem framing techniques are employed to identify all relevant influences on a problem, explore the given problem, and restructure or revise it to suggest a route to a solution. Solution generation techniques, including approaches to idea generation (ideation), are employed to identify a range of possible design solutions which are based on:

- existing known solutions, possibly in the form of variants, patterns or other adaptations;
- applying different forms of design-related reasoning to achieve innovative solutions;
- iterating between decomposing functional requirements and design solutions to achieve optimal design – see, for example, Axiomatic Design (Suh, 2001); and,
- using successive divergent and convergent phases of design synthesis and analysis with respect to the value provided for the stakeholders resulting from business and operational processes.

Design Thinking enables SE practitioners and other team members to understand the stakeholders, challenge assumptions, redefine problems, and realize innovative solutions by drawing upon logic, imagination, intuition, and systemic reasoning. Design Thinking can also be utilized for anticipating and addressing emergent features of systems, and in technical management and organization of engineering processes.

As Design Thinking approaches use solution-based methods, they can be used in various system life cycle stages. Examples are to support business or mission analysis (see Section 2.3.5.1), to identify and validate stakeholder or system requirements (see Sections 2.3.5.2 and 2.3.5.3), or to define the system architecture or its design solution (see Sections 2.3.5.4 and 2.3.5.5).

FIGURE 3.18 Examples of natural systems applications and biomimicry. INCOSE SEH original figure created by McNamara and Anway derived from Studor (2016) and Hoeller, et al. (2016) using NASA images. Usage per the INCOSE Notices page. All other rights reserved.

3.2.8 Biomimicry

Definition *Natural systems* include living and non-living systems—anything that is not human-made. Natural systems differ from engineered systems (see Section 1.1), which are the primary focus of this handbook.

"*Biomimicry* is a practice that learns from and mimics the strategies found in nature to solve human design challenges—and find hope" (Biomimicry Institute, 2022).

Purpose *Nature inspired SE* and biomimicry can improve processes, practices, and products through the understanding of how nature is structured, behaves, adapts, interacts, accomplishes functions, and recovers from disturbance. Applying natural systems thinking and engineering can improve system capability, efficiency, and performance, while benefiting operations, support, and the effects on external environments. Examples include optimized information processing and sensing, operation in extreme environments, innovative materials application, distributed architectures, understanding of how emergence arises, lowered environmental impact, and system resilience. Nature has strategies to improve performance in all these areas, including circular approaches to materials and energy. To utilize nature-inspired solutions, SE looks to a universal solution space and asks regularly, "Can nature help me solve this problem?" and, "How can nature help me improve my SoI, product, or process?"

Examples Examples of successful natural systems applications and biomimicry abound. Select examples are shown in Figure 3.18: Velcro® inspired by burdock (Velcro, 2023); an impeller inspired by the calla lily and nautilus shell (Pax Water Technologies, 2022); grippers inspired by the gecko (NASA JPL, 2013, 2014, 2015); and a sensor inspired by an insect's compound eye (Frost, et al, 2016).

Description Over time, natural systems have developed a very close fit to their surroundings and other systems. The result is that they exhibit optimized attributes that often exceed the performance of engineered systems. In addition, they often have positive impacts on the environment. The study of natural systems includes forms, structures, materials, behaviors, processes, regenerative strategies, and interactions. Studying natural systems will increase an SE practitioners' repertoire of solutions, architectural variations, and strategies. The SE practitioner on a project is ideally suited to explore opportunities for application of natural systems across all life cycle stages.

To develop natural systems solutions, the SE practitioner uses a systematic process that:

- Begins by being open to alternate solutions;
- Defines requirements in terms of abstract functions or goals, including specific relevant metrics whenever practical;

- In the early stages of solution exploration, uses the abstracted functions to search for and identify multiple natural systems that could satisfy the desired function and examines characteristics of each;
- Selects one or more candidate natural systems;
- Abstracts the strategy that accomplishes the function in nature;
- Explores architectural variations that translates the strategy and generates alternate system element alternatives;
- Transfers the strategy to the SoI;
- Evaluates system element performance at the system level; and
- Evaluates the environmental impact of the system production, operation, support, and retirement.

Partnering with and supporting natural systems scientists can be essential to a successful implementation. An SE team gains from the in-depth knowledge provided by a cross-disciplinary team.

For more information, see INCOSE NS Primer (2023).

4

TAILORING AND APPLICATION CONSIDERATIONS

This section provides considerations for the application of SE with respect to different methodologies, approaches, system types, product sectors, and application domains.

4.1 TAILORING CONSIDERATIONS

There are many standards and handbooks that address life cycle models and SE processes. However, in most cases, these cannot be directly applied to a given organization or project. There is usually a need to tailor them for the specific project, organization, environment, or other situational factors.

The principle behind tailoring is to adapt the processes to ensure that they meet the needs of an organization or a project while being scaled to the level of rigor that allows the system life cycle activities to be performed with an acceptable level of risk. In general, all system life cycle processes can be applied during all stages of the system life cycle, tailoring determines the process level that applies to each stage. Additionally, processes are applied iteratively, recursively, and concurrently as shown in Figure 2.10.

Tailoring scales the rigor of application to an appropriate level based on risk. Figure 4.1 is a notional graph for balancing formal process against the risk of cost and schedule overruns (Salter, 2003). Insufficient SE effort is generally accompanied by high risk of schedule and cost overruns. If too little rigor is applied, the risk of technical issues increases. However, as illustrated in Figure 4.1, too much formal process may also lead to increased cost. If too much rigor is applied or unnecessary process activities or tasks are performed, the risk of cost and schedule slips increases. Tailoring occurs dynamically over the system life cycle depending on risk and the situational environment. Therefore, it should be continually monitored and adjusted as needed.

This section describes the process of tailoring the system life cycle models and SE processes to meet organization and project needs.

INCOSE Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities, Fifth Edition.
Edited by David D. Walden, Thomas M. Shortell, Garry J. Roedler, Bernardo A. Delicado, Odile Mornas, Yip Yew-Seng, and David Endler.

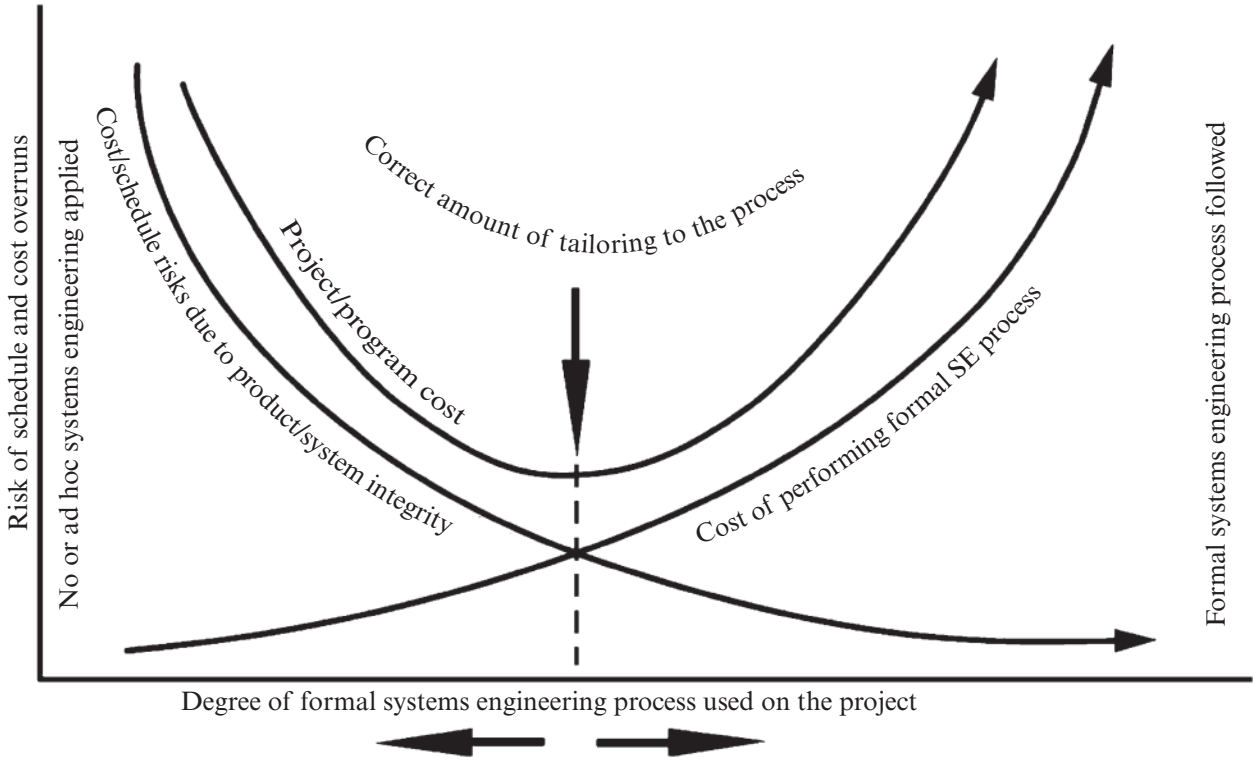


FIGURE 4.1 Tailoring requires balance between risk and process. INCOSE SEH original figure created by Krueger adapted from Salter (2003). Usage per the INCOSE Notices page. All other rights reserved.

Tailoring Process

Overview

Purpose As stated in ISO/IEC/IEEE 15288,
[A.2.1] The purpose of the Tailoring process is to adapt the processes of ISO/IEC/IEEE 15288 (and of this handbook) to satisfy particular circumstances or factors that:

a) Surround an organization that is employing ISO/IEC/IEEE 15288 in an agreement;
b) Influence a project that is required to meet an agreement in which ISO/IEC/IEEE 15288 is referenced;
c) Reflect the needs of an organization in order to supply products or services.

Description At the organization level, the tailoring process adapts external standards in the context of the organizational processes to meet the needs of the organization. At the project level, the tailoring process adapts organizational processes for the unique needs of the project.

Inputs/Outputs Inputs and outputs for the Tailoring process are listed in Figure 4.2. Descriptions of each input and output are provided in Appendix E.

Process Activities The Tailoring process includes the following activities:

- *Identify and record the circumstances that influence tailoring*.
  - Identify the strategic, programmatic, and technical risks for the organization or project.
  - Identify the level of novel concepts or the complexity of the system solution.

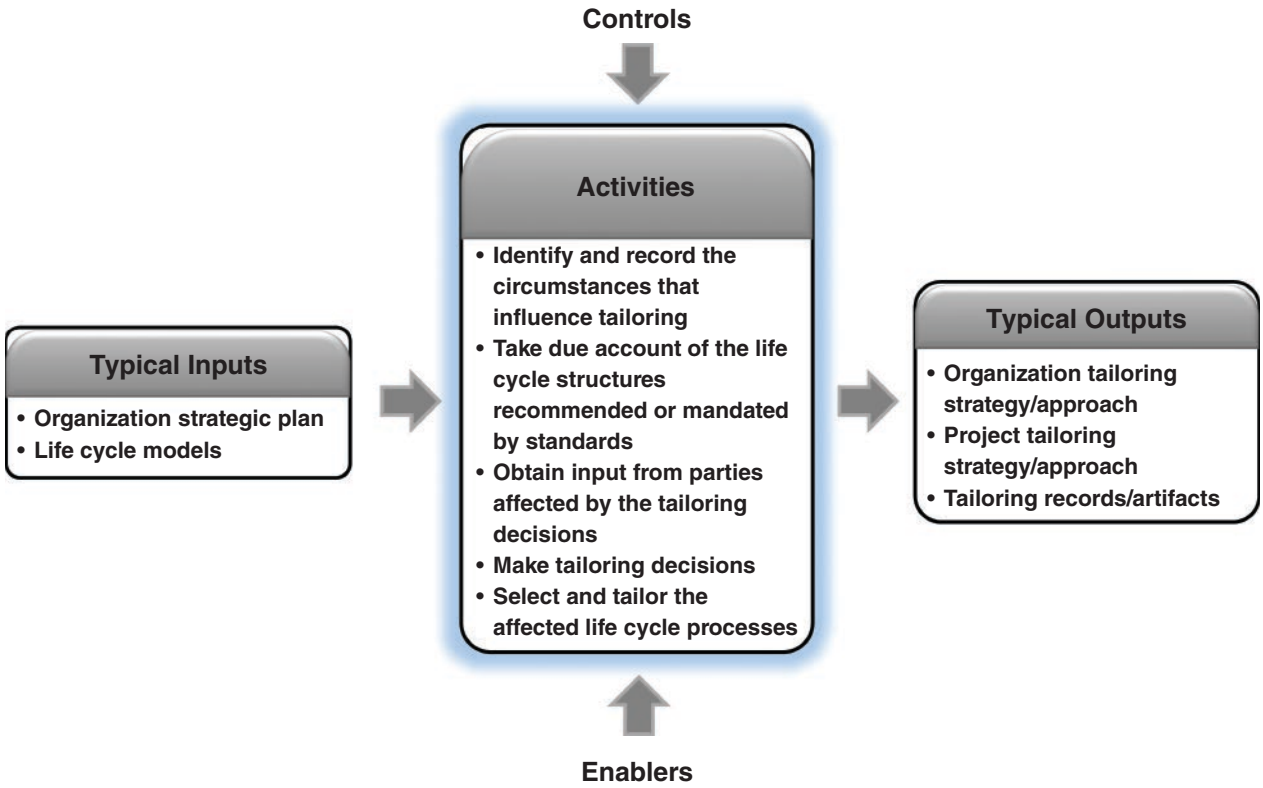


FIGURE 4.2 IPO diagram for Tailoring process. INCOSE SEH original figure created by Shortell, Walden, and Yip. Usage per the INCOSE Notices page. All other rights reserved.

  - Identify administrative effects (e.g., geographic distribution, organizational distribution, team size) that may impact tailoring.
  - Record the relative importance of circumstances that influence tailoring.
  - Identify tailoring criteria for each stage.
- *Take due account of the life cycle structures recommended or mandated by standards*.
  - Identify relevant standards.
  - Evaluate impact on tailoring and implementation across the life cycle.
- *Obtain input from parties affected by the tailoring decisions*.
  - Determine the level of expertise and experience of project members for the processes.
  - Determine the expectations or requirements from stakeholders.
  - Determine the risk tolerance of the stakeholders.
- *Make tailoring decisions*.
  - Assess candidate life cycle models.
  - Record assessment results.
- *Select and tailor the affected life cycle processes*.
  - Capture and maintain rationale for selected life cycle processes.
  - Establish means to continuously evaluate performance of tailored processes.

Note: Tailoring can include the deletion, modification, or addition of outcomes, activities, tasks, typical inputs, or typical outputs.

Common approaches and tips:

- Base decisions on facts and obtain approval from an independent authority.
- Use the Decision Management process to assist in tailoring decisions.
- Constrain the tailoring based on agreements between organizations.
- Control the extent of tailoring based on issues of compliance to stakeholder, customer, and organization policies, objectives, and legal requirements.
- Have different organizations propose and approve the tailoring.
- Influence the extent of tailoring of the agreement process activities based on the methods of procurement or intellectual property.
- Remove extra activities as the level of trust builds between parties.
- Identify the assumptions and criteria for tailoring throughout the life cycle to optimize the use of formal processes.
- Document the rationale for tailoring decisions.

Elaboration *Organizational Tailoring* Organizational tailoring applies specifically to the creating and maintaining organizational-level processes used by all projects. It is done in conjunction with the Life Cycle Model Management process (see Section 2.3.3.1). When contemplating if and how to incorporate a new or updated external standard into an organization, the following should be considered (Walden, 2007):

- Understand the organization;
- Understand the new standard;
- Adapt the standard to the organization (not vice versa);
- Institutionalize standards compliance at the "right" level;
- Allow for tailoring.

Project Tailoring Project tailoring applies specifically to the work executed through projects. It is done in conjunction with the Project Planning process (see Section 2.3.4.1). Factors that influence tailoring at the project level typically include, but are not limited to:

- Stakeholders and acquirers (e.g., number of stakeholders, quality of working relationships);
- Project budget, schedule, and requirements;
- Risk tolerance;
- Complexity and precedence of the system;
- The need for horizontal and vertical integration (see Section 2.3.5.8).

As mentioned in SE principle #12 (see Section 1.4.3), complex systems are engineered by complex organizations. Consequently, today's systems are more often jointly developed by many different organizations. Cooperation must transcend the boundaries of any one organization. Harmony between multiple organizations is often best maintained by agreeing to follow a set of consistent processes, methods, and tools.

Traps in Tailoring Common traps in the tailoring process include, but are not limited to, the following:

- Reuse of a tailored baseline from another system without repeating the tailoring process;
- Using all processes and activities "just to be safe";
- Assuming there is a single set of measures, risks, or other controls that apply to all projects without tailoring;
- Using a pre-established tailored baseline;
- Failure to include relevant stakeholders.

Tailoring for Very Small Enterprises The ISO/IEC/IEEE 29110 series defines Very Small Enterprises (VSEs) as enterprises, organizations, departments, or projects with up to 25 people. In many cases, VSEs find it difficult to apply international standards to their business needs and to justify the application of standards to their business practices. Typical VSEs do not have a comprehensive infrastructure, and the limited personnel usually are performing multiple roles. This may also happen in a large organization when the task is to perform a small project with less than 25 people involved. In this case, it can be extremely challenging to downscale the organization's life cycle model that is designed for much larger projects.

The ISO/IEC/IEEE 29110 series defines guides for VSEs based on a set of VSE characteristics (e.g., business models, situational factors, risk levels). From that, four profiles were derived:

- *Entry* (less than 6 people or start-ups);
- *Basic* (single application by a single work team);
- *Intermediate* (more than one project in parallel with more than one work team); and
- *Advanced* (for VSEs that want to sustain and grow as an independent competitive system developer).

These profiles cover the needs of most VSEs. Each of the profiles defines subsets of international standards (e.g., ISO/IEC/IEEE 15288) relevant to the VSE's respective context. For critical projects, such as mission critical or safety critical, these profiles do not apply, since the criticality of the projects would dictate a much greater level of rigor and comprehensive SE.

4.2 SE METHODOLOGY/APPROACH CONSIDERATIONS

The system definition activities, especially the partitioning of the system, requires that integration is considered throughout the development stage of the life cycle. The integration considerations may also require refinement based on when and how the work is performed. The choice of SE methodology or approach, along with the chosen life cycle model, often affects the sequence of work, which can help determine the focus of resources to address the unique challenges of the project.

This section introduces considerations for the following SE methodologies and approaches:

- Model-Based Systems Engineering (MBSE);
- Agile Systems Engineering;
- Lean Systems Engineering;
- Product Line Engineering (PLE).

Note that other types of SE methodologies and approaches exist.

4.2.1 Model-Based SE

This section provides an overview of the Model-Based Systems Engineering (MBSE) approach and includes a summary of its benefits relative to a more document-based approach. It also references a set of MBSE methodologies and provides a brief description of one representative methodology called the Object-Oriented Systems Engineering Method (OOSEM).

MBSE Overview The *INCOSE Systems Engineering Vision 2020* (2007) defines MBSE as:

> The formalized application of modeling to support system requirements, design, analysis, verification, and validation activities beginning in the [concept stage] and continuing throughout development and later life cycle [stages].

MBSE is often contrasted with a document-based approach to SE. In a document-based SE approach, there is often considerable information generated about the system that is contained in documents and other artifacts such as specifications, interface control documents, system description documents, trade studies, analysis reports, and verification plans, procedures, and reports. The information contained within these documents is often difficult to synchronize and maintain, and difficult to assess in terms of its quality (correctness, completeness, and consistency). Although many systems have been developed using a traditional document-based approach, a model-based approach is becoming essential to address the increasing complexity of systems and support approaches that can more effectively and efficiently adapt to requirements and design changes.

MBSE enhances the ability to capture, analyze, share, and manage the information associated with the specification of a product that can result in the benefits listed below. There is some quantitative data (Rogers and Mitchell, 2021) and considerable qualitative data (OMG MBSE Wiki, 2023, MBSE Events and Related Meetings) from industry papers and presentations that support the following benefits of MBSE:

- *Improved communications* among the development stakeholders (e.g., the acquirer, project management, SE practitioners, hardware and software developers, testers, quality characteristic disciplines).
- *Increased ability to manage system complexity* by enabling the system to be viewed from multiple perspectives.
- *Improved product quality* by providing an unambiguous and precise model of the system that can be evaluated for consistency, correctness, and completeness.
- *Reduced cycle time* by enabling better control of the technical baseline, more rapid impact analysis, improved specification and design reuse, early insight for design decisions, and early discovery of potential defects.
- *Reduced risk by surfacing requirements and design issues early*.
- *Enhanced knowledge capture and reuse* of the information by capturing information in more standardized ways and reducing redundancy of information.
- *Improved ability to teach and learn SE fundamentals* by providing a clear and unambiguous representation of systems and system concepts.

MBSE Methodologies In an MBSE approach, much of the information that has been traditionally captured in informal diagrams, text, and tables is captured in a *descriptive system model* (see Section 3.2.1). This includes information about the system context, the requirements on the system and its elements, the system architecture including its structure and behavior, the critical parameters needed to specify the analysis of the system, and information about how the system is verified to satisfy its requirements. Modeling languages such as SysML™ (OMG SysML, 2021) are often used to capture this information in a standard way (see Section 3.2.1). The system descriptive model is augmented by other models, such as models to capture the system geometric configuration and various analytical models, to analyze the performance and other quality characteristics of the system. Each kind of model captures different kinds of information about the system. The different models must be managed as the design evolves to ensure a coherent representation of the overall system.

In an MBSE approach, the system descriptive model is a primary artifact of the SE process. MBSE formalizes the application of SE by creating the system descriptive model and integrating it with the other kinds of models. The kind of information and the level of detail of the information that is captured in models and maintained throughout the life cycle depends on the scope of the MBSE effort. An effective MBSE methodology supported by appropriate tools and a team with the requisite SE skills and knowledge are essential to fully realize the benefits of MBSE.

An *MBSE methodology* describes how MBSE is performed to capture the required information in the system descriptive model and related artifacts. Like any methodology, it must be tailored to the particular need of the organization and/or project (see Section 4.1). This includes defining the appropriate life cycle model, tailoring the activities and work products to align with the project scope and modeling objectives, and selecting the appropriate tools to create and manage the models and other relevant data. Estefan (2008) published a survey of candidate MBSE methodologies

under the auspices of an INCOSE technical publication. Information on these methodologies is available on the Methodology and Metrics web page of the INCOSE MBSE Wiki (2022). These methodologies and others continue to evolve based on their application to real world projects. OOSEM is summarized below as a representative MBSE method.

OOSEM Summary OOSEM is an MBSE method intended to help architect systems that satisfy evolving mission and system requirements and can accommodate changes in technology and design. OOSEM is generally consistent with processes in this handbook. It can be adapted to different life cycle models to support the specification, analysis, design, and verification of systems. The method enables the flow-down of requirements from mission, to system, to system element levels, which are realized by applicable hardware, software, data, and other discipline-specific design methods.

OOSEM describes fundamental SE activities whose outputs are model-based artifacts. The modeling artifacts are captured in a system descriptive model using the Systems Modeling Language (SysML™) along with other analytical models. A process model for OOSEM can be downloaded from the INCOSE OOSEM Working Group website (2022).

The OOSEM supports a development process that includes the following subprocesses and activities:

- *Manage the system development*—activities include plan and control the technical effort, including planning, risk management, configuration management, and other project monitoring and control activities;
- *Specify and design the system*—activities include analyze stakeholder needs, specify the system requirements, develop the system architecture, and allocate the system requirements to system elements;
- *Develop the system elements*—activities include design the elements, implement the elements, and verify the elements satisfy the allocated requirements; and
- *Integrate and verify the system*—activities include integrate the system elements and verify that the integrated system elements satisfy the system requirements.

This OOSEM process can be applied at each level of the system hierarchy to specify the requirements of the system and its elements. Applying the process recursively at successive levels of the hierarchy may involve multiple iterations throughout the development process. To be effective, the fundamental tenets of SE must be applied including the use of multi-disciplinary teams and a disciplined management process.

4.2.2 Agile Systems Engineering

The knowledge of requirements for an effective system often continues to change during the system life cycle. Common causes include insufficient initial knowledge, new knowledge revealed during development and utilization, and continual evolution in the targeted operational environment of the system. When evolution of the system's operational environment doesn't stop with initial deployment, a system's functional capabilities must evolve if it is to remain viable. Under these circumstances system engineering is virtually never ending, and retirement is generally an issue of safe and secure functional capability disposal rather than system decommissioning.

Agile Systems Engineering is a principle-based approach for designing, building, sustaining, and evolving systems when knowledge is uncertain, or environments are dynamic. Thus, Agile System Engineering is a what, not a how. As stated in Section 2.2, there are many life cycle models (e.g., Vee, Incremental Commitment Spiral Model (ICSM), DevSecOps (Development, Security, Operations)). Some of them are targeted on a single engineering domain (e.g., XP (Extreme Programming), Scrum, DevOps (Development, Operations), and various scaled approaches such as SAFe (Scaled Agile Framework) in the software engineering domain). Most of them have a strong focus on the development stage.

Agile Systems Engineering is best understood when contrasted to the sequential life cycle approach and in how the two relate to the system life cycle spectrum. Figure 4.3 shows extreme forms of these two life cycle approaches in

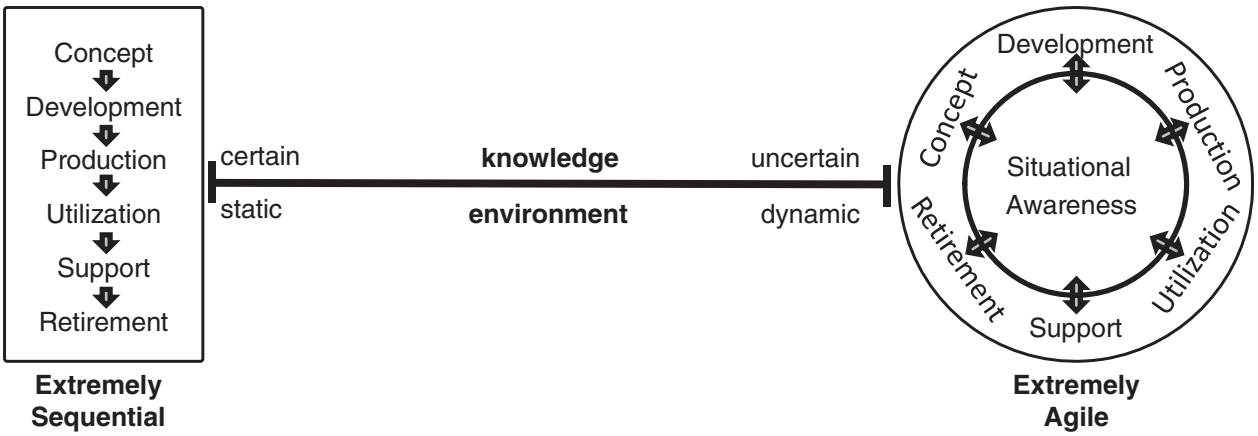


FIGURE 4.3 SE life cycle spectrum. From Dove (2022). Used with permission. All other rights reserved.

terms of their activity stages and data flows. All life cycle approaches fall somewhere between the two ends of the spectrum, depending upon the process-encoded degree of attentiveness and responsiveness to dynamics in knowledge and environment. It is unlikely that either depicted extreme would be effective in actual practice.

An Agile Systems Engineering process is based on strategies for timely and continual knowledge development and affordable application of new knowledge in system development activity. Virtually all forms of Agile Systems Engineering employ incremental or evolutionary development in some way (see Sections 2.2.2 and 2.2.3) as a means to produce demonstrable and/or usable work in process that provokes feedback for real time learning and subsequent application.

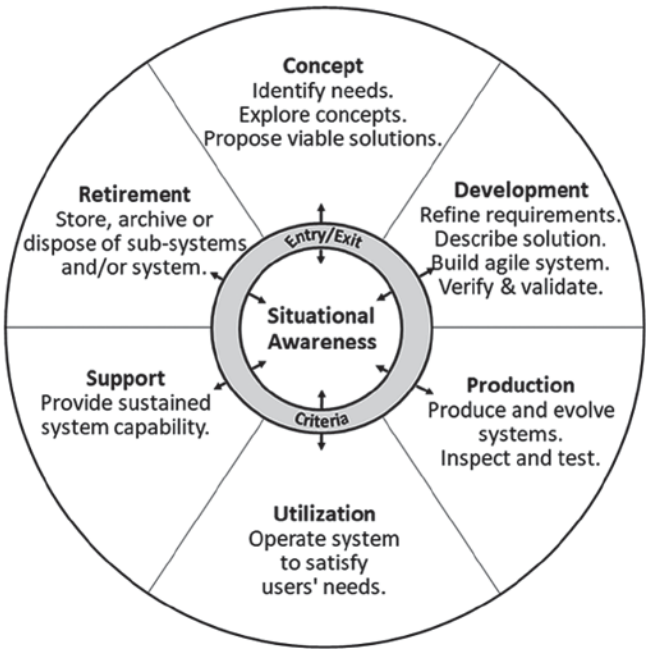


FIGURE 4.4 Agile SE life cycle model. From Dove and Schindel (2019). Used with permission. All other rights reserved.

Software plays an increasingly major role in most systems today. Codified agile software development methodologies offer relevant approaches for rapid knowledge discovery and deployment in the software domain. Patterns from these software approaches can inform agile engineering approaches in other domains and in the encompassing domain of SE; but each domain has unique differences (e.g., external dependencies, fabrication techniques, development cycle time constraints, development support tools).

Agile Systems Engineering Life Cycle Model The Agile Systems Engineering life cycle model is depicted on the far right of Figure 4.3 and with more detail in Figure 4.4. The six system life cycle stages run around the perimeter, with situational awareness featured in the center. The agile life cycle model can accommodate activities in any and all of the stages concurrently without progressive sequencing. Figure 4.4 depicts a life cycle model (Dove and Schindel, 2019), not to be confused with the Vee model (see Section 2.2.1) which depicts relationships among SE activities applicable in sequential, iterative, and evolutionary approaches.

Situational awareness has no entry or exit criteria, as it should, in principle, be a continuous activity. Entry criteria for all of the other stages begins with a decision to act upon specific triggering awareness, and may require process-prudent, or contract-required, engagement criteria for a stage or stages to be entered. An Agile Systems Engineering process is predicated upon real-time experimentation and learning in all stages, and as such, the entry criteria may be as simple as the decision to enter a stage for experimental knowledge development that may or may not produce artifacts for use in other stages. On the other hand, exit criteria for a stage that produces artifacts for use in other stages should have some fixed requirements for satisfactory stage completion, with recognition that the outcome of stage activity may simply be valuable learned knowledge that aborts the need for producing artifacts of use in other stages.

The retirement stage deals with system elements and older system versions that are retired frequently, as the "current" system evolves. This has implications for safe and secure maintenance, disposal, and reversion processes.

Fleshing out a generic Agile SE Life Cycle Model for a specific project likely starts with default standard processes in each stage, tailored and augmented for specific agile SE differences. Adapting the generic model to a specific organization's process will tailor and augment the generic model as the organization's standard process evolves.

Agile Systems Engineering Life Cycle Operational Considerations As described in Section 1.3.4, Figure 1.6 depicts the life cycle operational "pattern" as three nested systems:

- **System 1 – The Engineered System** is the target system under development.
- **System 2 – The Life Cycle Project Management System** includes the basic SE development and maintenance processes, and their operational domains that produces System-1.
- **System 3 – The Enterprise Process and Innovation System** is the process improvement system that learns, configures, and matures System-2. System 3 is responsible for situational awareness, evolution, and knowledge management, the provider of operational agility. Intent is continuous, not episodic, information flow among the three systems. Principles and strategies that facilitate operational agility in action include:

Sensing:

- External awareness (proactive alertness);
- Internal awareness (proactive alertness);
- Sense making (risk analysis, trade space analysis).

Responding:

- Decision making (timely, informed);
- Action making (invoke/configure process activity to address the situation);
- Action evaluation (verification and validation).

Evolving:

- Experimentation (variations on process ConOps);
- Evaluation (internal and external judgement);
- Memory (evolving culture, response capabilities, and process ConOps).

The architecture and structural principles that enable system agility are covered in Section 3.1.3.

Agile Systems Engineering Examples Agile system engineering methodologies are project-context dependent. What is common across methodologies are certain fundamental strategies that get tailored for a specific context. Four published examples illuminate this tailoring in four different contexts:

- As shown in Figure 2.9, Rockwell Collins employed a product line engineering (PLE) (see Section 4.2.4) approach for a large family of radio products composed of software, firmware, and electronic circuit boards with a continuous integration platform that accommodated asynchronous evolution of mixed-domain system element work in process (Dove, et al., 2017).
- The US Navy SpaWar delivered innovative off-road autonomous vehicle technology in continuous six-month development increments with parallel tracks of integration, test, and architecture evolution (Dove, et al., 2016).
- Northrop Grumman evolved user capabilities in six month increments for a software systems-of-systems single-point hub that provided access to 22 independent logistics data bases, with three successive generations under active life cycle control at all times (Dove and Schindel, 2017).
- Lockheed Martin evolved F16, F22, and F35 weapons capabilities with internally developed software and externally subcontracted hardware in roughly six month development increments in a tailored SAFe approach; featuring a continuous integration and demonstration platform with asynchronous evolution of system element simulations, low fidelity proxies, work in process, and completed system elements (Dove, et al., 2018).

4.2.3 Lean Systems Engineering

SE is regarded as an established, sound practice, but is not always delivered efficiently. US Government Accountability Office (GAO, 2008) and NASA (2007a) studies of space systems document major budget and schedule overruns. Similarly, studies by the MIT-based Lean Advancement Initiative (LAI) have identified a significant amount of waste in government projects, averaging 88% of charged time (LAI MIT, 2013; McManus, 2005; Oppenheim, 2004; Slack, 1998). Most projects are burdened with some form of *waste*: politicization, poor coordination, premature and unstable requirements, quality problems, and management frustration. This waste represents a vast productivity reserve in projects and major opportunities to improve project efficiency.

Lean system development and the broader methodology of *lean thinking* have their roots in the Toyota "just-in-time" philosophy, which aims at "producing quality products efficiently through the complete elimination of waste, inconsistencies, and unreasonable requirements on the production line" (Toyota, 2009). *Lean SE* is the application of lean thinking to SE and related aspects of organization and project management. SE is the discipline that enables flawless development and integration of complex technical systems. Lean thinking is a holistic paradigm that focuses on delivering maximum value to the customer and minimizing waste. A popular description of lean is "doing the right job right the first time." Lean thinking has been successfully applied in manufacturing, healthcare, administration, supply chain management, and product development, including engineering.

Lean SE is the area of synergy between lean thinking and SE, with the goal to deliver the best life-cycle value for technically complex systems with minimal waste. The early use of the term lean SE is sometimes met with concern that this might be a "repackaged faster, better, cheaper" initiative, leading to cuts in SE at a time when the profession is struggling to increase the level and quality of SE effort in projects. Lean SE does not take away anything from SE and it does not mean *less* SE. It means *better* SE with higher responsibility, authority, and accountability, leading to better, waste-free workflows with increased mission assurance.

Three concepts are fundamental to the understanding of lean: value, waste, and the process of creating value without waste (captured into lean principles).

Value The value proposition in engineering projects is often a multiyear, complex, and expensive acquisition process involving numerous stakeholders and resulting in hundreds or even thousands of requirements, which, notoriously, are

rarely stable. In lean SE, *value* is defined simply as mission assurance (i.e., the delivery of a flawless complex system, with flawless technical performance, during the product or mission development life cycle) and satisfying the customer and all other stakeholders. This implies completion with minimal waste, minimal cost, and the shortest possible schedule.

Waste in Product Development *Waste is* "the work element that adds no value to the product or service in the eyes of the customer. Waste only adds cost and time" (Womack and Jones, 1996). The LAI classifies waste into seven categories (McManus, 2005). An eighth category, the waste of human potential, is increasingly added. These categories are defined and illustrated as follows.

- *Overprocessing*—Processing more than necessary to produce the desired output; excessive refinement, beyond what is needed for value.
- *Waiting*—Waiting for people, material or information, or people waiting for information or material; late delivery of material or information, or delivery too early—leading to eventual rework.
- *Unnecessary movement*—Moving people (or people moving) unnecessarily to access or process material or information; unnecessary motion in the conduct of the task; lack of direct access; manual intervention.
- *Overproduction*—Creating too much material or information; performing a task that nobody needs; information over-dissemination and pushing data.
- *Transportation*—Moving material or information unnecessarily; unnecessary hand-offs between people; incompatible communication—lost transportation through communication failures.
- *Inventory*—Maintaining more material or information than needed; too much "stuff" buildup; complicated retrieval of needed "stuff"; outdated, obsolete information.
- *Defects*—Errors or mistakes causing the effort to be redone to correct the problem.
- *Waste of human potential*—Not utilizing or even suppressing human enthusiasm, energy, creativity, and ability to solve problems and general willingness to perform excellent work.

Lean Principles and Lean Enablers for Systems Engineering Womack and Jones (1996) captured the *process of creating value without waste* into six *lean principles* described in (Oppenheim, 2011), as follows:

The *value principle* promotes a robust process of establishing the value of the system to the customer with crystal clarity early in the project. The process should be customer-centric, involving the customer frequently and aligning employees accordingly.

The *value stream principle* emphasizes detailed project planning and waste-preventing measures, solid preparation of the personnel and processes for subsequent efficient workflow, and healthy relationships between stakeholders (e.g., acquirer, contractor, suppliers, and employees); project frontloading; and use of leading indicators and quality measures. SE practitioners should prepare for and plan all end-to-end linked actions and processes necessary to realize streamlined value, after eliminating waste.

The *flow principle* promotes the uninterrupted flow of robust quality work and first-time-right products and processes, broad steady competence instead of hero behavior in crises, excellent communication and coordination, concurrency, frequent clarification of the requirements, and making project progress visible to all.

The *pull principle* is a powerful guard against the waste of rework and overproduction. It promotes pulling tasks and outputs based on internal and external customer needs (and rejecting others as waste), and better coordination between the pairs of employees handling any transaction before their work begins so that the result can be first-time right.

The *perfection principle* promotes excellence in the SE and organization processes, utilization of the wealth of lessons learned from previous projects into the current project, the development of perfect collaboration policy across people and processes, and driving out waste through standardization and continuous improvement. Imperfections should be made visible in real time, and continuous improvement tools (Six Sigma) should be applied as soon as possible. It calls for a more important role of SE practitioners, with responsibility, accountability, and authority for the overall technical success of the project.

Finally, the *respect-for-people principle* promotes the enterprise culture of trust, openness, honesty, respect, empowerment, cooperation, teamwork, synergy, and good communication and coordination and enables people for excellence.

In 2011, a project undertaken jointly by PMI, INCOSE, and the LAI at MIT developed the Lean Enablers for Managing Engineering Programs (LEfMEP, 2012), adding lean enablers for project management and holistically integrating lean project management with lean SE. A major section of the book is devoted to a rigorous analysis of challenges in managing engineering projects. They are presented under the following 10 challenge themes:

1. Firefighting—reactive project execution;
2. Unstable, unclear, and incomplete requirements;
3. Insufficient alignment and coordination of the extended enterprise;
4. Processes that are locally optimized and not integrated for the entire enterprise;
5. Unclear roles, responsibilities, and accountability;
6. Mismanagement of project culture, team competency, and knowledge;
7. Insufficient project planning;
8. Improper metrics, metric systems, and key performance indicators;
9. Lack of proactive project risk management; and
10. Poor project acquisition and contracting practices.

4.2.4 Product Line Engineering (PLE)

Rarely does anyone build just one edition, just one flavor, just one point solution of anything. In many cases, SE is performed in the context of a product line—a family of similar systems with variations in features and functions. *Product Line Engineering* (PLE) addresses this mismatch by providing models, tools, and methods for holistic engineering of system families.

A note on terminology: Where the PLE field and standards refer to "product," "product line," and "product line engineering," the equivalent terms in SE are "system," "system family," and "system family engineering," respectively. These terms can be used interchangeably (Krueger, 2022).

Challenges with Early Generation System Family Engineering Approaches When systems in a *system family* are engineered as individual point solutions, techniques such as *clone-and-own reuse* or *branch-and-merge* result in ever-growing duplicate and divergent engineering effort. Trying to manage the commonality and variability among these individually engineered systems in the family has traditionally relied on tribal knowledge and high bandwidth, error-prone interpersonal communication. Furthermore, when each engineering discipline adopts a different ad hoc technique for managing variations among the members of the system family, the result is error prone dissonance when trying to translate and communicate across the different life cycle disciplines.

This is a self-inflicted complexity, over and above the complexity inherent in the systems being engineered. It consumes engineering teams with low-value, trivial, replicative work that deprives them of time and energy that would be better spent on high-value innovative work that advances system and business objectives.

Feature-based Product Line Engineering *Feature-based PLE* is the modern digital engineering industry good practice for PLE, as defined in the INCOSE PLE Primer (2019) and ISO/IEC 26580 (2021). Feature-based PLE offers significant improvements and benefits in effort, cost, time, scale, and quality by exploiting system similarity while formally managing variation.

Feature-based PLE is used to engineer a system family as a single holistic system rather than a multitude of individual systems. Engineering assets in each engineering discipline are consolidated to eliminate duplication and

divergence. A single authoritative variation management model is applied consistently across all assets in all engineering disciplines to eliminate that source of dissonance across the life cycle and to enable organizations to make informed and deliberate cost-benefit decisions about the variations designed into their system family.

Key Elements of a Feature-based PLE Factory Feature-based PLE uses a *PLE Factory* metaphor, as illustrated in Figure 4.5. See ISO/IEC 26580 (2021) for a full description.

- **Feature Catalogue**, as shown in the upper left, captures a formal model of the distinguishing characteristics about how the members of the system family differ from each other and provides a common language and single authoritative source of truth about variation throughout the engineering organization.
- **Bill-of-Features**, as shown in the upper right, specifies the features selected from the Feature Catalogue for each system in a system family portfolio.
- **Shared Asset Supersets**, as shown in the lower left, are the engineering artifacts that support the creation, design, implementation, deployment, and operation of systems in a system family. They contain *variation points*, which are pieces of content that can be included, omitted, generated, or transformed for a system instance, based on the features selected in a Bill-of-Features for that system.
- **PLE Factory Configurator**, shown in the center, is an automation that applies a Bill-of-Features for a system to each variation point in the Shared Asset Supersets, to determine each variation point's content for the system instance.
- **Product Asset Instances**, shown in the lower right, each contain only the shared asset content suited for that one system in the system family.

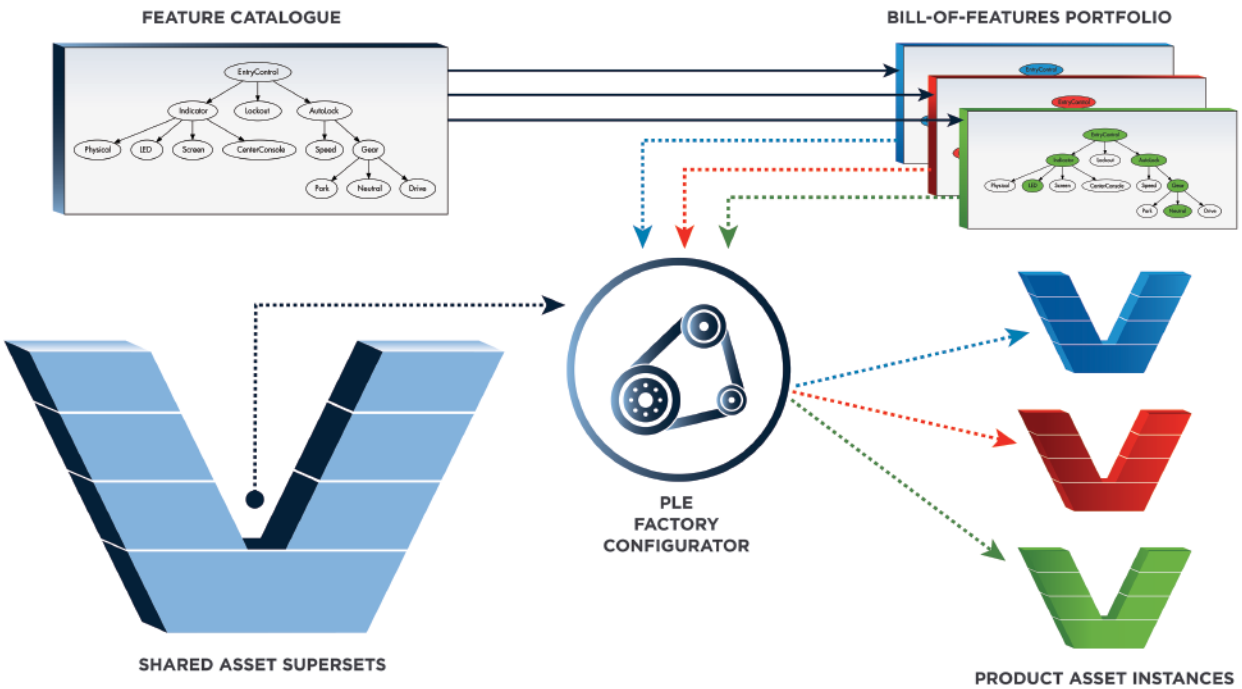


FIGURE 4.5 Feature-based PLE factory. From INCOSE PLE Primer (2019). Usage per the INCOSE Notices page. All other rights reserved.

With Feature-based PLE, engineers now work in the PLE Factory on the Shared Asset Supersets, the Feature Catalogue, and the Bills-of-Features rather than on the individual system instances. Once the PLE Factory is established, engineering assets for the individual systems are automatically instantiated rather than manually engineered. Feature-based PLE transforms the task of engineering a plethora of individual systems into the much more efficient task of producing a single system: The PLE Factory itself. This consolidation also means that change management and configuration management are performed on the single PLE Factory rather than separately on each of the system instances.

Shared Asset Supersets and Variation Points To work in a PLE Factory, engineers must learn how to create and maintain Shared Asset Supersets, including variation points, for their discipline. For example, requirements engineers learn how to create requirements Shared Asset Supersets with variation points, test engineers learn how to create verification and validation Shared Asset Supersets with variation points, and software engineers learn how to create source code Shared Asset Supersets with variation points.

A Shared Asset Superset contains a single copy of all content used in any system—that is, there is no duplication of content. Content that appears in every system is said to be *common* content, while content that varies from system to system is encapsulated in a variation point. Consistent treatment of variation points in Shared Asset Supersets across all disciplines is a hallmark of Feature-based PLE. Variation points are places in an asset that denote content that is configured according to feature selections in a Bill-of-Features for a particular system instance. Variation point configuration mechanisms typically include selection or omission of the content; selection from among mutually exclusive content alternatives; generation of content based on feature specifications; and feature-based transformation of content from one form into another.

Perhaps one of the easiest examples of Shared Asset Supersets to understand is requirements. A superset of requirements combines individual system requirements to establish all of the system family requirements. Variation points express inclusion and omission, define mutual exclusion, and transform requirement wording in the system specification—all based on feature selections. Requirement transformation can replace numbers, units, or other text with information that is derived from the Bill-of-Features. Requirements that have no variation are common and apply to every system.

MBSE models can be developed as Shared Asset Supersets and instrumented with variation points. For example, system design or architecture models using SysML™ include variation points to specify optional, mutually exclusive, and varying structural elements such as blocks, ports, relationships, objects, classes, activities, states, transitions, use-cases, packages, and others, as well as generation or transformation of values, attributes, and constraints associated with those elements.

Shared Asset Supersets for Electronic Design Automation, Mechanical Design Automation, and Computer-aided Design (CAD) for electronic, mechanical, mechatronic, and cyber-physical systems take the form of supersets of parts, properties, relationships, assemblies, system elements, circuit boards, wiring harnesses, and more. Variation points instrument optional, mutually exclusive, and varying content in these models.

In software systems, Shared Asset Supersets are constructed for source code, resources, and build scripts. Source code variation points can be defined in several ways, including blocks of code, optional or mutually exclusive source files, and macro substitutions.

Verification and validation Shared Asset Supersets for automated and manual test plans and test cases can be instrumented with variation points to identify and configure the tests for each system, based on feature selections. It is possible to streamline or even eliminate redundant testing of common capability across multiple systems in the system family.

A broad array of additional assets with digital representations can serve as Shared Asset Supersets in system families. These include system budgets or cost models, schedules and work plans, user manuals and installation guides, process documentation, marketing brochures, simulation models, system descriptions, digital twins, supply chain orders, manufacturing specs, contract proposals, and much more. Feature-based PLE can be applied to all SoI types defined in Section 4.3.

Organizational Change and Return-on-Investment with Feature-based Product Line Engineering For many organizations, Feature-based PLE represents a shift in engineering approach that requires organizational change, along with commitment from engineering and business leadership to make that change. The ROI to justify the organizational change is in most cases compelling, based on the elimination of low-value, mundane, replicative work, with doubling, tripling and larger improvements in engineering metrics such as: lowering engineering complexity; reducing overall engineering time, cost, and effort; increasing portfolio scalability; and improving system quality (Gregg, et al., 2015 and McNicholas, 2021). In consideration of this ROI, the question to leadership is, "What if your engineers could do their normal day's work before lunch; what would you have them do in the afternoon?" There are many answers to this question, all of them good.

4.3 SYSTEM TYPES CONSIDERATIONS

The concept of SoI was introduced in Section 1.3.1. The type of SoI has significant implications on SE. This section introduces SE considerations for the following types of SoIs:

- Greenfield/Clean Sheet Systems
- Brownfield/Legacy Systems
- Commercial-off-the-Shelf (COTS)-Based Systems
- Software-Intensive Systems
- Cyber-Physical Systems (CPS)
- System of Systems (SoS)
- Internet of Things (IoT)/Big Data-Driven Systems
- Service Systems
- Enterprise Systems

Note that other types of SoIs exist.

4.3.1 Greenfield/Clean Sheet Systems

"Greenfield" and "brownfield" are terms used in real estate. Greenfield land is previously undeveloped space, such as a (green) farmer's field. Brownfield land has been previously developed, typically has existing structures and services in place, and may contain undesirable or hazardous materials (also known as waste) that must be remediated. Greenfield SE, also known as "clean sheet" or "blank slate" SE, involves systems that are new designs and have no, or limited, legacy systems constraints, other than system interfaces. Given the incremental and spiral development life cycle approaches of today, a greenfield system may evolve toward brownfield even before it is delivered, from the developer's perspective.

Traditionally, SE has been taught by considering systems from a greenfield perspective. One starts with a "clean or blank sheet of paper" and determines the set of stakeholders and their needs and requirements, translates them into system requirements, architects and designs a system solution, implements the system elements, and then integrates, verifies, and validates the system elements and the system solution. While this is an effective way to teach SE and to prepare practitioners with skills applicable to the entire system life cycle, few system development efforts are truly greenfield. Greenfield, therefore, is an almost theoretical situation that is rarely seen in practice. The remaining considerations provide different perspectives and implications beyond greenfield SE.

Sometimes, it can be quite traumatic for organizations that make brownfield updates to its legacy products over a long period of time to transition from brownfield development back to greenfield (Axehill, 2021). They may need to "relearn" how to do greenfield.

4.3.2 Brownfield/Legacy Systems

As described in Section 4.3.1, brownfield (and greenfield) are terms used in real estate. Brownfield land has been previously developed, typically has existing structures and services in place, and may contain undesirable or hazardous materials (also known as waste) that must be remediated. Brownfield SE, also known as "legacy" SE, involves significant modifications, extensions, or replacement of an existing "as-is" system in an existing environment to an updated "to-be" system. Brownfield systems often contain waste (e.g., technical debt) that may need to be remediated (Seacord, et al., 2003) (Hopkins and Jenkins, 2008). "In-service" systems are another example of brownfield system (Kemp, 2010) (Van De Ven, et al., 2012). Brownfield systems typically have explicit continuity requirements, where the operation of the as-is system needs to continue, resulting in a deliberate transition to the updated system.

The nature of greenfield and brownfield systems drives different life cycle approaches that reflect different areas of emphasis. Table 4.1 lays out some of the key differences across a set of aspects important to SE (Walden, 2019) (Baley and Belcham, 2010). This impacts not only the system solution, but also the team that is put in place to develop the system. As with all development efforts, SE processes need to be tailored to fit the needs of a given project (see Section 4.1). SE in a brownfield environment augments the SE life cycle processes described in this handbook with site surveys and reconstruction activities to understand the as-is systems, identify gaps, and engineer the to-be system (Walden, 2019).

TABLE 4.1 Considerations of greenfield and brownfield development efforts

Aspect	Greenfield	Brownfield
Life Cycle Stage(s) (of Initial SoI)	Concept, Development	Utilization, Support
Focus	New or novel features	Maintenance or adding new features while retaining select legacy functionality
Maturity (of Initial SoI)	Low to Moderate	High for maintenance; Mix for existing system and environment, plus new development for upgrade or replacement
Architecture and Design Review	Reviewed and modified at multiple levels	Reviewed only when significant updates are made/performed
Verification	The entire SoI typically needs to verified	Only the updated and impacted parts of the system need to be verified (there may be regression testing for the unchanged parts)
Validation	The entire SoI typically needs to validated with the customer/user	The entire SoI (including changes) typically needs to validated with the customer/user to check for new emergent behavior
Manufacturing/ Production	May be in place if using the existing line, or is developed (or tailored) as development progresses	Mostly in place, reverse engineering of existing designs may be required if the original design can no longer be produced (e.g., due to as-is use of banned materials)
Maintenance and Logistics	Developed (or tailored) as development progresses	Mostly in place, but may need changes or upgrades depending on the replacement system elements
Practices and Processes	Developed (or tailored) as work progresses	Mostly in place, though not necessarily relevant to the new team
Team Composition	Newly formed group	Mix of old and new, bringing both historical biases and fresh ideas

From Walden (2019) derived from Baley and Belcham (2010). Used with permission. All other rights reserved.

4.3.3 Commercial-off-the-Shelf (COTS)-Based Systems

One of the key trade-off studies SE practitioners perform is the "make vs. buy" decision on system elements. "Make" represents custom-built solutions; "buy" represents outsourced development and commercial-off-the-shelf (COTS) solutions. Directed use also can result in COTS. Most systems have some COTS content. The following characteristics can be useful when deciding if a particular system or system element can be characterized as COTS (Oberndorf, et al., 2000) (Tyson, et al., 2003):

- Sold, leased, or licensed to the general public;
- Offered by a vendor trying to profit from it;
- Supported and evolved by the vendor, who retains the intellectual property rights;
- Vendor (not acquirer) controls the frequency of the product's maintenance and updates;
- Available in multiple, identical copies;
- Used without hardware or source code modification.

The promise of COTS is to save development time, reduce technical risk, reduce time-to-market, reduce cost-to-market, and take advantage of latest technology. However, often these promises are not realized. Considerations for COTS-based systems include (Long, 2000):

- COTS products not built to your specific requirements (including missing functionality, extra functionality, and unwanted behaviors).
- Unique, or different than expected, interfaces, including a vendor's use of proprietary data formats and/or communications protocols, may occur.
- Vendor claims and decisions may impact schedule.
- The details needed to understand how COTS products may impact the safe and secure operation of the SoI may not be readily available, including the trustworthiness of the vendor, the use of open-source software, and the use of third-party software of unknown origin.
- COTS product may be insufficiently documented.
- "Not Invented Here" (NIH) syndrome may deter engineers from using COTS.
- Delivery times may not be met.
- Special integration challenges may occur.
- COTS products are often not verified to your specific requirements and may lack verification data for the operating environment for the SoI.
- "Sole Source" suppliers result in more risk.
- Need to consider the entire life cycle cost (LCC) of maintenance and technology rolls/refresh due to COTS obsolescence and diminishing manufacturing sources and material shortages (DMSMS) (Note: IEC 62402 (2019) provides guidance for establishing a framework for obsolescence management process which is applicable through all stages of system life cycle).

There are differences in approaches to SE for COTS-based systems development. Some of the key COTS-based SE considerations are shown in Table 4.2. Effective use of COTS generally requires COTS evaluation starting during needs analysis. In some circumstances, an "internal sales pitch" for each viable candidate COTS-based system needs to be developed, highlighting which requirements are met, which are partially or not met, and what additional capabilities and cost advantages (now and potentially in the future) each possible system provides. For an organization which

TABLE 4.2 Considerations for COTS-based development efforts

Aspect	Traditional Systems Engineering	COTS-Based Considerations
Focus	The SoI	The SoI as well as how potential COTS products in the marketplace could be assembled to meet most/all the needs
Stakeholder Needs and Requirements	Fairly explicit stakeholder requirements	Flexible and prioritized capabilities stated in broad terms
System Requirements and Functionality	Requirements and functionality are defined and allocated based on technical considerations	COTS capabilities and functionality form the basis for the system requirements allocation and evolution and COTS may introduce additional system-level constraints
System Element Requirements	Extra or missing system element requirements are typically bad	Need to strike a balance between what the system needs and what the market can provide, missing or extra COTS requirements may be a reality due to the marketplace and they may also necessitate extra COTS wrappers and glue
System Architecture and Design	Focus is on optimizing the SoI	Focus is on optimizing the set of COTS and custom components that make up the SoI
Integration, Verification, and Validation	Done with known (internal) system element owners	Can typically get an early version of the system up and operating dramatically sooner than with a "make" system, criteria more difficult to establish since COTS performance must satisfy the market requirements while balancing the needs of the system, execution and defect resolution more difficult due to external COTS element owners
Technical Management	Well understood set of processes	More challenging decision environment, additional risks are present for COTS, and potentially increased configuration and information management (CM and IM) activities
Agreements	Acquisition agreements are primarily outsourced development efforts	Acquisitions also include COTS items and must consider other aspects such as licensing, additional COTS vendor support, and obsolescence management
Quality Characteristics	Known with internal team support and data, but the full picture may not be obtainable until after system deployment	For proven COTS, can be determined up front, may have to rely on COTS vendors for some data, consideration of life cycle cost (LCC) is critical for COTS

has only done "make" system development in the past, moving to a COTS-based development requires a different mind-set and different development skills, including the new role of COTS integrator. These skills are often different than those needed for non-COTS-based system development.

4.3.4 Software-Intensive Systems

ISO/IEC/IEEE 42010 (2022) defines a software-intensive system as:

> Any system where software contributes essential influences to the design, construction, deployment, and evolution of the system as a whole.

Software, like physical entities, is encapsulated in system elements. Software elements often contribute to system functionality, behavior, quality characteristics, interfaces, and observable performance indicators for software-intensive systems. Encapsulated software is sometimes custom-built and is sometimes obtained by using software

elements from libraries. Reused software elements may be modified by tailoring them for intended use. Also, software elements may be licensed from software vendors. In most cases, licensed software packages cannot be modified. Software engineers sometimes encapsulate licensed software packages in software shells or wrappers that provide the interfaces needed to integrate the needed capabilities of the software into a system while masking unwanted capabilities.

Software is widely incorporated in systems and provides "essential influences" because software is a malleable entity composed of textual and iconic symbols that in many cases, but not always, can be constructed, modified, or procured sooner and at less expense than fabricating, modifying, or procuring physical elements that have equivalent capabilities. In some cases, but not always, software elements can provide capabilities that would be difficult to realize in hardware.

But in some cases, a physical element of equivalent capability may be preferred to a software element because software may not provide certifiable safety, security, or performance at the necessary levels of assurance. Making tradeoffs between physical elements and software elements is an essential aspect of developing and modifying software-intensive systems. Trade studies are best conducted when SE practitioners consult physical engineers and software engineers who are working collaboratively.

Interfaces provided by software in a software-intensive system can provide passive pass-through connections or, if desired, software included in an interface can actively coordination interactions among the connected elements. Software interfaces may be internal and/or external to a system. Internal software interfaces can provide connections that coordinate interactions among physical elements, among software elements, and between physical elements and software elements. External software interfaces can provide passive and active connections to entities in the physical environment, including humans and other sentient entities that are software-enabled, plus connections to the external interfaces of other physical-only, software-only, and software-intensive systems. A "software-only" system is a software application implemented on a stable computing platform. The computing platform includes hardware and software that support the software application. The platform is often implemented using commodity items.

Sometimes the essential influences of software are observable in a system's external interfaces, as in human-user interface displays, and sometimes not, as in the interfaces for direct interaction with an external system. In the latter case, the software is said to be embedded in the SoI because performance of the interface is not directly observable by a human, although the effects of interface performance may be observable. Software is also used to provide interfaces among the systems that constitute an SoS. Software can contribute to functionality, behavior, quality attributes, external interfaces, and performance indicators for a composite SoS.

Developing and modifying software-intensive systems presents challenges for SE practitioners when partitioning system requirements and elements of the architecture, allocating performance parameters to physical elements and to software elements, establishing and controlling physical-software interfaces, and facilitating integration of physical elements with software elements (Fairley, 2019). These challenges sometimes arise because the culture, terminology, processes, and practices of software engineering are unfamiliar to SE practitioners and conversely, the various aspects of SE may be unfamiliar to software engineers. Techniques for improving communication between SE practitioners and software engineers are presented in Section 5.3.1 and in Fairley (2019).

4.3.5 Cyber-Physical Systems (CPS)

CPS are the integration of physical and cyber (software) processes in which the software monitors and controls the physical processes and is, in turn, affected by them. CPS are enabled by sensors and feedback loops and their provenance has increased because of significant advances in sensor technology and affordability. Figure 4.6 illustrates the behavior of a CPS: sensors may be deployed in the system hardware and/or its environment. Sensor data is used by software algorithms to control the hardware in responses to changes in the environment and/or in the hardware itself. The algorithms control the dynamic behavior of the CPS to achieve one or more goals, which could include homeostasis (maintaining equilibrium). An example might be an automobile with sensors to detect obstacles (in the environment) and take evasive action if an obstacle is detected ahead (the actuators would be steering or braking). Other

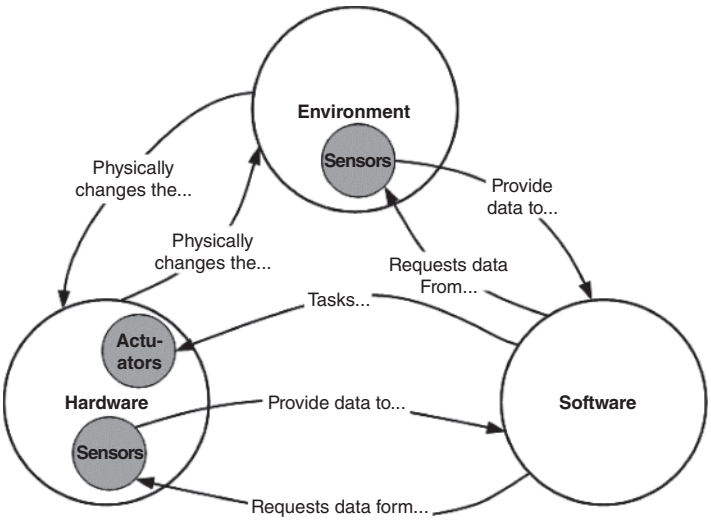


FIGURE 4.6 Schematic diagram of the operation of a Cyber-Physical System. INCOSE SEH original figure created by Henshaw. Usage per the INCOSE Notices page. All other rights reserved.

sensors may be deployed to monitor the health of the vehicle, for instance, wear of the brake pads may trigger an action to modify the driving or notify the owner of the need for maintenance. Digital twins are also an important concept in this regard that refers to a digital surrogate that is a dynamic physic-based description of physical assets (physical twin), processes, people, places, systems, and devices that can be used for various purposes. The digital representation provides both the elements and the dynamics of how an Internet of Things (IoT) device operates and lives throughout its life cycle (see Section 4.3.7).

The CPS concept is closely aligned to Industry 4.0, an initiative to revolutionize industry through so-called smart systems (Kagermann, 2013). CPS always include both software and hardware and are almost always networked, in which case they are Cyber-Physical Systems of Systems (CPSoS). If the individual CPS are networked using Internet Protocols, then they form part of an IoT, but they may interact through other mechanisms (e.g., mechanical, electromagnetic, thermal). The relationships between the concepts of SoS, CPS, and IoT are illustrated in Figure 4.7. The "Things" in IoT are constituent systems and they are always networked, and therefore, always an SoS.

CPS are a feature of almost every industry and many aspects of society. They provide automated, and even autonomous, control of technologies ranging from robotic manufacture to SMART cities and from automated insulin delivery to control of critical infrastructure. They may form a contribution to business models that include physical systems and can even underpin a business concept by providing resilience or cost savings.

There are many implications of CPS for engineers. Two of the most significant are complexity and ethics. The increased level of complexity is due to both extensive networks and the problem of modeling the combination of physical dynamics with computational processes. Lee (2015) has pointed out that such models are nearly always nondeterministic due to the lack of temporal semantics in the cyber and physical modeling programs. This has significant implications for modeling and verification within the SE processes. The lack of determinism, together with questions concerning the transfer of decision making from humans to machines have ethical challenges that engineers must face in terms of definition and realization of CPS (European Parliament Research Service, 2016).

The scope and dimensions of CPS is well-illustrated by the CPS Concept Map (Asare, et al., 2012), which essentially defines CPS as feedback systems that are applicable across a wide range of applications such as infrastructure, healthcare, manufacturing, military and many more. One might consider them relevant to any application in which the system is required to be dynamic and the control thereof is managed by software. The concept map also highlights the explicit need for security and safety considerations in design as well as the challenges of verification and validation in large complex systems. It points toward the need for improved modeling and, indeed, progress has been made in the use of MBSE for CPS development through a framework to implement suitable tool chains (Lu, 2019).

Given that software forms an integral part of many systems and devices, it can reasonably be stated that SE practitioners are very often concerned with CPS and usually CPSoS.

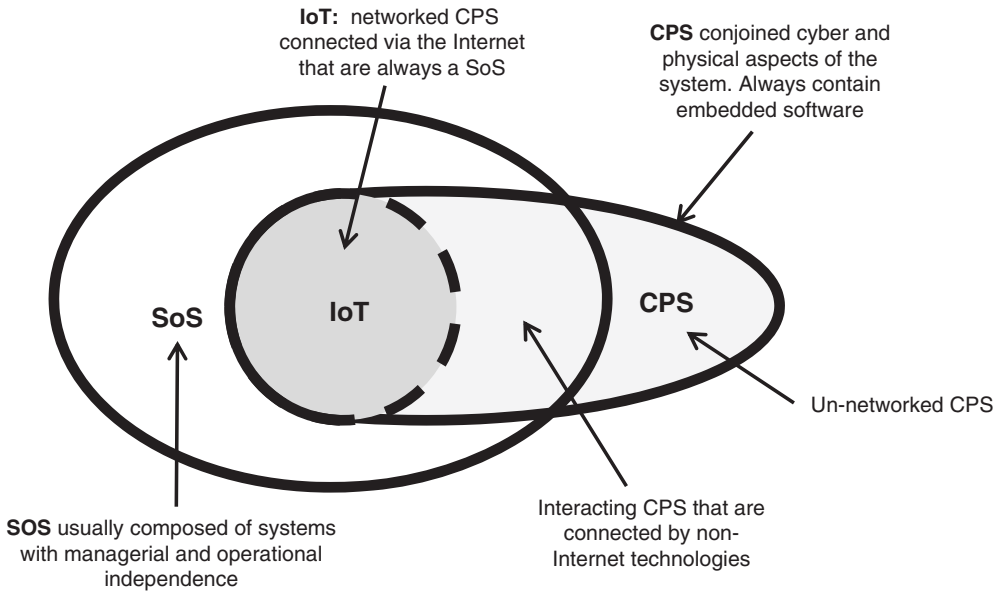


FIGURE 4.7 The relationship between Cyber-Physical Systems (CPS), Systems of Systems (SoSs), and an Internet of Things (IoT). From Henshaw (2016). Used with permission. All other rights reserved.

4.3.6 Systems of Systems (SoS)

ISO/IEEE/IEC 21839 (2019) defines a System of Systems (SoS) as:

> A set of systems or system elements that interact to provide a unique capability that none of the constituent systems can accomplish on its own.

Constituent systems can be part of one or more SoS. Each constituent system is a useful system by itself, having its own development, management goals, and resources, but interacts within the SoS to provide the unique capability of the SoS (ISO/IEEE/IEC 21839, 2019).

The following characteristics can be useful when deciding if a particular SoI can better be understood as an SoS (Maier, 1998):

- Operational independence of constituent systems;
- Managerial independence of constituent systems;
- Geographical distribution;
- Emergent behavior;
- Evolutionary development processes.

Of these, operational independence and managerial independence are the two principal distinguishing characteristics for applying the term SoS.

Figure 4.8 illustrates the concept of an SoS. The air transport system is an SoS comprising multiple aircraft, airports, air traffic control systems, and ticketing systems, which along with other systems such as security and financial

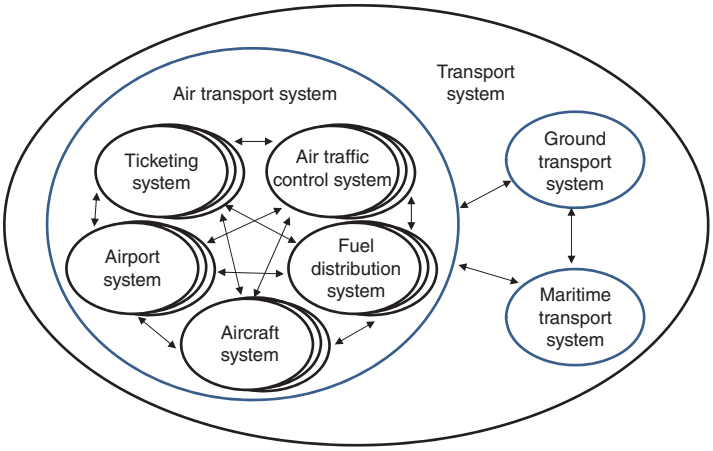


FIGURE 4.8 Example of the systems and systems of systems within a transport system of systems. From ISO/IEC/IEEE 21841 (2019). Used with permission. All other rights reserved.

systems facilitate passenger transportation. There are equivalent ground and maritime transportation SoS that are all in turn part of the overall transport system (an SoS in the terms of this description).

There are three international standards that provide useful guidance on SoS:

- ISO/IEC/IEEE 21839 (2019)—"System of systems (SoS) considerations in life cycle stages of a system" focuses on the SE of an individual constituent system and identifies considerations to be addressed as the engineering of the systems progresses from concept through retirement.
- ISO/IEC/IEEE 21840 (2019)—"Guidelines for the utilization of ISO/IEC/IEEE 15288 in the context of system of systems (SoS)" provides guidance on the application of the processes to the special case of SoS, including considerations for how constituent systems relate within the SoS.
- ISO/IEC/IEEE 21841 (2019)—"Taxonomy of systems of systems" lays out a taxonomy of SoS types based on authority relationships as (shown in Table 4.3)

ISO/IEC/IEEE 21840 (2019) guidance on application of ISO/IEC/IEEE 15288 (2023) life cycles processes (see Section 2.3) is based on the differences between systems and SoS and the impact on the SE processes as shown in Table 4.4.

Dahmann (2014) identified the following challenges that influence the engineering of an SoS:

- *SoS authorities*—In an SoS, each constituent system has its own local "owner" with its stakeholders, users, business processes, and development approach. As a result, the type of organizational structure assumed for most traditional SE under a single authority responsible for the entire system is absent from most SoS. In an SoS, SE relies on crosscutting analysis and on composition and integration of constituent systems, which in turn depend

TABLE 4.3 SoS types

Directed	The SoS is created and managed to fulfill specific purposes and the constituent systems are subordinated to the SoS. The constituent systems maintain an ability to operate independently; however, their normal operational mode is subordinated to the central managed purpose.
Acknowledged	The SoS has recognized objectives, a designated manager, and resources for the SoS; however, the constituent systems retain their independent ownership, objectives, funding, and development and sustainment approaches. Changes in the systems are based on cooperative agreements between the SoS and the constituent systems.
Collaborative	The constituent systems interact more or less voluntarily to fulfill agreed upon central purposes. The central players collectively decide how to provide or deny service, thereby providing some means of enforcing and maintaining standards.
Virtual	The SoS lacks a central management authority and a centrally agreed upon purpose for the SoS. Large-scale behavior emerges-and may be desirable-but this type of SoS must rely on relatively invisible mechanisms to maintain it.

From ISO/IEC/IEEE 21841 (2019) derived from SEBOK. Used with permission. All other rights reserved.

TABLE 4.4 Impact of SoS considerations on the SE processes

SE Process	Implementation as Applied to SoS
Agreement Processes	Because there is often no top level SoS authority, effective agreements among the systems in the SoS are key to successful SoSE.
Organizational Project Enabling Processes	SoSE develops and maintains those processes which are critical for the SoS within the constraints of the system level processes.
Technical Management Processes	SoSE implements Technical Management Processes applied to the particular considerations of SoS engineering - planning, analyzing, organizing, and integrating the capabilities of a mix of existing and new systems into a system of systems capability while systems continue to be responsible for technical management of their systems.
Technical Processes	SoSE Technical Processes define the cross-cutting SoS capability, through SoS level business or mission analysis and stakeholder needs and requirements definition. SoS architecture and design frame the planning, organization, and integration of the constituent systems, constrained by system architectures. Development, integration, verification, transition, and validation are implemented by the systems. with SoSE monitoring and review. SoSE integration, verification, transition and validation applies when constituent systems are integrated into the SoS and performance is verified and validated.

on an agreed common purpose and motivation for these systems to work together toward collective objectives that may or may not coincide with those of the individual constituent systems.

- *Leadership*—Recognizing that the lack of common authorities and funding poses challenges for SoS, a related issue is the challenge of leadership in the multiple organizational environments of an SoS. This question of leadership is experienced where a lack of structured control normally present in SE requires alternatives to provide coherence and direction, such as influence and incentives.
- *Constituent systems*—SoS are typically composed, at least in part, of in-service systems, which were often developed for other purposes and are now being leveraged to meet a new or different application with new objectives. This is the basis for a major issue facing the application of SE to SoS, that is, how to technically address issues that arise from the fact that the systems identified for the SoS may be limited in the degree to which they can support the SoS. These limitations may affect initial efforts at incorporating a system into an SoS, and systems' commitments to other users may mean that they may not be compatible with the SoS over time. Further, because the systems were developed and operate in different situations, there is a risk that there could be a mismatch in understanding the services or data provided by one system to the SoS if the particular system's context differs from that of the SoS.
- *Capabilities and requirements*—Traditionally (and ideally), the system engineering process begins with a clear, complete set of initial user requirements and provides a disciplined approach to develop and evolve a system to meet these and emerging requirements. Typically, SoS are composed of multiple independent systems with their own requirements, working toward broader capability objectives. In the best case, the SoS capability needs are met by the constituent systems as they meet their own local requirements. However, in many cases, the SoS needs may not be consistent with the requirements for the constituent systems. In these cases, SE of an SoS needs to identify alternative approaches to meeting those needs either through changes to the constituent systems or through the addition of other systems to the SoS. In effect, this is asking the systems to take on new requirements with the SoS acting as the "user."
- *Autonomy, interdependence, and emergence*—The independence of constituent systems in an SoS is the source of a number of technical issues when applying SE to an SoS. The fact that a constituent system may continue to change independently of the SoS, along with interdependencies between that constituent system and other

constituent systems, adds to the complexity of the SoS and further challenges SE at the SoS level. These dynamics can lead to unanticipated effects at the SoS level leading to unexpected or unpredictable behavior in an SoS even if the behavior of the constituent systems is well understood.

- *Testing*—The fact that SoS are typically composed of constituent systems that are independent of the SoS poses challenges in conducting end-to-end SoS testing, as is typically done with systems. First, unless there is a clear understanding of the SoS-level expectations and measures of those expectations, it can be very difficult to assess the level of performance as the basis for determining areas that need attention or to ensure users of the capabilities and limitations of the SoS. Even when there is a clear understanding of SoS objectives and metrics, testing in a traditional sense can be difficult. Depending on the SoS context, there may not be funding or authority for SoS testing. Often, the development cycles of the constituent systems are tied to the needs of their owners and original ongoing user base. With multiple constituent systems subject to asynchronous development cycles, finding ways to conduct traditional end-to-end testing across the SoS can be difficult if not impossible. In addition, many SoS are large and diverse, making traditional full end-to-end testing with every change in a constituent system prohibitively costly. Often, the only way to get a good measure of SoS performance is from data collected from actual operations or through estimates based on modeling, simulation, and analysis. Nonetheless, the SoS SE team needs to enable continuity of operation and performance of the SoS despite these challenges.
- *SoS principles*—SoS is a an area where there has been limited attention given to ways to extend systems thinking to the issues particular to SoS. The community is beginning to identify and articulate the crosscutting principles that apply to SoS in general and to develop working examples of the application of these principles. There is a major learning curve for the average SE practitioner moving to an SoS environment and a problem with SoS knowledge transfer within or across organizations.

Beyond these general SE challenges, in today's environment, SoS pose particular issues from a security perspective. This is because constituent system interface relationships are rearranged and augmented asynchronously and often involve COTS elements from a wide variety of sources. Security vulnerabilities may arise as emergent phenomena from the overall SoS configuration even when individual constituent systems are sufficiently secure in isolation.

The SoS challenges cited in this section require SE approaches that combine both the systematic and procedural aspects described in this handbook with holistic, nonlinear, iterative methods. There is a growing set of approaches to applying SE to SoS (Cook and Unewisse, 2019). These include SoS life cycle engineering approaches such as the SoS Wave Model (Dahmann, et al., 2011) and the Designing for Adaptability and evolutioN in System of Systems Engineering (DANSE). These approaches address both functionality of constituents to create coherent aggregate SoS capability (Axelsson, 2020) as well as management of interfaces among constituents (Hoehne, 2020).

4.3.7 Internet of Things (IoT)/Big Data-Driven Systems

SE is based on engineering requirements, engineering calculations, testing, modeling, and simulations—and all are based on data or data generation. SE practitioners often make decisions based on intuition, previous experience, or qualitative assessments. The 4th Industrial Revolution, with its proliferation of sensors of various types and big data analytics, creates an opportunity for SE, as a discipline, and for SE practitioners, as professionals and decisions makers, to be more data-driven. The following recommendations apply to modern SE tasks and decisions:

- Bring as much diverse data and as many diverse viewpoints to maximize the generation of information quality.
- Use data to develop a deeper understanding of the business context and the problem at hand.
- Develop an appreciation for the impact of variation, both in data and in the overall business.

- Deal with uncertainty, which means that SE also recognizes mistakes.
- Recognize the importance of high-quality data and invest in trusted sources and in making improvements.
- Conduct good experiments and research to supplement existing data and address new questions.
- Recognize the criteria used to make decisions and adapt under varying circumstances.
- Realize that making a decision is only the first step; SE practitioners must keep an open mind and revise decisions if new data suggests a better course of action.
- Work to bring new data and new data technologies into the organization.
- Learn from mistakes and help others to do so, by applying lessons-learned processes.
- As SE practitioners, strive to be a role model when it comes to data, working with leaders, peers, and subordinates to help them become data driven.

There are three general goals in analyzing data:

1. *Prediction*: To predict the response to future values of the input variables.
2. *Estimation*: To infer how response variables are associated with input variables.
3. *Explanation*: To understand the relative contribution of input variables to response values.

Predictive modeling is the process of applying models and algorithms to data for the purpose of predicting new observations. In contrast, explanatory models aim to explain the causality and relationship between the independent variables and the dependent variables. Classical statistics focuses on modeling the stochastic system generating the data. Statistical learning, or computer age statistics, builds on big data and the modeling of the data itself. If the former aimed at properties of the model, the latter is looking at the properties of computational algorithms. SE practitioners need to be educated in data sciences to enable them to practice the above tools and methods as an integral part of SE.

Data analytics and IoT are wide-scope revolutions of digital surroundings. They create complex CPS that add new functionalities and capabilities to the existing physical environment. Designing an IoT system that has analytic capabilities involves "multi stack" layers, addressing SoS and network of networks.

SE practitioners should view a system as interconnected system elements performing the system functions. To meet this challenge, the SE practitioner who leads data-driven designs needs interdisciplinary knowledge of the main aspects of IoT: computing, sensors/actuators, software, network, analytics and data science.

4.3.8 Service Systems

OASIS (2012) defines a service as:

> A mechanism to enable access to one or more capabilities, where the access is provided using a prescribed interface and is exercised consistently with constraints and policies as specified by the service description.

It involves application of specialized competences (knowledge and skills) through deeds, processes, and performances for the benefit of another entity or the entity itself in real world. The entity involved with the service can be technical, socio-technical, or strictly social.

For service systems, understanding the integration needs among loosely coupled systems and system elements, along with the information flows required for both governance and operations, administration, maintenance, and provisioning of the service, presents major challenges in the definition, design, and implementation of services (Domingue, et al., 2009; Maier, 1998). Cloutier, et al. (2009) presented the importance of Network-Centric Systems (NCS) for dynamically binding different system entities in engineered systems rapidly to realize adaptive SoSs that, in the case

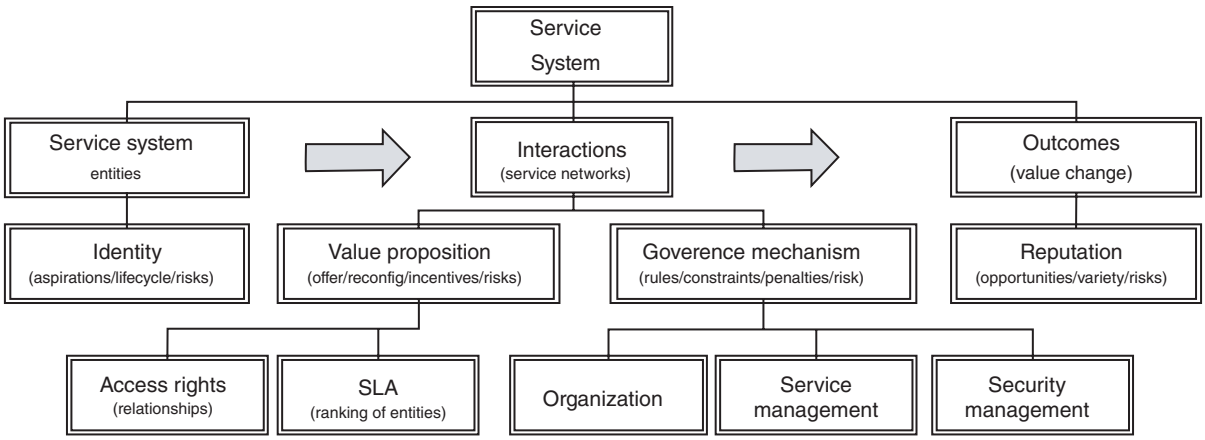


FIGURE 4.9 Service system conceptual framework. From Spohrer (2011). Used with permission. All other rights reserved.

of service systems, are capable of knowledge emergence and real-time behavior emergence for service discovery and delivery.

Figure 4.9 illustrates the conceptual framework of a service system. Typically, a service system is composed of service system entities that interact through processes defined by governance and management rules to create different types of outcomes in the context of stakeholders with the purpose of providing improved customer interaction and value cocreation.

Services not only involve the interaction between the service provider and the consumer to produce value, but have other intangible attributes like quality of service (e.g., ambulance service availability, response time to an emergency request). The demand for service may have loads dependent on time of day, day of week, season, or unexpected needs (e.g., natural disasters), and services are rendered at the time they are requested. Thus, the design and operations of service systems "is all about finding the appropriate balance between the resources devoted to the systems and the demands placed on the system, so that the quality of service to the customer is as good as possible" (Daskin, 2010).

In many cases, taking a service SE approach is imperative for the service-oriented, customer-centric, holistic view to select and combine service system entities to define and discover relationships among service system entities to plan, design, adapt, or self-adapt to cocreate value. Typically, five types of resources need to be considered: people; tangible products and environment infrastructure; organizations and institutions; protocols; and shared information and symbolic knowledge in the service delivery process. Major challenges include the dynamic nature of service systems evolving and adapting to constantly changing operations and/or business environments and the need to overcome silos of knowledge. Interoperability of service system entities through interface agreements must be at the forefront of the service SE design process for the harmonization of operations, administration, maintenance, and provisioning procedures of the individual service system entities (Pineda, 2010). In addition, service systems require open collaboration among all stakeholders, but recent research on mental models of multidisciplinary teams shows integration and collaboration into cohesive teams has proven to be a major challenge (Carpenter, et al., 2010).

In summary, in a service system environment, SE practitioners should bring a customer focus to promote service excellence and to facilitate service innovation through the use of emerging technologies to propose creation of new service systems and value cocreation. SE practitioners must play the role of an integrator, considering the interface requirements for the interoperability of service system entities—not only for technical integration but also for the processes and organization required for optimal customer experience during service operations.

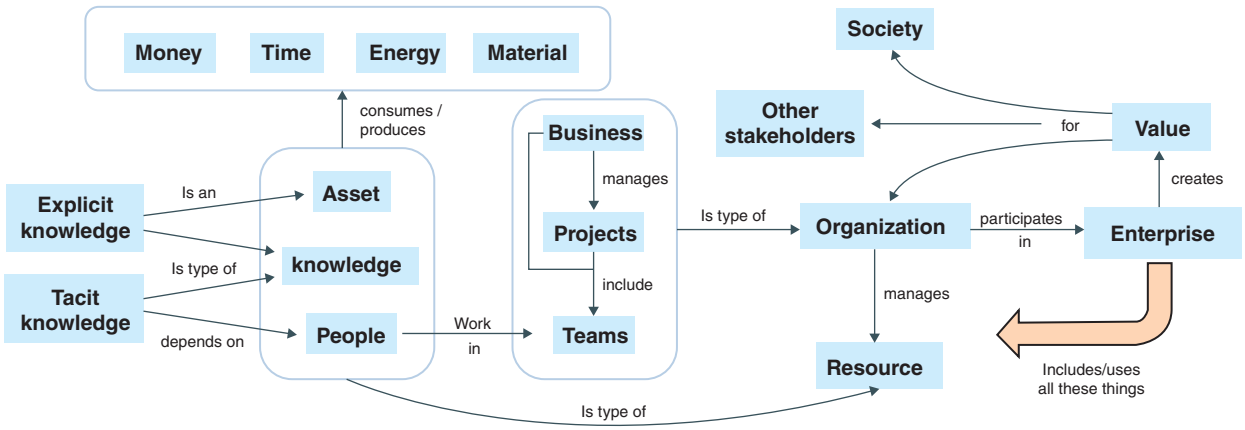
4.3.9 Enterprise Systems

This section illustrates the applications of SE principles and concepts when the SoI is an enterprise. The aim is to continuously improve and help transform the enterprise to better deliver value and to survive in a globally competitive environment. Enterprise SE is an emerging discipline that focuses on frameworks, tools, and problem-solving approaches for dealing with the inherent complexities of the enterprise including exploitation of new opportunities that can facilitate achievement of enterprise goals. A good overall description of enterprise SE is provided in Rebovich and White (2011). For more detailed information on this topic, please see the Enterprise SE articles in Part 4 of SEBoK (2023).

Enterprise An enterprise consists of a purposeful combination (e.g., a network) of interdependent resources (e.g., people, processes, organizations, supporting technologies, and funding) that interact with each other to coordinate functions, share information, allocate funding, create workflows, and make decisions, and that interact with their environment(s) to achieve business and operational goals through a complex web of interactions distributed across geography and time (Rebovich and White, 2011).

An enterprise must do two things: (1) develop things within the enterprise to serve as either external offerings or as internal mechanisms to enable achievement of enterprise operations, and (2) transform the enterprise itself so that it can more effectively and efficiently perform its operations and survive in its competitive and constrained environment.

It is worth noting that an enterprise is not equivalent to an "organization." As shown in Figure 4.10, an enterprise has organizations that participate in it, but these organizations are not necessarily "part" of the enterprise. The organizations that participate in the enterprise will manage a variety of resources for the benefit of the enterprise, such as people, knowledge, and other assets such as processes, principles, policies, practices, culture, doctrine, theories, beliefs, facilities, land, and intellectual property. These organizational resources will consume or produce money, time, energy, and material when acting on behalf of the enterprise.



Notes:

1. All entities shown are decomposable, except people. For example, a business can have sub-businesses, a project can have subprojects, a resource can have sub-resources, an enterprise can have sub-enterprises.
2. All entities have other names. For example, a program can be a project comprising several subprojects (often called merely projects). Business can be an agency, team can be group, value can be utility, etc.
3. There is no attempt to be prescriptive in the names chosen for this diagram. The main goal of this is to show how this chapter uses these terms and how they are related to each other in a conceptual manner.

FIGURE 4.10 Organizations manage resources to create enterprise value. From SEBoK (2023). Used with permission. All other rights reserved.

Creating Value As shown in Figure 4.10, an enterprise creates value for society, for other stakeholders, and for the organizations that participate in that enterprise. It also shows other key elements that contribute to the value creation process. There are many types of organizations to implement value-creating enterprises: businesses (companies), networks of companies, programs and projects, virtual organizations, etc. A typical business may participate in multiple enterprises through its portfolio of projects. A large SE project can be an enterprise in its own right (implemented as a virtual organization), with participation by many different businesses, and may be organized as a number of interrelated subprojects. In many cases, enterprises find themselves in a rapidly changing environment where stakeholder needs change over time. Therefore, an enterprise must constantly adapt its capabilities to meet the enterprise strategic goals and objectives.

Capabilities in the Enterprise As shown in Figure 4.11, the enterprise acquires or develops systems or individual elements of a system. The enterprise can also create, supply, use, and operate systems or system elements. Since there could possibly be several organizations involved in this enterprise venture, each organization could be responsible for particular systems or perhaps for certain kinds of elements. Each organization brings their own organizational capability with them, and the unique combination of these organizations leads to the overall operational capability of the whole enterprise.

The word "capability" is used in SE in the sense of "the ability to do something useful under a particular set of conditions." This section discusses three different kinds of capabilities: organizational capability, system capability, and operational capability. It uses the word "competence" to refer to the ability of people relative to the SE task. Individual competence (sometimes called "competency") contributes to, but is not the sole determinant of, organizational capability. This competence is translated to organizational capabilities through the work practices that are adopted by the organizations. New systems (with new or enhanced system capabilities) are developed to enhance enterprise operational capability in response to stakeholder's concerns about a problem situation.

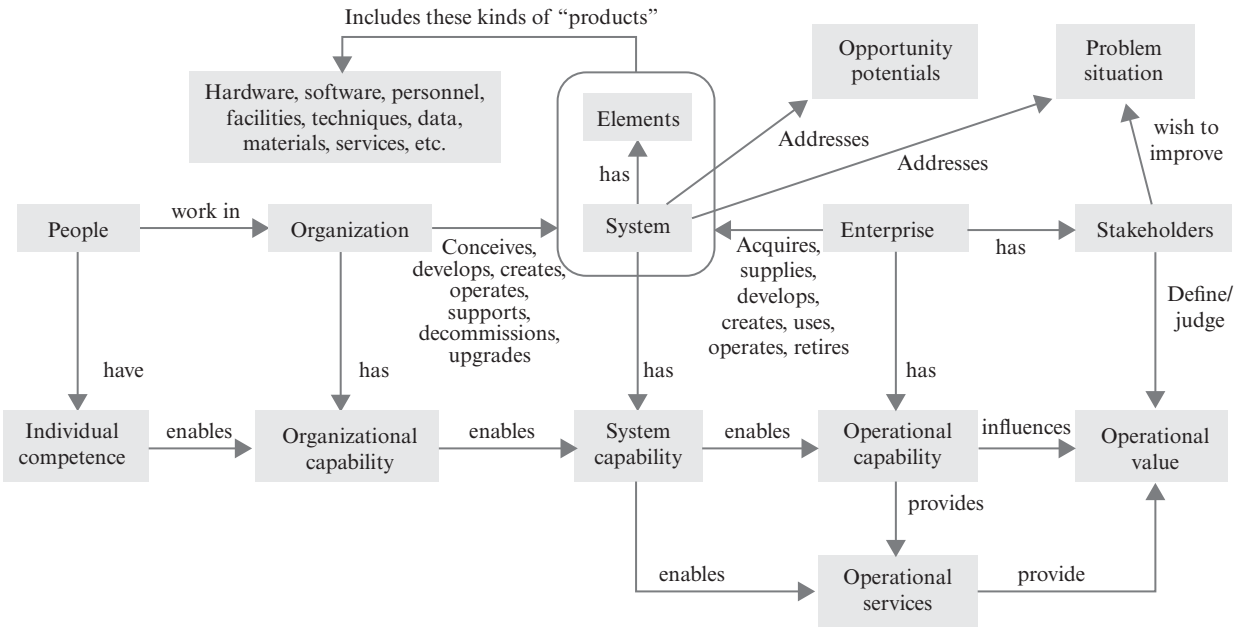


FIGURE 4.11 Individual competence leads to organizational, system, and operational capability. From SEBoK (2023). Used with permission. All other rights reserved.

As also shown in Figure 4.11, operational capabilities provide operational services that are enabled by system capabilities. These system capabilities are inherent in the system that is conceived, developed, created, and/or operated by an enterprise. Enterprise SE concentrates its efforts on maximizing operational value for various stakeholders, some of whom may be interested in the improvement of some problem situation.

Enterprise SE, however, addresses more than just solving problems; it also deals with the exploitation of opportunities for better ways to achieve the enterprise goals. These opportunities might involve lowering operating costs, increasing market share, decreasing deployment risk, reducing time to market, and any number of other enterprise goals. The importance of addressing opportunity potentials should not be underestimated in the execution of enterprise SE practices.

The operational capabilities of an enterprise will have a contribution to operational value (as perceived by the stakeholders). Notice that the organization or enterprise can deal with either the system as a whole or with only one (or a few) of its elements. These elements are not necessarily hard items, like hardware and software, but can also include "soft" items, like people, processes, principles, policies, practices, organizations, doctrines, theories, beliefs, and so on.

Enterprise Drivers and Outcomes An enterprise needs to consider its own needs that relate to enabling assets (e.g., personnel, facilities, communication networks, computing facilities, policies and practices, tools and methods, funding and partnerships, equipment and supplies) when addressing the stakeholders' needs. The purpose of the enterprise's enabling assets is to effect state changes to relevant elements of the enterprise necessary to achieve targeted levels of performance. The enterprise "state" shown in Figure 4.12 is a complex web of past, current, and future states (Rouse, 2009). The enterprise work processes use these enabling assets to accomplish their work objectives to achieve the desired future states.

Since a high degree of complexity is to be assumed, it is advisable to apply formalized modeling methods to achieve the enterprise strategic goals and objectives. It has proven useful to use enterprise architecture analysis to model these states and the relative impact each enabling asset has on the desired state changes. This analysis can be used to determine how best to fill capability gaps and minimize the excess capabilities (or "capacities"). The needs and capacities are used to determine where in the architecture elements need to be added, dropped, or changed. Each modification represents a potential benefit to various stakeholders, along with associated costs and risks for introducing that modification.

Enterprise Opportunities and Opportunity Assessments The potential modifications that are identified represent opportunities for improvement. Usually, these opportunities require the investment of time, money, facilities, personnel, and so on. There might also be opportunities for "divestment," which could involve selling of assets, reducing capacity, canceling projects, and so on. Each opportunity can be assessed on its own merits, but usually these opportunities have dependencies and interfaces with other opportunities, with the current activities and operations of the enterprise, and with the enterprise's partners. Therefore, the opportunities may need to be assessed as a "portfolio," or, at least, as sets of related opportunities. Typically, a business case assessment is required for each opportunity or set of opportunities. If the set of opportunities is large or has complicated relationships, it may be necessary to employ portfolio management techniques. The portfolio elements could be bids, projects, products, services, technologies, intellectual property, etc., or

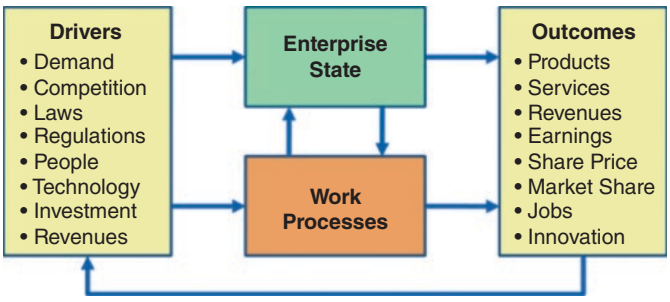


FIGURE 4.12 Enterprise state changes through work process activities. From Rouse (2009). Used with permission. All other rights reserved.

any combination of these items. Examples of an enterprise portfolio captured in an architecture modeling tool can be found in Martin (2005), Martin et al. (2004), and Martin (2003).

The results of the opportunity assessment can be compiled and laid out in an enterprise plan (sometimes conveyed as an enterprise ConOps) that considers all relevant factors, including system capabilities, organizational capabilities, funding constraints, legal commitments and obligations, partner arrangements, intellectual property ownership, personnel development and retention, and so on. The plan usually goes out to some long horizon, typically more than a decade, depending on the nature of the enterprise's business environment, technology volatility, market intensity, and so on. The enterprise plan needs to be in alignment with the enterprise's strategic goals and objectives and leadership priorities.

Practical Considerations When it comes to performing SE at the enterprise level, Rebovich and White (2011) provide several good practices:

- Set enterprise fitness as the key measure of system success. Leverage game theory and ecology, along with the practices of satisfying and governing the commons.
- Deal with uncertainty and conflict in the enterprise through adaptation: variety, selection, exploration, and experimentation.
- Leverage the practice of layered architectures with loose couplers and the theory of order and chaos in networks.

Enterprise governance involves shaping the political, operational, economic, and technical landscape. One should not try to control the enterprise like one would in a traditional SE effort at the project level.

4.4 APPLICATION OF SYSTEMS ENGINEERING FOR SPECIFIC PRODUCT SECTOR OR DOMAIN APPLICATION

This chapter presents how SE is applied in different product sectors or application domains. For each of these, unique and domain-specific terms, concepts, activities, methods, and practices are introduced.

The following domains are presented in alphabetical order:

- Automotive Systems;
- Biomedical and Healthcare Systems;
- Commercial Aerospace Systems;
- Defense Systems;
- Infrastructure Systems;
- Oil & Gas Systems;
- Power & Energy Systems;
- Space Systems;
- Telecommunication Systems;
- Transportation Systems.

Note that the application of SE is not limited to the product sectors and application domains listed above. SE is a generic discipline that can be applied in most situations and domains (with varying levels of value). However, the details of how the practice is applied will vary in different product sectors or application domains.

4.4.1 Automotive Systems

Overview of SE Applications within the Automotive Domain The automotive industry has a long history of engineering complicated and more and more complex consumer products, characterized by diverse product ranges, high production volumes, and a very competitive market. To make vehicles attractive, manufacturers have to balance efficiency for mass production with driving performance. Strong, well-orchestrated processes across the supply chain are key to meeting this challenge. Quality is highly dependent on good processes being followed rigorously, and economic performance relies on optimization. Electrification, connectivity, autonomy, and consumer choice are driving complexity, creating opportunities for SE to enter the mainstream in coming years. The INCOSE Automotive SE Vision 2025 document (2020) provides an excellent summary of the current and future trends in this domain. See Table 4.5 for a comparison of automotive with two other domains considering some of the characteristics that may affect SE approaches.

Emergence of SE in the Automotive Domain Since General Motors vehicles first shared a common chassis in 1908, the automotive industry has employed extensive reuse in combination with variant management techniques to manage costs and keep delivery cycles short. Until the late 1960s, activities familiar to the SE domain centered on parts development, promoting reuse through standards and matrix organizations. Architects mainly addressed geometry until the on-board electronics and software revolution started in the 1970s. Standalone mechatronic and automated control systems appeared first, delivering applications such as engine control, anti-lock braking, and automatic heating and air conditioning. During the 1990s, vehicles acquired extensive networks of interconnected electronic control units. In the same period, many manufacturers invested heavily in engineering teams that focused on elicitation and refinement of stakeholder needs to address emergent properties such as safety, environmental impact, dynamic performance, and occupant comfort. This subset of activities (e.g., stakeholder requirements, electrical and electronic architecture) provided the ingredients for the emergence of SE in automotive.

Contemporary SE in the Automotive Domain Increasing complexity, driven by the parallel trends of electrification, on- and off-board networks, automation, and autonomy has led to growing interest in automotive SE since 2000. Electronics and software began to outstrip mechanical design as a means for manufacturers to provide distinctive products. As shown in Table 4.6, the automotive industry is very standards-driven and the last decade has seen many new developments. These standards address, in particular, architectural approaches, software elements, safety, and security processes. Practices have evolved in isolated pockets, leading to significant variation in how the discipline is interpreted and executed. Convergence of capabilities at different maturity levels, led by engineering executives with different worldviews and backgrounds, has led to the adoption of many different SE paradigms with varying enthusiasm. SE practitioners might draw the system boundary at the vehicle level, or be restricted to applying their techniques to individual features, system elements, or domains, sometimes focusing on electronics and software. Approaches differ too, with some being requirements-led, others architecture-centric. SE has entered the automotive domain in incremental steps, and continues to do so, due to the need to accommodate the special features and legacy of each company. Disruptive change is hard for established players because their business models have low margins, and rely on trusted, repeatable processes that leverage extensive reuse of system elements. Similarly, production needs efficient processes to continuously deliver high volumes of system elements and assemblies on a just-in-time basis, making interruptions to manufacturing difficult to tolerate.

The explosion of electrical, electronic, and software systems is driving awareness that engineering based on assembling parts from suppliers without a systems approach is no longer enough. Full autonomy is on the horizon, and advanced driver assistance features like lane keeping and emergency braking are well established. Many new vehicles have high-integrity system elements linked to the outside world. Managing the complexity and cybersecurity risks this creates is a great challenge (see Section 6.5). Cultural and methodological changes are ongoing in industry incumbents. They increasingly face competition from newcomers with backgrounds in software-intensive industries (see Section 4.3.4), who in turn must adapt their culture and scale their business models to the realities of high-volume automotive manufacturing.

TABLE 4.5 Comparison of automotive, aerospace/defense, and consumer electronics domains

	Automotive	Aerospace/ Defense	Consumer Electronics
Customer requirements	Assumed by manufacturer	Defined by customer	Assumed by manufacturer
Legislative environment	Certification/inspection of product design, auditing of development and production processes (e.g., ISO/TS 16949 (2009)/IATF 16949, ISO 26262 (2018), E-marking, conformity of production, many regulatory Inc) standards (UNR, FMRSS, etc.) applicable to the overall system and its elements. Process regulations/standards are a new development.	Certification (e.g., DO-178C from RTCA Inc) in process, tooling and product. Auditing of development and production processes	CE/UL/FCC marking, according to a small number of standards (typically <10 per product)
User skill level	Somewhat trained	Highly trained	Untrained
Complexity (PLE/ component reuse)	High	Low	Medium
Complexity (sociodynamic)	High	High	Low
Product development cycle time	Medium (3–7 years)	Long (10+ years)	Short (1–2 years)
Delivery cadence	Annual	Decade	Annual
Product design life	Medium (10–20 years)	Long (30+ years)	Short (1–2 years)
Approach to maintenance	Repair	Repair	Replacement
Connectivity need	Medium (trend to high)	Low-medium (defined in stakeholder requirements)	Medium-high
Number of external integration interfaces	Medium (trend to high)	Medium (trend to high)	Medium
Safety/ cybersecurity criticality	High	High	Medium-high
Typical industry operating margin	4–6%	6–8%	8–10%
Annual production volume	10k–900k	100–100k	100k–100M

New Eco-Systems Involving the Automotive Industry: The Example of "Mobility as a Service" In many urban areas, local governments implement policies to reduce the number of private cars and to foster the deployment of mobility services as a complement to public transport. The number of privately owned, individual cars has decreased dramatically in some big cities. For instance, in Paris, capitol of France, the percentage of cars as a mode of transportation has decreased from 46% in 2002, to only 13% in 2022. Vehicle manufacturers should not expect that this trend

TABLE 4.6 Representative organizations and standards in the automotive industry

Organization/standard	Description
SAE International, formerly the Society of Automotive Engineers	One of the main organizations that coordinate the development of technical standards for the automotive industry. Currently, SAE International is a globally active professional association and standards organization for engineering professionals in various industries, whose principal emphasis is placed on transport industries such as automotive, aerospace, and commercial vehicles
Japan Society of Automotive Engineers (JSAE)	An organization that sets automotive standards in Japan, analogous to the SAE
Association for Standardization of Automation and Measuring	An incorporated association under German law whose members are primarily international car manufacturers, suppliers, and engineering service providers from the automotive industry. The ASAM standards define protocols, data models, file formats, and application programming interfaces (APIs) for the use in the development and testing of automotive electronic control units
AUTomotive Open System ARchitecture (AUTOSAR)	An open and standardized automotive software architecture, jointly developed by automobile manufacturers, suppliers, and tool developers. Some of its key goals include the standardization of basic system functions, scalability to different vehicle and platform variants, transferability throughout the network, integration from multiple suppliers, maintainability
The GENIVI Alliance	A nonprofit consortium whose goal is to establish a globally competitive, Linux-based operating system, middleware, and platform for the automotive in-vehicle infotainment (IVI) industry. GENIVI specifications cover the entire product life cycle and software updates and upgrades over the vehicle's lifetime
ISO/TS 16949 (2009)/ IATF 16949	An international standard for particular requirements for the application of ISO 9001 quality management systems for automotive production and relevant service part organizations
IEC 62196 (2022)	An international standard for set of electrical connectors and charging modes for electric vehicles maintained by the International Electrotechnical Commission (IEC)
ISO 26262 (2018)	Road vehicles - Functional safety
ISO/SAE 21434 (2021)	Road vehicles - Cybersecurity engineering

will change. Vehicles they used to sell are more and more being replaced by public transport, biking, and walking, as well as services still involving vehicles like car-sharing or ridesharing. All these service offers can be integrated into a larger service enabling them to combine and thus making on-demand mobility faster and easier. This is called Mobility as a Service (MaaS), with a lot of initiatives around the world triggered by the Sustainable Development Goals introduced in Section 3.1.10. However, MaaS is not a silver bullet nor standard yet. MaaS may be considered as the mission for an SoS involving mobility service operators, both public and private. They have to cooperate in order to offer a more attractive user experience and at the same time to make the conditions for local mobility more sustainable.

The Future of SE in the Automotive Domain Dealing with massively expanding complexity in an extremely challenging environment where standards and regulation trail fast-paced innovation is a challenge for this process-driven industry, but an opportunity for SE. As connectivity and autonomy become the norm, vehicles are built on highly configurable software platforms for providing mobility as a service. Development cycles that took five years are being compressed, where service updates that took a year will be expected in weeks. Delivering change like this means fundamental shifts in thinking are required across the board: from new business models, through service-centric architectures, to security-informed safety paradigms. SE is the means by which this can be achieved.

4.4.2 Biomedical and Healthcare Systems

Overview of SE Applications within the Biomedical and Healthcare Domain SE has become more important to the healthcare industry (SEBoK, 2023), especially as systems and processes get more complex and quality characteristics such as safety, security, reliability, and human systems integration become more challenging. SE offers numerous benefits to biomedical and healthcare systems including the following:

- Supports design and development of healthcare systems using well-defined processes and standards,
- Offers well-defined approaches to design and implement architectures for proper interfacing, networking and communications using open industry standards,
- Enables operators and enterprises to scale up without compromising quality of operations,
- Enables better insights and control of many production systems including quality assurance, inventory, and cost control, and
- Augments user experience of various stakeholders like doctors, and surgeons by system level integration of emerging digital platforms like augmented reality, virtual reality, and robotics.

In the medical industry, especially for medical devices, it is important to understand that "risk management" is generally centered around product (user safety) risk and (called system safety in this handbook—see Section 3.1.11) rather than project (technical or business) risk (called risk management in this handbook—see Section 2.3.4.4).

Unique Considerations for Healthcare Delivery SE applied to healthcare delivery differs significantly from conventional SE as applied in traditional fields such as defense, aerospace, and automotive. Most healthcare delivery projects involve improvement of an imperfect workflow or care process or the design of a limited scope new workflow or care process in a local clinic, hospital, laboratory, or in population health. If successful, solutions are shared with peer institutions in the same medical organization. As a result, most SE projects in healthcare delivery involve only a few stakeholders and a handful of requirements. Approaches leveraging lean SE have shown to be successful in many cases (Oppenheim, 2021) (see Section 4.2.3). Healthcare delivery operations have a critical need for the SE process to address pervasive healthcare problems such as care fragmentation (e.g., the systemic misalignment of incentives) or lack of coordination that spawn inefficient allocation of resources or harm to patients. Just as in medical device development, SE in healthcare delivery also strongly emphasizes patient safety. Methods such as the Systems Engineering Intervention for Patient Safety (SEIPS) (Carayon, 2006) focus on tailoring SE processes to the specific context of patient-centered medicine.

Unique Considerations for Medical Devices In contrast to healthcare delivery systems, some medical device and healthcare IT companies use a more traditional form of SE. However, some are heavily tailoring SE approaches to incrementally demonstrate the value of SE. Many devices must work in harsh environments, including inside the human body. Interoperability, interconnectivity, and transportability are increasingly critical for medical devices and SaMDs (Software as Medical Devices). During audits and submissions, regulators require device developers to follow standard quality system processes (e.g., ISO 13485). Standards such as ISO 14971 (application of risk management to medical devices), IEC 60601 (medical device safety), IEC 62304 (Medical Device software—Software life cycle processes), and IEC 62366 (application of usability engineering to medical devices) are driving medical device organizations to take a deeper look into system safety and the engineering practices behind it. Thus, SE practitioners are increasingly being brought on board to leverage their life cycle management skills and support validating that the final product does indeed meet the needs of its stakeholders. In addition to an emphasis on systems safety, the medical device sector is seeing an increasing need for several SE methodologies including, but not limited to, SoS management, stakeholder management, agile systems development, trade analysis, MBSE, and PLE.

Unique Activities, Methods, and Practices Healthcare Systems are often broad in context including a population of diverse patients, many healthcare professionals, many medical devices, many insurance companies, many delivery systems, regulators, and the government. One emphasis of SE practitioners in the biomedical and healthcare domain is patient safety risk, often more so than technical or business risks (see Section 6.1). Traceability is often a key factor in regulatory submissions and audits. Organizations that have strong SE practices are therefore in a better position to avoid pitfalls and to effectively defend their decisions if a regulatory audit does occur. In general, applicable standards do not need to be excessively tailored, although organizations with new or maturing practices may want to focus on lean implementations to obtain early and effective system adoption. Carefully balancing the trades between healthcare costs, better health outcomes for populations, and profits for shareholders is an ongoing challenge for Healthcare SE practitioners. On a larger scale, healthcare SE practitioners that can influence policy and incentives will become even more valuable to their organizations.

4.4.3 Commercial Aerospace Systems

Overview of SE Applications within the Commercial Aerospace Domain SE is part of the strategies for the development of solutions and products in the commercial aerospace system domain. Commercial aerospace systems are complex, and their complexity continues to increase. The increased use of software makes it possible to implement more functions than before, which contributes to a further increase in complexity. At the same time, the expectation is raised that the increased use of software will make solutions and products available more quickly than the historic mechanical systems of the past. As shown in Figure 4.8, commercial aerospace systems are often part of larger SoSs. Future commercial aerospace systems will include autonomy, artificial intelligence, neural networks, novel propulsion, advanced human system integration (HSI), and cybersecurity.

Commercial aerospace systems use sequential as well as incremental and evolutionary life cycle models, including agile methods with smaller cycles. Thus, the processes in this handbook can be used to address and help organizations manage these new factors derived from complexity settings. The adoption of new technologies and perspectives emphasize some concepts such as the systemic approaches and use of SoS approaches to support organizations by putting them on the forefront of the market with competitive products, adapted to the new reality of increasing interoperability.

Unique Terms & Concepts The commercial aerospace organizations of many countries have specific policies, standards, and guidebooks to guide the application of SE in their organizational environment. For example, ARP 4754A (2010) describes the standard practices for verifying commercial aircraft requirements.

There are many other systems related to this domain. For example, in the aviation domain, there are systems that go far beyond the aircraft itself according to the interaction characteristic of system elements described on system concept definition. Examples include an air traffic control system.

New applications of commercial aerospace systems are being continually introduced. For example, some organizations specifically created to address the new aerospace segment of flying cars have started a great race in a totally different way. By using new methodologies and approaches, these systems are being developed by considering their integration in completely new context. The same is happening with unmanned and autonomous vehicles. These new applications require an understanding of the ecosystem of the new operational contexts, as well as the lifestyles of its users.

Unique Activities, Methods, and Practices SE may help the realization of effective commercial aerospace systems through the following activities, methods, and practices:

- **Stakeholders.** Stakeholders vary greatly and can range from federal government services, to aircraft manufacturers, to passengers.
- **Design and construction practices.** Model-based design is generally used from construction model specifications, which enables and maintains traceability between requirements and models.
- **Interfaces.** Because commercial aerospace systems' system elements are developed in various parts of the world and brought to a single (or multiple) location for assembly, adherence to interface management principles is critical.
- **Risk management.** Risk management is essential, especially for the introduction of new technologies.
- **Safety.** Finally, it is important for SE management to assure that safety is not compromised by organizational factors, as described by Paté-Cornell (1990).

Examples of how SE helps is resolving unique domain challenges include:

For aircraft original equipment manufacturers (OEMs), SE:

- Helps in design and manufacturing of aircraft subsystems, assembly and integration testing using well-defined process, standards, and quality standards.
- Offers well-defined approaches to create designs or architectures, processes, and roadmaps for proper interfaces, instrumentation, and communications that enable better visibility of the static and dynamic operational data and status of the subsystems.
- Enables operators and enterprises to scale up without compromising quality of production using a well-defined SE framework, tools, and emerging technologies.
- Enables better insights and control of congestion and traffic control of many schedules like flights, passenger, luggage, and food.
- Augments user experience of various stakeholders by system level integration of emerging digital technologies like augmented reality and virtual reality for enriched cockpit and instruments.

For airlines, SE:

- Helps in the support stage, to maintain the fleet.
- Helps balance performance and environmental impacts.
- Offers a set of procedures and activities to manage the services that consider human resources, information, and operation data.

Other Unique Considerations SE is increasingly being applied in commercial practice. Petersen and Sutcliffe (1992), for example, discuss the principles of SE as applied to aircraft development. Life cycle functions of the commercial aerospace industry gives SE its own unique characteristics.

4.4.4 Defense Systems

Overview of SE Applications within the Defense Domain While SE has been practiced in some form from antiquity, what has now become known as the modern definition of SE has its roots in defense systems of the twentieth century. It became recognized as a distinct activity in the late 1950s and early 1960s due to technological advances taking place that led to increasing levels of system complexity and systems integration challenges, and the need for SE further increased with the large-scale introduction of digital computers and software.

SE within defense evolved to address systemic approaches to issues such as the widespread adaptation of COTS technologies and the use of SoS approaches. It offers well-defined designs/architecture, processes and roadmaps for proper interfacing, networking, and communications. This enables better integrity and interoperability of real-time intelligence data across various devices, from various vendors, and platforms using open industry standards. Today, with increasing emphasis on networks and capabilities the defense organizations of many countries are recognizing the criticality of end-to-end SoS performance and increasing focus on integration to deliver these capabilities.

Unique Considerations Defense systems have numerous characteristics and consequently, a huge complexity, making SE essential for their development:

- They are complex technical systems with many stakeholders and compressed development timelines.
- The systems must be highly available and work in extreme conditions all over the world—from deserts to rain forests and to arctic outposts.
- There are long system life cycles, so logistics is of prime importance.
- There is typically a strong human interaction, so usability/human systems integration is critical for successful operations.
- There is at times a need for defense operators and enterprises to accelerate development and production (e.g., quick response in event of national emergency or increased threats) without compromising quality of operations using a well-defined SE framework, tools, and emerging technologies

Unique Activities, Methods, and Practices SE has a strong heritage in defense, and much of the SE processes in this handbook can be used as is in a straightforward manner, with normal project tailoring to address unique aspects of the project. It is important to note that as ISO/IEC/IEEE 15288 (2023) has evolved into a more domain- and country-neutral SE standard, so care must be taken to ensure that the defense focus is reasserted upon application. An example of specific implementation of ISO/IEC/IEEE 15288 when utilized for US Department of Defense projects is provided in IEEE 15288.1 (2014). This standard provides the basis for selection, negotiation, agreement, and performance of necessary SE activities and delivery of products. Additionally, the standard allows flexibility for both innovative implementation and tailoring of the specific SE processes to be used by system suppliers, either contractors or government system developers, integrators, maintainers, or sustainers. The defense organizations of many countries also have specific policies, standards, and guidebooks to guide the application of SE in their environment.

4.4.5 Infrastructure Systems

Overview of SE Applications within the Infrastructure Domain This section addresses physical capital projects infrastructure including public works, transport, complex buildings, and industrial facilities. Within the infrastructure domain, SE practices are more developed in the high-technology system elements that involve software development, control systems, system security, or system safety. Infrastructure projects tend to define the high-level design solution without requirements decomposition, allocation, or interface identification. Architectures, traceability, and relationships within the project are often implied rather than specified. Infrastructure owners can benefit by applying SE to provide systematic, formal, verifiable connections between the business needs and the final product.

Unique Terms and Concepts Infrastructure projects are distinguished from manufacturing and production, as they usually focus on unique, large physical systems where construction takes place on site rather than in a factory. These projects are adapted and integrated to existing environments, and are often characterized by loosely defined boundaries, evolving system architectures, multiphase implementation efforts which can exceed a few decades, and multiple-decade asset life cycles. As a result, stakeholders' expectations and design solutions evolve over an extended

TABLE 4.7 Infrastructure and SE definition correlation

Systems Engineering Term	Infrastructure Term	Recommendation
Acquirer	Owner or Agency	
Acquisition	Contracting phase; Procurement	Share good practices and lessons learned to improve procurement documents to enable better owner control of the project.
Business requirements	Project need; Business case	Derives contractor requirements from the business requirements, hold requirement reviews and include in the contractors' scope.
Configuration control	Versioning	Configuration identification, change management, status accounting, configuration audit according to ISO 10007 (2017).
Decision gate	Milestone	Clearly define entry and exit criteria for decision gates
Life cycle	Project life cycle	Include how the infrastructure will deliver its intended function and long-term asset management. Add in contractor's scope expectations that will benefit the entire project life cycle.
Performance requirement	Often found in Technical Specifications	Allocate top-level system performance requirements to system elements, defining performance requirements. Best performed by the acquiring entity unless the procurement method is a PPP.
Requirements	Design Criteria; Scope of Work; or Specifications	Integrate full life cycle considerations into design criteria, including operations, maintenance, and disposal/replacement planning.
Supplier	Design Consultant; Contractor	Use requirements management to strengthen procurement language and enforce contract requirements during the project. Clearly define acceptance criteria and performance measures.
System architecture	Context diagram; Schematics; Process and Instrumentation Diagrams	Consider creating early in project life cycle to support requirement allocation and interface management. Use ICDs or N^2 diagrams to complement the system architecture.
Verification	Design Review; Quality Control (QC)/Quality Assurance (QA)	Provide sufficient schedule and budget for both QC and Q A activities, including specific audit periods and "pens down" dates for each milestone. In the design phase, confirm the design meets requirements (QC), and procedures were followed (QA)
Validation	Construction Inspection; Quality Control (QC) / Quality Assurance (QA)	Include sufficient budget and authority for QA to ensure compliance. Ensure acceptance testing refers back to stakeholder needs and includes a focus on whether it meets its intended use.

timeframe. Unlike other SE domains, most infrastructure projects cannot be standardized and do not involve a prototype.

Many of the processes described in this handbook can be used to manage infrastructure projects but in some cases with different terminology, as illustrated in Table 4.7. There are some areas where existing infrastructure practices could be adjusted slightly to better align with SE practices.

Unique Activities, Methods, and Practices SE may help the realization of effective infrastructure systems through the following activities, methods, and practices:

- **Stakeholders.** Stakeholders can range from governmental legislators who control funding for the project, to local/regional agencies that add beautification needs, to landowners with adjacent property impacted by a pro-

posed project. In all government-funded projects, segments of the public may also be a stakeholder group. The wide array of potential stakeholders makes requirements gathering, cost, and schedules volatile. Public and political pressure can cause premature initiation of projects, with incomplete project scope and ill-defined metrics.

- **Design and construction practices.** Within infrastructure, the engineering disciplines have well-established, traditional practices and are guided by independent industry codes and standards that are not shared between disciplines. Design requirements are generally dissociated from construction specifications, therefore limiting traceability between design and construction.
- **Interfaces.** Infrastructure projects have external, often uncontrollable interfaces that can impact the project. Interfaces can include existing built systems, natural systems, environmental, and other internal and external dynamics.
- **Risks.** The contractual framework and allocation of liability and commercial risk are major factors impacting procurement and contracting processes. SE practices may therefore help to manage risk associated with cost estimating, changing scope, system integration, and verification. They may also improve construction productivity, making infrastructure development more cost-effective.

Other Unique Considerations SE concepts are relatively newly applied in the infrastructure domain. As the application of SE grows within the infrastructure domain, an effort should be made to train engineering discipline specialists in SE concepts. Four key SE processes are useful to introduce SE on infrastructure projects: requirements management, interface management, verification, and validation. These processes can improve infrastructure project delivery, and total life cycle view that integrates design, construction, and asset management.

4.4.6 Oil and Gas Systems

Overview of SE Applications within the Oil and Gas Domain The emergence of SE within the Oil and Gas (O&G) domain is relatively new compared to other sectors of similar complexity. Most applications of SE have occurred within the past decade to varying levels of implementation. Due to fluctuation in oil prices, new systems with increasing complexity and efforts to reduce greenhouse gas emissions have motivated a risk-averse industry to adapt to, and in some cases drive, change. This has encouraged an entire culture known to resist change to challenge assumptions and traditional ways of working, especially working in a document-centric environment.

The greatest SE-related need has been in the system requirements definition and requirements management space. With a domain-wide focus on digitalization, the change in how requirements are defined and transmitted throughout the supply chain has benefited from an SE approach. The industry leaders have either switched, or are switching, to data-centric requirement sets. There has been collaboration between suppliers and acquirers to improve the quality and traceability of requirements and create metrics for measurement of progress. INCOSE and American Petroleum Institute (API) cooperated on some trials in 2017 and 2018 that explained the aims and elements of good requirement writing to a panel of experts involved in updating a standard. They then supported the engineers in the writing and recrafting of the content, resulting in higher quality requirements and clearly separating between instruction, information, and verification (IOGP, 2021).

Beyond requirements, additional SE practices are also being introduced in the O&G domain. For example, SE practitioners take advantage of requirement definition to develop system architectures and systematically define interfaces. By using requirement management tools, configuration management and change management of requirements can be implemented in projects. In other cases, systems thinking tools, such as context diagrams and functional trees, are used as a foundation for SE practices, such as functional modeling. Technical requirements are also leading to conversations and implementation of verification and validation strategies and realization. With the companies incorporating digital design data across disciplines and throughout the life cycle, the digitalization of requirements has led to the auto-creation of specifications and test plans.

Unique Considerations for SE within Oil and Gas One of the challenges when considering SE in O&G is that it is difficult to evaluate the entire domain as one. The long and complicated supply chain includes diverse and segmented companies across the globe. And it is not solely crude oil or natural gas. With the current energy transition affecting all aspects of engineering, most O&G companies have been shifting focus from fossil-based systems to include renewable sources that are efficient solutions with net-zero emissions. Many oil and gas companies have targets of net-zero greenhouse gas emissions for operations by 2050.

Another challenge for the domain is that since SE has not been implemented as a holistic approach, there are pockets of SE maturity that do not always intersect. This is a result of most O&G companies following a well-established and practiced sequential (waterfall) stage gate process. Therefore, SE implementation is generally only implemented where a clear case for change is needed and demonstrated, or when all other approaches have been exhausted. To help with this, the INCOSE O&G working group developed a scalable presentation for various high-level conversations and presenting success case studies from participating O&G companies. One area where SE continues to gain traction and show value is in new product development projects. By introducing SE principles and methods early in the project, the project team can see the benefits of applying SE and embrace the changes to the traditional ways of working.

4.4.7 Power & Energy Systems

Overview of SE Applications within the Power and Energy Domain During the first two decades of the twenty-first century, the global energy system has been subject to a complex set of requirements stemming from the Paris Agreement, the United Nations Sustainable Development Goals (see Section 3.1.10), reduction in greenhouse gas emissions, and other efforts to avoid degradation of our social foundations and ecological ceiling (Raworth, 2017). To provide an effective solution for such a complex set of problems demands the realization, or modification, of many new systems, elements, and enabling systems supported by a holistic systems approach. The United Kingdom's (UK) Council for Science and Technology stated with respect to the UK's Net Zero ambitions that "by drawing on SE principles, a detailed and credible plan can provide the framework required to drive change, give reassurance to businesses, investors and consumers, and engage the whole of society in delivering this change" (CST, 2020).

At the heart of the sustainability transition is the convergence of business, technology, and socio-politics to guide innovation around what is viable, feasible, and desirable. This new intersection of disciplines can be enabled through SE. Yet many incumbent organizations and legacy approaches dominate the power and energy landscape, meaning a change in thinking or practice is resisted, or even directly opposed. As a result, the application of SE in power and energy remains largely immature in the first quarter of the twenty-first century compared to more established sectors such as defense, space, and transportation. A cultural shift toward shared knowledge management systems, portfolio management, and organization infrastructure is required to fully embrace and adopt SE.

Unique Terms and Concepts The language used in SE is made effective through SE heuristics or when translated to real-world examples in the power and energy context. For example, an SoS may be considered an abstract SE term, yet it perfectly describes the nature of distributed energy resources.

Unique Activities, Methods, and Practices SE may help the realization of effective power and energy systems through the following activities, methods, and practices:

- **Architecture and design.** Provides robust architecture, design, and development processes for higher integrity and interoperability across the supply chain infrastructure from upstream (e.g., solar plant, nuclear, wind farms), mid-stream (e.g., large-scale storage, energy vector processing, transmission and distribution networks) and downstream (e.g., retail outlets, local area networks, domestic microgeneration, private storage).
- **Risks.** Enables better identification and handling of risks associated with energy security and resilience.

- **Portfolio management.** Provides the platform for joined up roadmaps and communication channels which enables stakeholder acceptance, transition, and utilization of emerging paradigms such as smart grids, district heat networks, renewable technologies, demand side response, and electric vehicles.
- **Sustainability.** Enables better management of reductions in greenhouse gas emissions through robust technical and management processes with a whole life cycle perspective, open industry standards, quality management, and assurance.

Other Unique Considerations Power and energy systems are typically at the SoS level, so activities follow the key characteristics and challenges associated with an SoS. As an example, achieving the goal of energy security requires as much understanding of geo-politics as it does the evolution of cybersecurity as digitalization grows.

The application of SE needs to transcend geographic boundaries and domain silos. For example, consideration for how SE supports the implementation of the Clydebank declaration, which calls for the establishment of green shipping corridors for zero-emission maritime transport between shipping ports, presents an energy, transport, and logistics challenge on an international scale.

On a global scale, power and energy systems must remain persistently operational for billions of system users whilst maintaining a constant state of equilibrium with demand balanced by supply (augmented by flexibility solutions such as demand side response and storage). In addition, power and energy systems typically have a life cycle of 30–100 years or even into the thousands of years for end-of-life decommissioning and waste storage from nuclear fission facilities. These considerations form complexity multipliers for the sustainable energy transition.

Adapting the mental models and behaviors of system users will be crucial to effecting change. This demands elements of social sciences, systems science, and systems thinking to complement the rigor of SE processes. To support this, there is a need to provide feedback mechanisms to influence the micro-behavior of the human actors in the system in such a way as to maintain the macro-stability of the system (Sillitto, 2010) achieved through HSI (see Section 3.1.4). But SE cannot focus on change in human behaviors without consideration for market dynamics and political levers. SE can help provide the coherence and joined-up thinking necessary to make energy policy an enabling system for delivering the overarching goals of energy decarbonization, digitalization, decentralization, and democratization. We must also avoid the trap of pushing technology solutions that deliver undesirable user experience. Considerations range from consumer price point, to acoustic noise of technologies, to retrofit disruptions, to the availability of energy in remote or isolated communities. Our challenge, as future ancestors, will be to find ways to support growing energy demands whilst delivering a sustainable energy supply chain that is available and affordable to everyone on a global scale.

4.4.8 Space Systems

Overview of SE Applications within the Space Domain Space systems are systems that are designed to operate and perform tasks into and within the space environment. This may consist of: spacecraft (and their associated payloads and instruments); mission packages(s); ground stations; data links between spacecraft and ground, launch systems; and directly related supporting infrastructure. Due to the relatively high costs of deploying assets into earth orbit or beyond, space systems typically require high reliability with little maintenance other than software changes (note that designing for maintainability in space one of the many trade-offs that need to be considered during conceptual design). This makes it necessary for all system elements to work the first time or be compensated by operational workarounds; this can impact the risk posture of the system being developed.

The space domain has evolved into three main areas of interest, with some overlap:

- Civil,
- Commercial, and
- National Security Space.

Each of these areas have their own motivations that can influence the way they develop systems.

Unique Activities, Methods, and Practices Key emphases of SE in the space domain are integration, verification (including testing), and validation of highly reliable, well-characterized systems. Risk management is also key in determining when to incorporate new technologies and how to react to changing requirements through multiyear developments and programmatic challenges. SE provides coordination for multi-disciplinary engineering expertise that enables optimized designs.

Civil systems are typically acquired by government agencies, which typically focus on performance risk, determining when to incorporate new technologies, and how to react to changing requirements through multiyear developments and programmatic challenges. This lends itself to the use of the sequential approaches, such as the SE Vee model, or, in some cases, the waterfall model.

Commercial systems strive for profitability (cost and schedule) and are more amenable to using incremental and evolutionary approaches. This allows them to deploy systems faster, and rapidly gain experience that can improve later iterations of their product.

National Security Space, much like Civil, may emphasize performance over cost and schedule, and have traditionally used the Vee and waterfall models for development. They typically are more tolerant of accepting risk from the injection of new technologies.

Unique Standards Overall, proper application of SE in the space domain helps in design and development of space elements for easier manufacturing and lowering maintenance cost using well-defined processes and standards. SE offers well-defined architecture and design processes and roadmaps for proper interfacing, networking, and communications that enable better integrity and interoperability of space systems and elements provided by various contractors, and across the supply chain using open industry standards.

Civil, commercial, and national security entities have their own drivers that determine how standards are created and adopted. Most space-faring nations and international consortiums have developed and adopted their own standards that are specific for space systems. Some examples include:

- In the United States, the Department of Defense has created Military (or "Mil") Standards that have been readily adopted by both Civil and Commercial primarily due to the need for high reliability and survivability in a hostile environment. NASA has also created a set of technical standards, as has the AIAA and other organizations.
- The European Cooperation for Space Standardization is an initiative established to develop a coherent, single set of user-friendly standards for use in all European space activities.
- The Euro-Asian Council for Standardization, Metrology and Certification, a regional standards organization operating under the auspices of the Commonwealth of Independent States, has developed GOST (Russian: ГОСТ),a set of spacecraft certification standards that is commonly used by Russia.
- JAXA (Japan Aerospace Exploration Agency) has developed a library of standards known as JERG (JAXA Engineering Requirement, Guideline).
- International organizations such as ISO and IEEE have also been involved in the development of standards that enhance interoperability.

Other Unique Considerations An additional challenge in the space domain occurs when humans are integrated into the system. Typically, it includes the incorporation of design features and capabilities that accommodate human interaction with the system to enhance overall safety and mission success. The system needs to ensure that the human needs are addressed in terms of effectively utilizing human capabilities and performance, hazards are controlled to a level considered safe for human operations, and provide, to the maximum extent practical, the capability to safely recover the crew from hazardous situations. At the time of this writing, only missions led by three nations (Russia, the United States , and China) have sent humans into space. Each country has developed their own set of standards:

- Rovcosmos for Russia, and
- Human Space Flight Requirements for Civil, typically governed by NASA, for the United States,
- CNSA for China.

Of the three, only the United States has begun to explore human commercial space flight, where such requirements are governed by the US Federal Aviation Administration (FAA).

4.4.9 Telecommunication Systems

Overview of SE Applications within the Telecommunication Domain Telecommunication systems are defined by having a route to transfer information across and to distinct endpoints that are used to share (send and/or receive) information. They differ from postal systems in that the information shared is in the form of electronic media (applications or services) rather than transporting packages or handwritten letters.

Telecommunication systems are enablers for other services. Almost all modern systems either make use of telecommunication technologies provided by other systems, or contain telecommunication technologies within them (e.g., digital signage and ticketing systems within public transport systems; battlespace communication systems used by the military; environmental monitoring systems such as those used to monitor/predict the weather or detect and provide advance warning of earthquakes and other environmental events). Lives and livelihoods depend on telecommunication systems.

Communication network complexity and the social cost of telecommunication failures will only increase (White and Tantsura, 2016). It is therefore opportune for telecommunication leaders and practitioners to advocate, apply, and extend the best telecommunication SE approaches to cope with this complexity and risk.

Unique Terms and Concepts Telecommunication systems are built on a wide range of technologies: satellite communication, cellular networks, land mobile radio, microwave, radio, television, Wi-Fi, Bluetooth, and global positioning systems. They are increasingly software-intensive systems (Donovan and Prabhu, 2017). Telecommunications includes communication networks owned by carriers, internet service providers, government agencies, and other enterprises, as well as broadcast networks (e.g., radio, cable, television) and over-the-top service provider applications (e.g., messaging, video conferencing, social media applications) (Adkins, et al., 2020) (Birman, 2012). Some telecommunication systems, like the internet and the public switched telephone network, have no single owner; their design depends upon collaboration in international telecommunication standards bodies.

The telecommunication transport network may be dedicated for a specific purpose (service or application) or shared for multiple services or applications. Communication networks may be employed for emergency services, defense, transportation, health, financial, industrial supervisory control, and data acquisition purposes. Many of these are considered to be national critical infrastructure (Lewis, 2019).

Telecommunication systems typically have some of the following characteristics:

- Diverse geographical distribution;
- Multi-party ownership and management (network domains or applications);
- Multiple constituent systems with independent life cycles that continuously evolve over many decades;
- A small stable set of functions, allocated across system elements, to achieve the common purpose of enabling communication;
- Many nodes and types of nodes; and
- Strong interdependence between nodes (failures within one node may cause other nodes to become isolated unless the specific failure mode is anticipated and the network is designed to withstand the failure).

Unique Activities, Methods, and Practices Network specifications, architectures, and models have a small number of functions and node types at their core, but they must also support many function and node variants. Engineering planning and architecture must be flexible enough to accommodate change in parts of the network owned and operated by others. Design and verification activities depend on a thorough analysis of failure modes and interactions across the network. Scale and variation lead to significant configuration management challenges (Xu and Zhou, 2015). These characteristics affect engineering activities throughout the life cycle. Like other types of networks, communication networks can be represented by a network model that defines a grouping of nodes and links to help understand how resources flow from one node to another.

While some telecommunication systems in slowly changing environments can survive on implicit engineering practices, such approaches have been found to be ineffective and inefficient. They typically fall short when one or more of the following exist:

- New and/or complex stakeholder needs;
- New operating models;
- Significant safety or security risks;
- High complexity; or
- Constrained, high-cost operating environments.

Other Unique Considerations Many vendor services and technologies are mature and are slow to evolve. Telecommunication networks typically comprise COTS vendor equipment/applications using industry interface standards and semi-standardized architectures (see Section 4.3.3). This equipment is typically integrated together without the use of SE. The promise from COTS vendors is that their equipment is suitable for rapid configuration and integration, and implicitly can survive with simpler requirements/business analysis and engineering processes, including outsourced vendor standardized engineering. This can lead to situations where there are unexpected outcomes. Without an appropriate set of engineering disciplines, such as those based on SE, it is difficult to assess, let alone manage, the risks**.**

4.4.10 Transportation Systems

Overview of SE Applications within the Transportation Domain Ground transportation systems, such as highways, busses, people-movers, mass transit, and rail involve complex capital programs for fleet acquisition and/or building of related infrastructures and are invariably within a SoS on an operational level. The system life cycle for ground transportation assets is 25 to 100+ years and often involves public funding and a related fiducial public trust. During this life cycle, operational processes are continuously optimized and improved using block changes.

The ground transportation industry segment is an emerging SE practice area, largely consisting of transit authorities, railroad operators rolling stock manufacturing industries, and civil engineering construction firms. Globally, some geographic regions are progressing the deployment and acceptance of SE more rapidly. They have created effective SoS approaches that are in concert with emergent societal trends such as smart cities, an embedded safety culture, and a through life approach to service delivery. For example, in the UK, the Institution of Civil Engineers has made solid progress in moving toward an SE approach within the civil engineering domain. Other regions are at the early stages of integrating SE into their ground transportation systems processes. These imbalances can be overcome in time through global collaboration.

Unique Considerations Additional mandates for the use of SE methods, skills, and competencies are generated from rapidly increasing system complexities in modern transportation systems, as evidenced by Intelligent Transportation System (ITS) concepts including regional, national, and local smart cities initiatives. Train control automation, bus

scheduling, and ride-share coordination, coupled with autonomous vehicles, micro-grid hybrid traction power, passenger fare collection, and related trip planner smartphone apps provide emerging complexities within modern ground transportation capability. However, public transportation must improve as public needs change, new technologies are introduced, or as environmental concerns and quality-of-life concerns encourage greater use of public transit. Ground transportation services are typically "in-service" brownfield systems and must migrate to an updated version, while still maintaining service, so that assets may be cost effectively managed in the public's interest (see Section 4.3.2). Some transit assets in large cities run 24/7 service as a matter of public safety and demand, requiring careful planning. A "whole" organization approach is needed for success.

Unique Terms and Concepts The transportation domain uses the same terms as the infrastructure domain, see Table 4.7.

Unique Activities, Methods, and Practices Some examples of early initiatives on the application of SE are in the United Kingdom, the European Union, and, specifically, the Netherlands. In the UK, Network Rail established the Governance for Railway Investment Projects (GRIP) to help manage projects and to align them with SE processes. Leading EU train operating companies and the main rolling stock manufacturers seek to develop a common, open architecture such as EULYNX and the Reference Command and Control System Architecture to support and optimize trackside and rolling stock acquisitions and upgrades. In the Netherlands, SE is advocated by the network management institution, ProRail, as part of their project management approach for delivering rail infrastructure projects. ProRail created a SE Handbook for their project life cycle approach which is comparable to GRIP.